

Aurumveil

Generated by Doxygen 1.17.0

1 Directory Hierarchy	1
1.1 Directories	1
2 Namespace Index	5
2.1 Namespace List	5
3 Hierarchical Index	7
3.1 Class Hierarchy	7
4 Class Index	9
4.1 Class List	9
5 File Index	11
5.1 File List	11
6 Directory Documentation	13
6.1 engine/compression Directory Reference	13
6.2 engine/engines/convex_hull Directory Reference	13
6.3 domain Directory Reference	14
6.4 engine Directory Reference	14
6.5 engine/engines Directory Reference	14
6.6 engine/foundation Directory Reference	14
6.7 interface Directory Reference	15
6.8 engine/engines/network_flow Directory Reference	15
6.9 engine/orchestration Directory Reference	15
6.10 engine/engines/range_queries Directory Reference	16
7 Namespace Documentation	17
7.1 content_panel Namespace Reference	17
7.2 control_panel Namespace Reference	17
7.2.1 Variable Documentation	17
7.2.1.1 MAXFLOW_NAMES	17
7.2.1.2 MINCOST_NAMES	18
7.3 data_dialog Namespace Reference	18
7.3.1 Variable Documentation	18
7.3.1.1 RESOURCES	18
7.4 execution_engine Namespace Reference	18
7.4.1 Function Documentation	19
7.4.1.1 get_executable_path()	19
7.4.1.2 run_cpp_algorithm()	19
7.4.2 Variable Documentation	20
7.4.2.1 EXECUTABLE_DIRECTORY	20
7.4.2.2 EXECUTABLES	20
7.4.2.3 PLATFORM_EXECUTABLE_SUFFIX	20

7.5 huffman_codec Namespace Reference	21
7.5.1 Function Documentation	21
7.5.1.1 build_codes()	21
7.5.1.2 build_tree()	22
7.5.1.3 compress_text()	22
7.5.1.4 decompress_text()	22
7.6 main Namespace Reference	23
7.6.1 Function Documentation	23
7.6.1.1 launch_application()	23
7.6.1.2 pause_terminal()	24
7.6.2 Variable Documentation	24
7.6.2.1 format	24
7.6.2.2 level	24
7.7 main_window Namespace Reference	24
7.8 models Namespace Reference	24
7.9 repository Namespace Reference	25
7.9.1 Function Documentation	25
7.9.1.1 _ensure_data_directories()	25
7.9.1.2 _validate_json_structure()	26
7.9.1.3 delete_data_file()	26
7.9.1.4 load_default_data()	26
7.9.1.5 load_from_file()	27
7.9.1.6 save_data_to_path()	27
7.9.1.7 save_default_data()	28
7.9.1.8 save_raw_data()	28
7.9.1.9 to_canonical_dict()	28
7.9.2 Variable Documentation	29
7.9.2.1 BASE_DIR	29
7.9.2.2 DATA_DIRECTORY	29
7.9.2.3 DEFAULT_DATA_FILE	29
7.9.2.4 RAW_DATA_DIRECTORY	29
7.10 runtime_initializer Namespace Reference	29
7.10.1 Function Documentation	30
7.10.1.1 build_application()	30
7.10.1.2 clear_terminal()	30
7.10.1.3 run_process()	30
7.10.2 Variable Documentation	31
7.10.2.1 format	31
7.10.2.2 level	31
7.10.2.3 PROCESS_FLAGS	31
7.11 serializer Namespace Reference	31
7.12 stylesheet Namespace Reference	31

7.12.1 Variable Documentation	32
7.12.1.1 APP_STYLESHEET	32
7.13 widget_effects Namespace Reference	32
7.13.1 Function Documentation	32
7.13.1.1 apply_shadow()	32
8 Class Documentation	33
8.1 BellmanFordStrategy Class Reference	33
8.1.1 Detailed Description	33
8.1.2 Member Function Documentation	34
8.1.2.1 findPath()	34
8.2 content_panel.ContentPanel Class Reference	34
8.2.1 Constructor & Destructor Documentation	36
8.2.1.1 __init__()	36
8.2.2 Member Function Documentation	37
8.2.2.1 _build_legend()	37
8.2.2.2 _build_toolbar()	37
8.2.2.3 _clear_data()	37
8.2.2.4 _clear_graph()	37
8.2.2.5 _configure_graph()	38
8.2.2.6 _draw_assignments()	38
8.2.2.7 _draw_convex_hull()	38
8.2.2.8 _draw_miners()	39
8.2.2.9 _draw_mines()	39
8.2.2.10 _draw_points()	39
8.2.2.11 _get_cached_result()	40
8.2.2.12 _initialize_ui()	40
8.2.2.13 _load_file()	40
8.2.2.14 _on_data_changed()	40
8.2.2.15 _open_data_dialog()	41
8.2.2.16 _open_logs_window()	41
8.2.2.17 _prepare_graph()	41
8.2.2.18 _refresh_visualization()	41
8.2.2.19 _save()	42
8.2.2.20 _save_as()	42
8.2.2.21 _save_existing_dataset()	42
8.2.2.22 _save_new_raw_dataset()	42
8.2.2.23 _show_save_error()	43
8.2.2.24 _toggle_data_labels()	43
8.2.2.25 _validate_dataset_for_save()	43
8.2.2.26 append_system_log()	44
8.2.2.27 is_cached_mode()	44

8.2.2.28 <code>redraw_graph()</code>	44
8.2.2.29 <code>set_highlighted_mine()</code>	45
8.2.3 Member Data Documentation	45
8.2.3.1 <code>btn_cached</code>	45
8.2.3.2 <code>btn_recompute</code>	45
8.2.3.3 <code>canvas</code>	45
8.2.3.4 <code>current_file_path</code>	45
8.2.3.5 <code>figure</code>	46
8.2.3.6 <code>highlighted_mine</code>	46
8.2.3.7 <code>log_index</code>	46
8.2.3.8 <code>mode_group</code>	46
8.2.3.9 <code>show_labels</code>	46
8.2.3.10 <code>stats</code>	46
8.2.3.11 <code>toggle_labels_button</code>	46
8.2.3.12 <code>toolbar</code>	47
8.3 <code>control_panel.ControlPanel</code> Class Reference	47
8.3.1 Detailed Description	50
8.3.2 Constructor & Destructor Documentation	50
8.3.2.1 <code>__init__()</code>	50
8.3.3 Member Function Documentation	51
8.3.3.1 <code>_branch_row()</code>	51
8.3.3.2 <code>_build_execution_signature()</code>	51
8.3.3.3 <code>_build_export_payload()</code>	52
8.3.3.4 <code>_build_ordered_payload_from_hull()</code>	52
8.3.3.5 <code>_calculate_total_execution_time()</code>	53
8.3.3.6 <code>_collect_execution_errors()</code>	53
8.3.3.7 <code>_execute_algorithms()</code>	53
8.3.3.8 <code>_execute_dwarf_search()</code>	54
8.3.3.9 <code>_export_compressed_payload()</code>	54
8.3.3.10 <code>_extract_dwarf_data()</code>	55
8.3.3.11 <code>_finalize_execution()</code>	55
8.3.3.12 <code>_get_content_panel()</code>	55
8.3.3.13 <code>_get_convex_hull_points()</code>	56
8.3.3.14 <code>_get_execution_count()</code>	56
8.3.3.15 <code>_get_result_file_path()</code>	56
8.3.3.16 <code>_get_selected()</code>	57
8.3.3.17 <code>_get_selected_algorithms()</code>	57
8.3.3.18 <code>_get_validated_dataset()</code>	57
8.3.3.19 <code>_handle_benchmark_results()</code>	58
8.3.3.20 <code>_handle_dwarf_search_error()</code>	58
8.3.3.21 <code>_handle_execution_errors()</code>	58
8.3.3.22 <code>_handle_execution_exception()</code>	59

8.3.3.23 _handle_execution_success()	59
8.3.3.24 _handle_multi_execution_results()	60
8.3.3.25 _handle_single_execution_results()	60
8.3.3.26 _highlight_loudest_dwarf()	60
8.3.3.27 _initialize_benchmark_metrics()	61
8.3.3.28 _initialize_execution_status()	61
8.3.3.29 _initialize_ui()	61
8.3.3.30 _load_file_cache()	62
8.3.3.31 _load_memory_cache()	62
8.3.3.32 _log_assignment_summary()	63
8.3.3.33 _log_boundary_points()	63
8.3.3.34 _log_boundary_segments()	63
8.3.3.35 _log_dwarf_result()	64
8.3.3.36 _log_execution_completion()	64
8.3.3.37 _log_flow_summary()	64
8.3.3.38 _log_geometry_summary()	65
8.3.3.39 _log_result_summary()	65
8.3.3.40 _parse_range_expression()	66
8.3.3.41 _reset_execution_state()	66
8.3.3.42 _run_benchmark_iterations()	66
8.3.3.43 _set_status()	67
8.3.3.44 _terminal_row()	67
8.3.3.45 _tree_prefix()	67
8.3.3.46 _update_benchmark_metrics()	68
8.3.3.47 _update_buttons_state()	68
8.3.3.48 _validate_dwarf_search()	68
8.3.3.49 on_find_loudest_dwarf()	69
8.3.3.50 on_start()	69
8.3.3.51 run_algorithm()	69
8.3.3.52 save_result()	69
8.3.4 Member Data Documentation	70
8.3.4.1 cache_hit	70
8.3.4.2 cached_result	70
8.3.4.3 dwarf_btn	70
8.3.4.4 execution_failed	70
8.3.4.5 execution_signature	70
8.3.4.6 geo_group	70
8.3.4.7 iterations_input	71
8.3.4.8 log_signal	71
8.3.4.9 maxflow_group	71
8.3.4.10 mincost_group	71
8.3.4.11 range_input	71

8.3.4.12 refresh_graph_signal	71
8.3.4.13 result_label	71
8.3.4.14 save_btn	72
8.3.4.15 seg_box	72
8.3.4.16 seg_group	72
8.3.4.17 start_btn	72
8.3.4.18 time_label	72
8.4 data_dialog.DataDialog Class Reference	73
8.4.1 Detailed Description	74
8.4.2 Constructor & Destructor Documentation	74
8.4.2.1 __init__()	74
8.4.3 Member Function Documentation	74
8.4.3.1 _add_mine()	74
8.4.3.2 _add_miner()	75
8.4.3.3 _build_identifier_item()	75
8.4.3.4 _build_table()	75
8.4.3.5 _clear_mark()	76
8.4.3.6 _delete_selected_mines()	76
8.4.3.7 _get_cell_text()	76
8.4.3.8 _initialize_ui()	77
8.4.3.9 _load_to_tables()	77
8.4.3.10 _mark_cell()	77
8.4.3.11 _refresh_identifiers()	78
8.4.3.12 _remove_selected_rows()	78
8.4.3.13 _safe_float()	78
8.4.3.14 _safe_int()	79
8.4.3.15 _safe_str()	79
8.4.3.16 _save()	79
8.4.3.17 _update_delete_buttons()	80
8.4.3.18 _validate()	80
8.4.3.19 get_data()	80
8.4.4 Member Data Documentation	81
8.4.4.1 data_changed	81
8.4.4.2 delete_mine_button	81
8.4.4.3 delete_miner_button	81
8.4.4.4 miners_table	81
8.4.4.5 mines_table	81
8.4.4.6 wardens_table	82
8.5 DEsopoPapeStrategy Class Reference	82
8.5.1 Detailed Description	82
8.5.2 Member Function Documentation	83
8.5.2.1 findPath()	83

8.6 models.Dwarf Class Reference	83
8.6.1 Detailed Description	83
8.7 Edge Struct Reference	84
8.7.1 Detailed Description	84
8.7.2 Member Data Documentation	84
8.7.2.1 capacity	84
8.7.2.2 cost	84
8.7.2.3 destination	84
8.7.2.4 reverseEdgeIndex	84
8.8 EdmondsKarpStrategy Class Reference	85
8.8.1 Detailed Description	85
8.8.2 Member Function Documentation	86
8.8.2.1 augment()	86
8.8.2.2 calculatePathLength()	86
8.8.2.3 computeFlow()	87
8.9 FlowOptimizer Class Reference	87
8.9.1 Detailed Description	88
8.9.2 Constructor & Destructor Documentation	88
8.9.2.1 FlowOptimizer()	88
8.9.3 Member Function Documentation	88
8.9.3.1 addEdge()	88
8.9.3.2 getGraph()	89
8.9.3.3 solve()	89
8.9.4 Member Data Documentation	90
8.9.4.1 flowNetwork	90
8.9.4.2 maxflowStrategy	90
8.9.4.3 mincostStrategy	90
8.10 FlowStatistics Struct Reference	90
8.10.1 Detailed Description	90
8.10.2 Member Data Documentation	91
8.10.2.1 augmentationCount	91
8.10.2.2 augmentationTimeMs	91
8.10.2.3 averagePathLength	91
8.10.2.4 maximumFlow	91
8.10.2.5 minimumCost	91
8.10.2.6 pathfindingTimeMs	91
8.10.2.7 totalPathLength	91
8.10.2.8 totalTimeMs	91
8.11 FordFulkersonStrategy Class Reference	92
8.11.1 Detailed Description	92
8.11.2 Member Function Documentation	92
8.11.2.1 augment()	92

8.11.2.2 computeFlow()	93
8.12 Huffman_codec.HuffmanNode Class Reference	94
8.12.1 Detailed Description	94
8.12.2 Constructor & Destructor Documentation	94
8.12.2.1 __init__()	94
8.12.3 Member Function Documentation	95
8.12.3.1 __lt__()	95
8.12.4 Member Data Documentation	95
8.12.4.1 left_branch	95
8.12.4.2 right_branch	95
8.12.4.3 symbol	95
8.12.4.4 weight	95
8.13 IMaxFlowStrategy Class Reference	96
8.13.1 Detailed Description	96
8.13.2 Constructor & Destructor Documentation	96
8.13.2.1 ~IMaxFlowStrategy()	96
8.13.3 Member Function Documentation	97
8.13.3.1 augment()	97
8.13.3.2 calculatePathLength()	97
8.13.3.3 computeFlow()	98
8.14 serializer.InputParser Class Reference	98
8.14.1 Detailed Description	98
8.14.2 Member Function Documentation	99
8.14.2.1 parse()	99
8.15 IShortestPathStrategy Class Reference	99
8.15.1 Detailed Description	100
8.15.2 Constructor & Destructor Documentation	100
8.15.2.1 ~IShortestPathStrategy()	100
8.15.3 Member Function Documentation	100
8.15.3.1 findPath()	100
8.16 main_window.MainWindow Class Reference	101
8.16.1 Detailed Description	101
8.16.2 Constructor & Destructor Documentation	101
8.16.2.1 __init__()	101
8.16.3 Member Function Documentation	102
8.16.3.1 _handle_log()	102
8.16.3.2 _initialize_ui()	102
8.16.3.3 resizeEvent()	102
8.16.4 Member Data Documentation	103
8.16.4.1 background_widget	103
8.16.4.2 content_panel	103
8.16.4.3 control_panel	103

8.17 Mine Struct Reference	103
8.17.1 Member Data Documentation	103
8.17.1.1 id	103
8.17.1.2 loudness	104
8.17.1.3 vertex	104
8.18 models.Mine Class Reference	104
8.18.1 Detailed Description	104
8.19 models.Miner Class Reference	104
8.19.1 Detailed Description	104
8.19.2 Member Data Documentation	105
8.19.2.1 resource	105
8.20 models.Point Class Reference	105
8.20.1 Detailed Description	105
8.21 Stopwatch Class Reference	105
8.21.1 Detailed Description	106
8.21.2 Constructor & Destructor Documentation	106
8.21.2.1 Stopwatch()	106
8.21.3 Member Function Documentation	106
8.21.3.1 elapsedMilliseconds()	106
8.21.3.2 restart()	106
8.21.4 Member Data Documentation	106
8.21.4.1 startTime	106
8.22 Vertex Struct Reference	107
8.22.1 Detailed Description	107
8.22.2 Member Data Documentation	107
8.22.2.1 id	107
8.22.2.2 x	107
8.22.2.3 y	107
8.23 models.Warden Class Reference	107
8.23.1 Detailed Description	108
8.24 models.WorldData Class Reference	108
8.24.1 Detailed Description	108
8.24.2 Member Function Documentation	108
8.24.2.1 to_dict()	108
8.24.3 Member Data Documentation	108
8.24.3.1 miners	108
8.24.3.2 mines	108
9 File Documentation	109
9.1 domain/models.py File Reference	109
9.1.1 Detailed Description	109
9.2 domain/repository.py File Reference	110

9.2.1 Detailed Description	110
9.3 domain/serializer.py File Reference	110
9.3.1 Detailed Description	111
9.4 engine/compression/huffman_codec.py File Reference	111
9.4.1 Detailed Description	111
9.5 engine/engines/convex_hull/convex_hull.hpp File Reference	112
9.5.1 Detailed Description	112
9.5.2 Function Documentation	112
9.5.2.1 computeConvexHullGraham()	112
9.5.2.2 computeConvexHullJarvis()	113
9.5.2.3 computeConvexHullMonotoneChain()	114
9.6 convex_hull.hpp	114
9.7 engine/engines/convex_hull/geometry_utils.cpp File Reference	115
9.7.1 Detailed Description	115
9.7.2 Function Documentation	115
9.7.2.1 comparePolar()	115
9.7.2.2 compareVertices()	116
9.7.2.3 crossProduct()	116
9.7.2.4 distanceSquared()	117
9.8 engine/engines/convex_hull/geometry_utils.hpp File Reference	117
9.8.1 Detailed Description	118
9.8.2 Function Documentation	118
9.8.2.1 comparePolar()	118
9.8.2.2 compareVertices()	119
9.8.2.3 crossProduct()	119
9.8.2.4 distanceSquared()	120
9.8.3 Variable Documentation	120
9.8.3.1 EPSILON	120
9.9 geometry_utils.hpp	120
9.10 engine/engines/convex_hull/graham_convex_hull.cpp File Reference	121
9.10.1 Detailed Description	121
9.10.2 Function Documentation	122
9.10.2.1 computeConvexHullGraham()	122
9.10.2.2 main()	122
9.11 engine/engines/convex_hull/jarvis_convex_hull.cpp File Reference	123
9.11.1 Detailed Description	123
9.11.2 Function Documentation	123
9.11.2.1 computeConvexHullJarvis()	123
9.11.2.2 main()	124
9.12 engine/engines/convex_hull/monotone_convex_hull.cpp File Reference	124
9.12.1 Detailed Description	125
9.12.2 Function Documentation	125

9.12.2.1 computeConvexHullMonotoneChain()	125
9.12.2.2 main()	126
9.13 engine/engines/network_flow/bellman_ford_strategy.hpp File Reference	126
9.13.1 Detailed Description	126
9.14 bellman_ford_strategy.hpp	127
9.15 engine/engines/network_flow/desopopape_strategy.hpp File Reference	128
9.16 desopopape_strategy.hpp	129
9.17 engine/engines/network_flow/edmonds_karp_strategy.hpp File Reference	130
9.17.1 Detailed Description	130
9.18 edmonds_karp_strategy.hpp	131
9.19 engine/engines/network_flow/flow_network.hpp File Reference	132
9.19.1 Detailed Description	132
9.19.2 Typedef Documentation	133
9.19.2.1 FlowNetwork	133
9.19.3 Variable Documentation	133
9.19.3.1 DISTANCE_INFINITY	133
9.19.3.2 INVALID_EDGE	133
9.20 flow_network.hpp	133
9.21 engine/engines/network_flow/flow_optimizer.cpp File Reference	134
9.21.1 Detailed Description	134
9.21.2 Typedef Documentation	134
9.21.2.1 json	134
9.21.3 Function Documentation	134
9.21.3.1 main()	134
9.22 engine/engines/network_flow/flow_optimizer.hpp File Reference	135
9.22.1 Detailed Description	135
9.23 flow_optimizer.hpp	136
9.24 engine/engines/network_flow/ford_fulkerson_strategy.hpp File Reference	138
9.24.1 Detailed Description	138
9.25 ford_fulkerson_strategy.hpp	138
9.26 engine/engines/network_flow/imaxflow_path_factory.hpp File Reference	139
9.26.1 Detailed Description	140
9.26.2 Function Documentation	140
9.26.2.1 createMaxFlowStrategy()	140
9.27 imaxflow_path_factory.hpp	141
9.28 engine/engines/network_flow/imaxflow_path_strategy.hpp File Reference	141
9.28.1 Detailed Description	141
9.29 imaxflow_path_strategy.hpp	142
9.30 engine/engines/network_flow/ishortest_path_factory.hpp File Reference	142
9.30.1 Detailed Description	143
9.30.2 Function Documentation	143
9.30.2.1 createShortestPathStrategy()	143

9.31 ishortest_path_factory.hpp	144
9.32 engine/engines/network_flow/ishortest_path_strategy.hpp File Reference	144
9.32.1 Detailed Description	144
9.33 ishortest_path_strategy.hpp	145
9.34 engine/engines/range_queries/brute_force.cpp File Reference	145
9.34.1 Detailed Description	145
9.34.2 Function Documentation	146
9.34.2.1 findLoudest()	146
9.34.2.2 main()	146
9.35 engine/engines/range_queries/range_utils.hpp File Reference	147
9.35.1 Function Documentation	147
9.35.1.1 loudnessMax()	147
9.35.2 Variable Documentation	148
9.35.2.1 neutralMine	148
9.36 range_utils.hpp	148
9.37 engine/engines/range_queries/segment_tree.cpp File Reference	149
9.37.1 Detailed Description	149
9.37.2 Function Documentation	149
9.37.2.1 buildTree()	149
9.37.2.2 main()	150
9.37.2.3 queryModuloRange()	150
9.37.2.4 queryRange()	151
9.38 engine/foundation/core_utils_json.hpp File Reference	151
9.38.1 Function Documentation	152
9.38.1.1 buildAssignments()	152
9.38.1.2 buildConvexOutput()	153
9.38.1.3 buildFlowOutput()	153
9.38.1.4 buildMineOutput()	154
9.38.1.5 extractAlgorithms()	154
9.38.1.6 extractMines()	155
9.38.1.7 extractPoints()	155
9.38.1.8 extractRange()	155
9.38.1.9 readInput()	156
9.39 core_utils_json.hpp	156
9.40 engine/foundation/core_utils_stopwatch.hpp File Reference	159
9.41 core_utils_stopwatch.hpp	159
9.42 engine/orchestration/execution_engine.py File Reference	160
9.42.1 Detailed Description	160
9.43 engine/orchestration/runtime_initializer.py File Reference	160
9.43.1 Detailed Description	161
9.44 interface/content_panel.py File Reference	161
9.44.1 Detailed Description	161

9.45 interface/control_panel.py File Reference	162
9.45.1 Detailed Description	162
9.46 interface/data_dialog.py File Reference	162
9.46.1 Detailed Description	163
9.47 interface/main_window.py File Reference	163
9.47.1 Detailed Description	163
9.48 interface/widget_effects.py File Reference	163
9.48.1 Detailed Description	163
9.49 main.py File Reference	164
9.49.1 Detailed Description	164
9.50 stylesheet.py File Reference	164
9.50.1 Detailed Description	164
Index	165

Chapter 1

Directory Hierarchy

1.1 Directories

compression	13
huffman_codec.py	111
convex_hull	13
convex_hull.hpp	112
geometry_utils.cpp	115
geometry_utils.hpp	117
graham_convex_hull.cpp	121
jarvis_convex_hull.cpp	123
monotone_convex_hull.cpp	124
domain	14
models.py	109
repository.py	110
serializer.py	110
engine	14
compression	13
huffman_codec.py	111
engines	14
convex_hull	13
convex_hull.hpp	112
geometry_utils.cpp	115
geometry_utils.hpp	117
graham_convex_hull.cpp	121
jarvis_convex_hull.cpp	123
monotone_convex_hull.cpp	124
network_flow	15
bellman_ford_strategy.hpp	126
desopopape_strategy.hpp	128
edmonds_karp_strategy.hpp	130
flow_network.hpp	132
flow_optimizer.cpp	134
flow_optimizer.hpp	135
ford_fulkerson_strategy.hpp	138
imaxflow_path_factory.hpp	139
imaxflow_path_strategy.hpp	141
ishortest_path_factory.hpp	142
ishortest_path_strategy.hpp	144
range_queries	16

brute_force.cpp	145
range_utils.hpp	147
segment_tree.cpp	149
foundation	14
core_utils_json.hpp	151
core_utils_stopwatch.hpp	159
orchestration	15
execution_engine.py	160
runtime_initializer.py	160
engines	14
convex_hull	13
convex_hull.hpp	112
geometry_utils.cpp	115
geometry_utils.hpp	117
graham_convex_hull.cpp	121
jarvis_convex_hull.cpp	123
monotone_convex_hull.cpp	124
network_flow	15
bellman_ford_strategy.hpp	126
desopopape_strategy.hpp	128
edmonds_karp_strategy.hpp	130
flow_network.hpp	132
flow_optimizer.cpp	134
flow_optimizer.hpp	135
ford_fulkerson_strategy.hpp	138
imaxflow_path_factory.hpp	139
imaxflow_path_strategy.hpp	141
ishortest_path_factory.hpp	142
ishortest_path_strategy.hpp	144
range_queries	16
brute_force.cpp	145
range_utils.hpp	147
segment_tree.cpp	149
foundation	14
core_utils_json.hpp	151
core_utils_stopwatch.hpp	159
interface	15
content_panel.py	161
control_panel.py	162
data_dialog.py	162
main_window.py	163
widget_effects.py	163
network_flow	15
bellman_ford_strategy.hpp	126
desopopape_strategy.hpp	128
edmonds_karp_strategy.hpp	130
flow_network.hpp	132
flow_optimizer.cpp	134
flow_optimizer.hpp	135
ford_fulkerson_strategy.hpp	138
imaxflow_path_factory.hpp	139
imaxflow_path_strategy.hpp	141
ishortest_path_factory.hpp	142
ishortest_path_strategy.hpp	144
orchestration	15
execution_engine.py	160

runtime_initializer.py	160
range_queries	16
brute_force.cpp	145
range_utils.hpp	147
segment_tree.cpp	149

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all namespaces with brief descriptions:

content_panel	17
control_panel	17
data_dialog	18
execution_engine	18
huffman_codec	21
main	23
main_window	24
models	24
repository	25
runtime_initializer	29
serializer	31
stylesheet	31
widget_effects	32

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

models.Dwarf	83
models.Miner	104
models.Warden	107
Edge	84
FlowOptimizer	87
FlowStatistics	90
huffman_codec.HuffmanNode	94
IMaxFlowStrategy	96
EdmondsKarpStrategy	85
FordFulkersonStrategy	92
serializer.InputParser	98
IShortestPathStrategy	99
BellmanFordStrategy	33
DEsopoPapeStrategy	82
Mine	103
models.Mine	104
models.Point	105
QDialog	
data_dialog.DataDialog	73
QWidget	
content_panel.ContentPanel	34
control_panel.ControlPanel	47
main_window.MainWindow	101
Stopwatch	105
Vertex	107
models.WorldData	108

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BellmanFordStrategy	
Bellman-Ford shortest-path strategy	33
content_panel.ContentPanel	34
control_panel.ControlPanel	
Analysis control panel	47
data_dialog.DataDialog	
World data management dialogue	73
DEsopoPapeStrategy	
D'Esopo-Pape shortest-path strategy	82
models.Dwarf	
Base dwarf entity	83
Edge	
Residual network edge	84
EdmondsKarpStrategy	
Edmonds-Karp flow augmentation strategy	85
FlowOptimizer	
Minimum-Cost Maximum-Flow optimizer	87
FlowStatistics	
Flow optimization statistics	90
FordFulkersonStrategy	
Ford-Fulkerson flow augmentation strategy	92
huffman_codec.HuffmanNode	
Node of a Huffman coding tree	94
IMaxFlowStrategy	
Maximum-flow strategy interface	96
serializer.InputParser	
World data text parser	98
IShortestPathStrategy	
Shortest-path strategy interface	99
main_window.MainWindow	
Main application window	101
Mine	103
models.Mine	
Mining site	104
models.Miner	
Mining unit	104

models.Point	
Two-dimensional point	105
Stopwatch	
High-resolution stopwatch	105
Vertex	
Two-dimensional geometric point	107
models.Warden	
Defensive unit	107
models.WorldData	
World state container	108

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

main.py	Application entry point for the Aurumveil platform	164
stylesheet.py	Global Qt stylesheet for the Aurumveil application	164
domain/models.py	Domain models used by the Aurumveil platform	109
domain/repository.py	Persistent storage utilities for world data	110
domain/serializer.py	Text parser for world data definitions	110
engine/compression/huffman_codec.py	Huffman-based text compression and decompression	111
engine/engines/convex_hull/convex_hull.hpp	Convex hull algorithm interfaces	112
engine/engines/convex_hull/geometry_utils.cpp	Implementation of geometric utility functions	115
engine/engines/convex_hull/geometry_utils.hpp	Geometric primitives and utility functions	117
engine/engines/convex_hull/graham_convex_hull.cpp	Graham Scan convex hull algorithm	121
engine/engines/convex_hull/jarvis_convex_hull.cpp	Jarvis March convex hull algorithm	123
engine/engines/convex_hull/monotone_convex_hull.cpp	Monotone Chain convex hull algorithm	124
engine/engines/network_flow/bellman_ford_strategy.hpp	Bellman-Ford shortest-path strategy	126
engine/engines/network_flow/desopopape_strategy.hpp		128
engine/engines/network_flow/edmonds_karp_strategy.hpp	Edmonds-Karp maximum-flow strategy	130
engine/engines/network_flow/flow_network.hpp	Residual flow network data structures	132
engine/engines/network_flow/flow_optimizer.cpp	Minimum-Cost Maximum-Flow application	134
engine/engines/network_flow/flow_optimizer.hpp	Minimum-Cost Maximum-Flow optimization engine	135

engine/engines/network_flow/ ford_fulkerson_strategy.hpp	
Ford-Fulkerson maximum-flow strategy	138
engine/engines/network_flow/ imaxflow_path_factory.hpp	
Maximum-flow strategy factory	139
engine/engines/network_flow/ imaxflow_path_strategy.hpp	
Maximum-flow strategy interface	141
engine/engines/network_flow/ ishortest_path_factory.hpp	
Shortest-path strategy factory	142
engine/engines/network_flow/ ishortest_path_strategy.hpp	
Shortest-path strategy interface	144
engine/engines/range_queries/ brute_force.cpp	
Brute-force range query algorithm	145
engine/engines/range_queries/ range_utils.hpp	
Range query utilities	147
engine/engines/range_queries/ segment_tree.cpp	
Segment tree range query algorithm	149
engine/foundation/ core_utils_json.hpp	
Core utilities for JSON	151
engine/foundation/ core_utils_stopwatch.hpp	
Core utilities for stopwatch	159
engine/orchestration/ execution_engine.py	
Native algorithm execution utilities	160
engine/orchestration/ runtime_initializer.py	
Runtime initialization and build utilities	160
interface/ content_panel.py	
Visualization and dataset management panel	161
interface/ control_panel.py	
Analysis and execution control panel	162
interface/ data_dialog.py	
Data management dialogue for the Aurumveil platform	162
interface/ main_window.py	
Main application window for the Aurumveil platform	163
interface/ widget_effects.py	
Utility functions for Qt widget visual effects	163

Chapter 6

Directory Documentation

6.1 engine/compression Directory Reference

Directory dependency graph for compression:

Files

- file [huffman_codec.py](#)
Huffman-based text compression and decompression.

6.2 engine/engines/convex_hull Directory Reference

Directory dependency graph for convex_hull:

Files

- file [convex_hull.hpp](#)
Convex hull algorithm interfaces.
- file [geometry_utils.cpp](#)
Implementation of geometric utility functions.
- file [geometry_utils.hpp](#)
Geometric primitives and utility functions.
- file [graham_convex_hull.cpp](#)
Graham Scan convex hull algorithm.
- file [jarvis_convex_hull.cpp](#)
Jarvis March convex hull algorithm.
- file [monotone_convex_hull.cpp](#)
Monotone Chain convex hull algorithm.

6.3 domain Directory Reference

Files

- file [models.py](#)
Domain models used by the Aurumveil platform.
- file [repository.py](#)
Persistent storage utilities for world data.
- file [serializer.py](#)
Text parser for world data definitions.

6.4 engine Directory Reference

Directory dependency graph for engine:

Directories

- directory [compression](#)
- directory [engines](#)
- directory [foundation](#)
- directory [orchestration](#)

6.5 engine/engines Directory Reference

Directory dependency graph for engines:

Directories

- directory [convex_hull](#)
- directory [network_flow](#)
- directory [range_queries](#)

6.6 engine/foundation Directory Reference

Directory dependency graph for foundation:

Files

- file [core_utils_json.hpp](#)
- file [core_utils_stopwatch.hpp](#)

6.7 interface Directory Reference

Files

- file [content_panel.py](#)
Visualization and dataset management panel.
- file [control_panel.py](#)
Analysis and execution control panel.
- file [data_dialog.py](#)
Data management dialogue for the Aurumveil platform.
- file [main_window.py](#)
Main application window for the Aurumveil platform.
- file [widget_effects.py](#)
Utility functions for Qt widget visual effects.

6.8 engine/engines/network_flow Directory Reference

Directory dependency graph for network_flow:

Files

- file [bellman_ford_strategy.hpp](#)
Bellman-Ford shortest-path strategy.
- file [desopopape_strategy.hpp](#)
- file [edmonds_karp_strategy.hpp](#)
Edmonds-Karp maximum-flow strategy.
- file [flow_network.hpp](#)
Residual flow network data structures.
- file [flow_optimizer.cpp](#)
Minimum-Cost Maximum-Flow application.
- file [flow_optimizer.hpp](#)
Minimum-Cost Maximum-Flow optimization engine.
- file [ford_fulkerson_strategy.hpp](#)
Ford-Fulkerson maximum-flow strategy.
- file [imaxflow_path_factory.hpp](#)
Maximum-flow strategy factory.
- file [imaxflow_path_strategy.hpp](#)
Maximum-flow strategy interface.
- file [ishortest_path_factory.hpp](#)
Shortest-path strategy factory.
- file [ishortest_path_strategy.hpp](#)
Shortest-path strategy interface.

6.9 engine/orchestration Directory Reference

Directory dependency graph for orchestration:

Files

- file [execution_engine.py](#)
Native algorithm execution utilities.
- file [runtime_initializer.py](#)
Runtime initialization and build utilities.

6.10 engine/engines/range_queries Directory Reference

Directory dependency graph for range_queries:

Files

- file [brute_force.cpp](#)
Brute-force range query algorithm.
- file [range_utils.hpp](#)
- file [segment_tree.cpp](#)
Segment tree range query algorithm.

Chapter 7

Namespace Documentation

7.1 content_panel Namespace Reference

Classes

- class [ContentPanel](#)

7.2 control_panel Namespace Reference

Classes

- class [ControlPanel](#)
Analysis control panel.

Variables

- dict [MAXFLOW_NAMES](#)
- dict [MINCOST_NAMES](#)

7.2.1 Variable Documentation

7.2.1.1 MAXFLOW_NAMES

dict control_panel.MAXFLOW_NAMES

Initial value:

```
= {  
    "Ford-Fulkerson Method": (  
        "Ford-Fulkerson"  
    ),  
    "Edmonds-Karp Method": (  
        "Edmonds-Karp"  
    ),  
}
```

7.2.1.2 MINCOST_NAMES

```
dict control_panel.MINCOST_NAMES
```

Initial value:

```
= {
    "Bellman-Ford Method": (
        "Bellman-Ford"
    ),
    "d'Esopo-Pape Method": (
        "D'Esopo-Pape"
    ),
}
```

7.3 data_dialog Namespace Reference

Classes

- class [DataDialog](#)
World data management dialogue.

Variables

- list [RESOURCES](#)

7.3.1 Variable Documentation

7.3.1.1 RESOURCES

```
list data_dialog.RESOURCES
```

Initial value:

```
= [
    "Gold",
    "Coal",
    "Copper",
    "Iron",
    "Silver",
    "Diamond",
]
```

7.4 execution_engine Namespace Reference

Functions

- Path [get_executable_path](#) (str executable_name)
Resolves an executable path.
- dict[str, object] [run_cpp_algorithm](#) (str algorithm_name, WorldData|dict[str, object] input_data, tuple[int, int]↔|None value_range=None)
Executes a native algorithm executable.

Variables

- tuple [EXECUTABLE_DIRECTORY](#)
Directory containing compiled executables.
- tuple [PLATFORM_EXECUTABLE_SUFFIX](#)
Platform-specific executable extension.
- dict [EXECUTABLES](#)
Registry of available native algorithms.

7.4.1 Function Documentation

7.4.1.1 get_executable_path()

```
Path get_executable_path (
    str executable_name)
```

Resolves an executable path.

Constructs an absolute path to a native algorithm executable located in the build output directory.

Parameters

<i>executable_name</i>	Base name of the executable.
------------------------	------------------------------

Returns

Path Absolute executable path.

7.4.1.2 run_cpp_algorithm()

```
dict[str, object] run_cpp_algorithm (
    str algorithm_name,
    WorldData | dict[str, object] input_data,
    tuple[int, int] | None value_range = None)
```

Executes a native algorithm executable.

Serializes the provided input data, invokes the selected C++ executable, captures its output, and converts the resulting JSON response into a Python object.

Input data is transferred through a temporary JSON file created for the duration of the execution.

Parameters

<i>algorithm_name</i>	Display name of the algorithm to execute.
<i>input_data</i>	World model or serialized payload supplied to the executable.
<i>value_range</i>	Optional range query interval used by range-processing algorithms.

Returns

dict[str, object] Parsed executable response or error information.

7.4.2 Variable Documentation

7.4.2.1 EXECUTABLE_DIRECTORY

`tuple execution_engine.EXECUTABLE_DIRECTORY`

Initial value:

```
= (
    Path(__file__).parent.parent
    / "build"
    / "bin"
)
```

Directory containing compiled executables.

Stores native algorithm binaries generated by CMake.

7.4.2.2 EXECUTABLES

`dict execution_engine.EXECUTABLES`

Initial value:

```
= {

    # ===== MINIMUM COST MAXIMUM FLOW =====
    "Min-Cost Max-Flow Method": get_executable_path("mcmf"),

    # ===== COMPUTATIONAL GEOMETRY =====
    "Graham Scan Method": get_executable_path("graham"),
    "Jarvis March Method": get_executable_path("jarvis"),
    "Monotone Chain Method": get_executable_path("monotonic"),

    # ===== RANGE QUERY STRUCTURES =====
    "Brute Force Method": get_executable_path("brute"),
    "Segment Tree Method": get_executable_path("segment"),

}
```

Registry of available native algorithms.

Maps user-visible algorithm names to executable files generated during the native build process.

7.4.2.3 PLATFORM_EXECUTABLE_SUFFIX

`tuple execution_engine.PLATFORM_EXECUTABLE_SUFFIX`

Initial value:

```
= (
    ".exe"
    if sys.platform.startswith("win")
    else ""
)
```

Platform-specific executable extension.

Determines the executable suffix required by the current operating system.

7.5 huffman_codec Namespace Reference

Classes

- class [HuffmanNode](#)
Node of a Huffman coding tree.

Functions

- [HuffmanNode build_tree](#) (str text)
Constructs a Huffman tree.
- dict[str, str] [build_codes](#) ([HuffmanNode](#) node, str prefix="", dict[str, str]|None codebook=None)
Generates Huffman codes from a tree.
- bytes [compress_text](#) (str text)
Compresses text using Huffman coding.
- str [decompress_text](#) (bytes data)
Decompresses Huffman-encoded data.

7.5.1 Function Documentation

7.5.1.1 build_codes()

```
dict[str, str] build_codes (  
    HuffmanNode node,  
    str prefix = "",  
    dict[str, str] | None codebook = None)
```

Generates Huffman codes from a tree.

Traverses the Huffman tree and assigns binary prefix codes to all symbols.

Parameters

<i>node</i>	Current Huffman tree node.
<i>prefix</i>	Binary prefix accumulated during traversal.
<i>codebook</i>	Code mapping being constructed.

Returns

dict[str, str] Generated symbol-to-code mapping.

Here is the call graph for this function: Here is the caller graph for this function:

7.5.1.2 build_tree()

```
HuffmanNode build_tree (  
    str text)
```

Constructs a Huffman tree.

Builds an optimal prefix tree based on symbol frequencies extracted from the input text.

Parameters

<i>text</i>	Source text used to build the tree.
-------------	-------------------------------------

Returns

[HuffmanNode](#) Root node of the constructed Huffman tree.

Here is the caller graph for this function:

7.5.1.3 compress_text()

```
bytes compress_text (  
    str text)
```

Compresses text using Huffman coding.

Generates a Huffman tree, encodes the input text, and serializes the compressed representation.

Parameters

<i>text</i>	Text to compress.
-------------	-------------------

Returns

bytes Serialized compressed payload.

Here is the call graph for this function:

7.5.1.4 decompress_text()

```
str decompress_text (  
    bytes data)
```

Decompresses Huffman-encoded data.

Restores the original text from a serialized Huffman-compressed payload.

Parameters

<i>data</i>	Serialized compressed data.
-------------	-----------------------------

Returns

str Decompressed text.

7.6 main Namespace Reference

Functions

- None [pause_terminal](#) ()
Pauses the terminal before application shutdown.
- int [launch_application](#) ()
Builds and launches the Aurumveil application.

Variables

- [level](#)
- [format](#)

7.6.1 Function Documentation

7.6.1.1 [launch_application\(\)](#)

```
int launch_application ()
```

Builds and launches the Aurumveil application.

Executes the runtime initialization process, configures the Qt application environment, applies the global stylesheet, creates the main window, and starts the application event loop.

Returns

int Exit code returned by the Qt event loop.

Exceptions

<i>Exception</i>	Re-raises any exception encountered during the build phase.
------------------	---

Here is the call graph for this function: Here is the caller graph for this function:

7.6.1.2 pause_terminal()

```
None pause_terminal ()
```

Pauses the terminal before application shutdown.

Prevents the console window from closing immediately after an initialization failure, allowing the user to review the reported error message.

Returns

None

Here is the caller graph for this function:

7.6.2 Variable Documentation

7.6.2.1 format

```
main.format
```

7.6.2.2 level

```
main.level
```

7.7 main_window Namespace Reference

Classes

- class [MainWindow](#)
Main application window.

7.8 models Namespace Reference

Classes

- class [Point](#)
Two-dimensional point.
- class [Dwarf](#)
Base dwarf entity.
- class [Miner](#)
Mining unit.
- class [Warden](#)
Defensive unit.
- class [Mine](#)
Mining site.
- class [WorldData](#)
World state container.

7.9 repository Namespace Reference

Functions

- None [_ensure_data_directories](#) ()
- None [_validate_json_structure](#) (dict json_data)
Validates the structure of serialized world data.
- WorldData|None [load_default_data](#) ()
Loads the default world state.
- WorldData [load_from_file](#) (str file_path)
Loads world data from a compressed file.
- None [save_default_data](#) (WorldData world_data)
Saves the world state to the default location.
- None [save_data_to_path](#) (WorldData world_data, str file_path)
Saves world data to a compressed file.
- str [save_raw_data](#) (WorldData world_data)
Saves a dataset snapshot to the dataset repository.
- dict[str, list[dict[str, object]]] [to_canonical_dict](#) (WorldData world_data)
Creates a canonical representation of world data.
- None [delete_data_file](#) ()
Removes the default runtime state file.

Variables

- tuple [BASE_DIR](#)
- tuple [DATA_DIRECTORY](#)
- tuple [DEFAULT_DATA_FILE](#)
Default runtime state file.
- tuple [RAW_DATA_DIRECTORY](#)
Directory containing importable datasets.

7.9.1 Function Documentation

7.9.1.1 _ensure_data_directories()

None [_ensure_data_directories](#) () [protected]

Here is the caller graph for this function:

7.9.1.2 `_validate_json_structure()`

```
None _validate_json_structure (
    dict json_data) [protected]
```

Validates the structure of serialized world data.

Verifies that the provided JSON object contains all mandatory top-level sections required by the application.

Parameters

<code>json_data</code>	JSON object to validate.
------------------------	--------------------------

Exceptions

<code>ValueError</code>	Raised when the structure is invalid.
-------------------------	---------------------------------------

Here is the caller graph for this function:

7.9.1.3 `delete_data_file()`

```
None delete_data_file ()
```

Removes the default runtime state file.

Deletes the persisted runtime state when it exists, allowing the application to start with an empty world. Here is the call graph for this function:

7.9.1.4 `load_default_data()`

```
WorldData | None load_default_data ()
```

Loads the default world state.

Loads and reconstructs the world model stored in the default runtime state file.

Returns

WorldData | None Loaded world data or None if no state file exists.

Here is the call graph for this function:

7.9.1.5 load_from_file()

```
WorldData load_from_file (  
    str file_path)
```

Loads world data from a compressed file.

Reads a Huffman-compressed data file, decompresses its contents, validates the resulting JSON structure, and reconstructs the corresponding domain model.

Parameters

<i>file_path</i>	Path to the compressed data file.
------------------	-----------------------------------

Returns

WorldData Reconstructed world model.

Exceptions

<i>ValueError</i>	Raised when the file contains invalid data.
-------------------	---

Here is the call graph for this function: Here is the caller graph for this function:

7.9.1.6 save_data_to_path()

```
None save_data_to_path (  
    WorldData world_data,  
    str file_path)
```

Saves world data to a compressed file.

Serializes the provided world model into JSON, compresses the result using Huffman coding, and writes the compressed data to disk.

Parameters

<i>world_data</i>	World model to serialize.
<i>file_path</i>	Destination file path.

Here is the call graph for this function: Here is the caller graph for this function:

7.9.1.7 save_default_data()

```
None save_default_data (
    WorldData world_data)
```

Saves the world state to the default location.

Serializes and persists the specified world model using the default runtime state file.

Parameters

<i>world_data</i>	World model to persist.
-------------------	-------------------------

Here is the call graph for this function:

7.9.1.8 save_raw_data()

```
str save_raw_data (
    WorldData world_data)
```

Saves a dataset snapshot to the dataset repository.

Serializes the specified world model, generates a content-based hash, and stores the compressed dataset if an identical snapshot does not already exist.

Parameters

<i>world_data</i>	World model to persist.
-------------------	-------------------------

Returns

str Generated dataset filename.

Here is the call graph for this function:

7.9.1.9 to_canonical_dict()

```
dict[str, list[dict[str, object]]] to_canonical_dict (
    WorldData world_data)
```

Creates a canonical representation of world data.

Produces a deterministic dictionary representation independent of entity ordering. The resulting structure is suitable for hashing, comparisons, and duplicate dataset detection.

Parameters

<i>world_data</i>	World model to normalize.
-------------------	---------------------------

Returns

dict[str, list[dict[str, object]]] Canonical world data representation.

7.9.2 Variable Documentation

7.9.2.1 BASE_DIR

```
tuple repository.BASE_DIR
```

Initial value:

```
= (
    Path(__file__).resolve().parent.parent
)
```

7.9.2.2 DATA_DIRECTORY

```
tuple repository.DATA_DIRECTORY
```

Initial value:

```
= (
    BASE_DIR / "assets"
)
```

7.9.2.3 DEFAULT_DATA_FILE

```
tuple repository.DEFAULT_DATA_FILE
```

Initial value:

```
= (
    DATA_DIRECTORY / "runtime_state.huff"
)
```

Default runtime state file.

7.9.2.4 RAW_DATA_DIRECTORY

```
tuple repository.RAW_DATA_DIRECTORY
```

Initial value:

```
= (
    DATA_DIRECTORY / "datasets"
)
```

Directory containing importable datasets.

7.10 runtime_initializer Namespace Reference

Functions

- None [run_process](#) (list[str] command, Path working_directory)
Executes an external process and streams its output.
- None [clear_terminal](#) ()
Clears the terminal window.
- None [build_application](#) ()
Configures and builds the native engine.

Variables

- [level](#)
- [format](#)
- tuple [PROCESS_FLAGS](#)

Platform-specific process creation flags.

7.10.1 Function Documentation

7.10.1.1 `build_application()`

```
None build_application ()
```

Configures and builds the native engine.

Creates the build directory when necessary, executes the CMake configuration step, compiles the native C++ components, and clears the terminal after a successful build.

Exceptions

<code>subprocess.CalledProcessError</code>	Raised when the configuration or compilation process fails.
--	---

Here is the call graph for this function:

7.10.1.2 `clear_terminal()`

```
None clear_terminal ()
```

Clears the terminal window.

Invokes the operating system specific command used to clear the current console session. Here is the caller graph for this function:

7.10.1.3 `run_process()`

```
None run_process (
    list[str] command,
    Path working_directory)
```

Executes an external process and streams its output.

Launches a subprocess, forwards its output to the application logger, and raises an exception when the process terminates with an error.

Parameters

<code>command</code>	Command and arguments passed to the subprocess.
----------------------	---

<code>working_directory</code>	Working directory used during execution.
--------------------------------	--

Exceptions

<code>subprocess.CalledProcessError</code>	Raised when the process exits with a non-zero code.
--	---

Here is the caller graph for this function:

7.10.2 Variable Documentation**7.10.2.1 format**

```
runtime_initializer.format
```

7.10.2.2 level

```
runtime_initializer.level
```

7.10.2.3 PROCESS_FLAGS

```
tuple runtime_initializer.PROCESS_FLAGS
```

Initial value:

```
= (
    CREATE_NO_WINDOW
    if os.name == "nt"
    else 0
)
```

Platform-specific process creation flags.

Prevents console windows from being displayed on Microsoft Windows during subprocess execution.

7.11 serializer Namespace Reference**Classes**

- class [InputParser](#)
World data text parser.

7.12 stylesheet Namespace Reference**Variables**

- str [APP_STYLESHEET](#)
Global application stylesheet.

7.12.1 Variable Documentation

7.12.1.1 APP_STYLESHEET

```
str stylesheet.APP_STYLESHEET
```

Global application stylesheet.

Contains the complete Qt Style Sheet (QSS) definition used by the graphical user interface.

7.13 widget_effects Namespace Reference

Functions

- None [apply_shadow](#) (QWidget widget)
Applies a drop shadow effect to a widget.

7.13.1 Function Documentation

7.13.1.1 apply_shadow()

```
None apply_shadow (  
    QWidget widget)
```

Applies a drop shadow effect to a widget.

Creates and configures a QGraphicsDropShadowEffect instance and attaches it to the specified widget to improve visual depth and separation from the background.

Parameters

<i>widget</i>	Target widget that will receive the shadow effect.
---------------	--

Returns

None

Chapter 8

Class Documentation

8.1 BellmanFordStrategy Class Reference

Bellman-Ford shortest-path strategy.

```
#include <bellman_ford_strategy.hpp>
```

Inheritance diagram for BellmanFordStrategy:

Collaboration diagram for BellmanFordStrategy:

Public Member Functions

- bool [findPath](#) ([FlowNetwork](#) &graph, int source, int sink, vector< int > &parentVertex, vector< size_t > &parentEdge, vector< int > &distance) override

Finds a minimum-cost augmenting path.

Public Member Functions inherited from [IShortestPathStrategy](#)

- virtual [~IShortestPathStrategy](#) ()=default

8.1.1 Detailed Description

Bellman-Ford shortest-path strategy.

Computes minimum-cost augmenting paths using repeated edge relaxation and supports residual networks containing negative-cost edges.

This strategy is used during Minimum-Cost Maximum-Flow optimization.

8.1.2 Member Function Documentation

8.1.2.1 findPath()

```
bool BellmanFordStrategy::findPath (
    FlowNetwork & graph,
    int source,
    int sink,
    vector< int > & parentVertex,
    vector< size_t > & parentEdge,
    vector< int > & distance) [inline], [override], [virtual]
```

Finds a minimum-cost augmenting path.

Executes a Bellman-Ford style shortest-path search on the residual network and stores the resulting path reconstruction data.

Negative-cost cycle detection is performed using relaxation counters.

Parameters

<i>graph</i>	Residual flow network.
<i>source</i>	Source vertex.
<i>sink</i>	Sink vertex.
<i>parentVertex</i>	Parent vertex table used for path reconstruction.
<i>parentEdge</i>	Parent edge table used for path reconstruction.
<i>distance</i>	Computed shortest-path distances.

Returns

bool True when a valid augmenting path exists. False when no path is available or a negative cycle is detected.

Implements [IShortestPathStrategy](#).

The documentation for this class was generated from the following file:

- [engine/engines/network_flow/bellman_ford_strategy.hpp](#)

8.2 content_panel.ContentPanel Class Reference

Inheritance diagram for content_panel.ContentPanel:

Collaboration diagram for content_panel.ContentPanel:

Public Member Functions

- None `__init__` (self)
Creates and initializes the content panel.
- bool `is_cached_mode` (self)
Returns the current execution mode.
- None `redraw_graph` (self)
Refreshes the graph visualization.
- None `set_highlighted_mine` (self, str|None mine_id)
Highlights a selected mine.
- None `append_system_log` (self, str message)
Appends an entry to the system log.

Public Attributes

- tuple `current_file_path`
- int `log_index` = 0
- figure
- bool `show_labels` = True
- tuple `highlighted_mine`
- canvas
- toolbar
- stats = QTextEdit()
- tuple `toggle_labels_button`
- mode_group
- btn_cached
- btn_recompute

Protected Member Functions

- None `_initialize_ui` (self)
Builds the content panel user interface.
- QHBoxLayout `_build_toolbar` (self)
Creates the content panel toolbar.
- None `_draw_points` (self)
Renders the complete graph visualization.
- None `_clear_graph` (self)
Clears the current graph visualization.
- Axes `_prepare_graph` (self)
Creates and configures a plotting area.
- None `_draw_miners` (self, Axes graph_axis, WorldData dataset)
Renders miner locations.
- None `_draw_mines` (self, Axes graph_axis, WorldData dataset)
Renders mine locations.
- None `_configure_graph` (self, Axes graph_axis)
Applies final graph configuration.
- dict[str, object] `_get_cached_result` (self)
Retrieves the most recent analysis result.
- None `_load_file` (self)
Imports a dataset from a file.
- None `_save` (self)

- *Saves the current dataset.*
- None [_save_as](#) (self)
- *Exports the current dataset to a file.*
- None [_save_new_raw_dataset](#) (self, WorldData dataset)
- *Stores a new RAW dataset snapshot.*
- None [_save_existing_dataset](#) (self, WorldData dataset)
- *Updates an existing dataset file.*
- WorldData|None [_validate_dataset_for_save](#) (self)
- *Validates dataset availability.*
- None [_refresh_visualization](#) (self)
- *Refreshes the visualization.*
- None [_show_save_error](#) (self, Exception save_error)
- *Displays a save operation error.*
- None [_open_data_dialog](#) (self)
- *Opens the dataset editor.*
- None [_on_data_changed](#) (self)
- *Handles dataset modification events.*
- None [_clear_data](#) (self)
- *Removes the current dataset.*
- None [_toggle_data_labels](#) (self)
- *Toggles point label visibility.*
- None [_open_logs_window](#) (self, _event)
- *Opens the log viewer window.*

Static Protected Member Functions

- None [_draw_assignments](#) (Axes graph_axis, dict[str, object] cached_result)
- *Renders resource allocation assignments.*
- None [_draw_convex_hull](#) (Axes graph_axis, dict[str, object] cached_result)
- *Renders the protected kingdom boundary.*
- None [_build_legend](#) (Axes graph_axis)
- *Creates the graph legend.*

8.2.1 Constructor & Destructor Documentation

8.2.1.1 `__init__()`

```
None content_panel.ContentPanel.__init__ (
    self)
```

Creates and initializes the content panel.

Configures visualization state, creates the plotting canvas, initializes the user interface, loads persisted datasets, and renders the initial visualization. Here is the call graph for this function: Here is the caller graph for this function:

8.2.2 Member Function Documentation

8.2.2.1 _build_legend()

```
None content_panel.ContentPanel._build_legend (
    Axes graph_axis) [static], [protected]
```

Creates the graph legend.

Builds and displays a legend describing all graphical entities and analytical overlays rendered within the visualization.

Parameters

<i>graph_axis</i>	Target plotting axes.
-------------------	-----------------------

Here is the caller graph for this function:

8.2.2.2 _build_toolbar()

```
QHBoxLayout content_panel.ContentPanel._build_toolbar (
    self) [protected]
```

Creates the content panel toolbar.

Builds file management controls, dataset actions, visualization settings, and execution mode selection controls.

Returns

QHBoxLayout Fully configured toolbar layout.

Here is the call graph for this function: Here is the caller graph for this function:

8.2.2.3 _clear_data()

```
None content_panel.ContentPanel._clear_data (
    self) [protected]
```

Removes the current dataset.

Requests user confirmation, deletes persisted runtime data, resets the current file state, and clears the visualization. Here is the call graph for this function: Here is the caller graph for this function:

8.2.2.4 _clear_graph()

```
None content_panel.ContentPanel._clear_graph (
    self) [protected]
```

Clears the current graph visualization.

Removes all rendered elements from the figure and refreshes the drawing canvas. Here is the caller graph for this function:

8.2.2.5 `_configure_graph()`

```
None content_panel.ContentPanel._configure_graph (
    self,
    Axes graph_axis) [protected]
```

Applies final graph configuration.

Configures layout parameters, coordinate formatting, and interactive visualization behavior after all graph elements have been rendered.

Parameters

<i>graph_axis</i>	Target plotting axes.
-------------------	-----------------------

Here is the caller graph for this function:

8.2.2.6 `_draw_assignments()`

```
None content_panel.ContentPanel._draw_assignments (
    Axes graph_axis,
    dict[str, object] cached_result) [static], [protected]
```

Renders resource allocation assignments.

Visualizes assignment relationships between miners and mines using connection lines derived from the latest optimization result.

Parameters

<i>graph_axis</i>	Target plotting axes.
<i>cached_result</i>	Cached algorithm output containing assignment data.

Here is the caller graph for this function:

8.2.2.7 `_draw_convex_hull()`

```
None content_panel.ContentPanel._draw_convex_hull (
    Axes graph_axis,
    dict[str, object] cached_result) [static], [protected]
```

Renders the protected kingdom boundary.

Draws the convex hull generated by the selected computational geometry algorithm and annotates its boundary segments.

Parameters

<i>graph_axis</i>	Target plotting axes.
<i>cached_result</i>	Cached algorithm output containing convex hull data.

Here is the caller graph for this function:

8.2.2.8 _draw_miners()

```
None content_panel.ContentPanel._draw_miners (  
    self,  
    Axes graph_axis,  
    WorldData dataset) [protected]
```

Renders miner locations.

Draws all miners contained in the current dataset and optionally displays their identifiers.

Parameters

<i>graph_axis</i>	Target plotting axes.
<i>dataset</i>	Dataset containing miner information.

Here is the caller graph for this function:

8.2.2.9 _draw_mines()

```
None content_panel.ContentPanel._draw_mines (  
    self,  
    Axes graph_axis,  
    WorldData dataset) [protected]
```

Renders mine locations.

Draws all mining sites contained in the current dataset and optionally displays their identifiers. Highlighted mines are rendered using a distinct visual style.

Parameters

<i>graph_axis</i>	Target plotting axes.
<i>dataset</i>	Dataset containing mine information.

Here is the caller graph for this function:

8.2.2.10 _draw_points()

```
None content_panel.ContentPanel._draw_points (  
    self) [protected]
```

Renders the complete graph visualization.

Loads the current dataset, prepares the graph, renders all entities and analytical overlays, and updates the visualization canvas. Here is the call graph for this function: Here is the caller graph for this function:

8.2.2.11 `_get_cached_result()`

```
dict[str, object] content_panel.ContentPanel._get_cached_result (  
    self) [protected]
```

Retrieves the most recent analysis result.

Obtains the cached algorithm output stored by the control panel and returns an empty result when no analysis has been executed.

Returns

dict[str, object] Cached algorithm result.

Here is the caller graph for this function:

8.2.2.12 `_initialize_ui()`

```
None content_panel.ContentPanel._initialize_ui (  
    self) [protected]
```

Builds the content panel user interface.

Creates the visualization canvas, navigation toolbar, logging area, and all layout containers required by the content panel. Here is the call graph for this function: Here is the caller graph for this function:

8.2.2.13 `_load_file()`

```
None content_panel.ContentPanel._load_file (  
    self) [protected]
```

Imports a dataset from a file.

Opens a file selection dialogue, loads the selected dataset, updates the current application state, and refreshes the visualization. Here is the call graph for this function: Here is the caller graph for this function:

8.2.2.14 `_on_data_changed()`

```
None content_panel.ContentPanel._on_data_changed (  
    self) [protected]
```

Handles dataset modification events.

Invalidates cached analysis results and updates the visualization after the input dataset has been modified. Here is the call graph for this function:

8.2.2.15 _open_data_dialog()

```
None content_panel.ContentPanel._open_data_dialog (
    self) [protected]
```

Opens the dataset editor.

Displays the data management dialogue, detects dataset modifications, invalidates outdated analysis results, and refreshes the visualization. Here is the call graph for this function: Here is the caller graph for this function:

8.2.2.16 _open_logs_window()

```
None content_panel.ContentPanel._open_logs_window (
    self,
    _event) [protected]
```

Opens the log viewer window.

Displays the complete system log history inside a dedicated dialogue window.

Parameters

<code>_event</code>	Mouse event triggering the action.
---------------------	------------------------------------

8.2.2.17 _prepare_graph()

```
Axes content_panel.ContentPanel._prepare_graph (
    self) [protected]
```

Creates and configures a plotting area.

Initializes the graph axes, applies visual configuration, and prepares the canvas for entity rendering.

Returns

Axes Configured matplotlib plotting axes.

Here is the caller graph for this function:

8.2.2.18 _refresh_visualization()

```
None content_panel.ContentPanel._refresh_visualization (
    self) [protected]
```

Refreshes the visualization.

Clears the current graph and rebuilds the complete visualization using the latest dataset and analysis results. Here is the call graph for this function: Here is the caller graph for this function:

8.2.2.19 _save()

```
None content_panel.ContentPanel._save (
    self) [protected]
```

Saves the current dataset.

Persists the active dataset, either to its existing location or to the RAW dataset repository when no file association exists. Here is the call graph for this function: Here is the caller graph for this function:

8.2.2.20 _save_as()

```
None content_panel.ContentPanel._save_as (
    self) [protected]
```

Exports the current dataset to a file.

Opens a save dialogue, allows the user to choose an export location, and stores the dataset as a Huffman-compressed file. Here is the call graph for this function: Here is the caller graph for this function:

8.2.2.21 _save_existing_dataset()

```
None content_panel.ContentPanel._save_existing_dataset (
    self,
    WorldData dataset) [protected]
```

Updates an existing dataset file.

Saves the specified dataset to the currently associated file location and performs RAW repository maintenance when required.

Parameters

<i>dataset</i>	Dataset to persist.
----------------	---------------------

Here is the call graph for this function: Here is the caller graph for this function:

8.2.2.22 _save_new_raw_dataset()

```
None content_panel.ContentPanel._save_new_raw_dataset (
    self,
    WorldData dataset) [protected]
```

Stores a new RAW dataset snapshot.

Creates a content-addressed dataset file within the RAW dataset repository and updates the current file association.

Parameters

<i>dataset</i>	Dataset to persist.
----------------	---------------------

Here is the call graph for this function: Here is the caller graph for this function:

8.2.2.23 _show_save_error()

```
None content_panel.ContentPanel._show_save_error (
    self,
    Exception save_error) [protected]
```

Displays a save operation error.

Presents a dialogue describing an unexpected failure encountered during dataset persistence.

Parameters

<i>save_error</i>	Exception describing the failure.
-------------------	-----------------------------------

Here is the caller graph for this function:

8.2.2.24 _toggle_data_labels()

```
None content_panel.ContentPanel._toggle_data_labels (
    self) [protected]
```

Toggles point label visibility.

Enables or disables entity labels displayed within the graph visualization and refreshes the rendered view. Here is the call graph for this function:

8.2.2.25 _validate_dataset_for_save()

```
WorldData | None content_panel.ContentPanel._validate_dataset_for_save (
    self) [protected]
```

Validates dataset availability.

Ensures that a dataset is currently loaded before a save operation is performed.

Returns

WorldData | None Dataset ready for persistence or None when no dataset is available.

Here is the caller graph for this function:

8.2.2.26 `append_system_log()`

```
None content_panel.ContentPanel.append_system_log (  
    self,  
    str message)
```

Appends an entry to the system log.

Creates a timestamped log entry and displays it within the integrated log viewer.

Parameters

<i>message</i>	Log message to append.
----------------	------------------------

Here is the caller graph for this function:

8.2.2.27 `is_cached_mode()`

```
bool content_panel.ContentPanel.is_cached_mode (  
    self)
```

Returns the current execution mode.

Determines whether cached results should be used instead of executing a fresh algorithm run.

Returns

bool True when cache mode is enabled.

8.2.2.28 `redraw_graph()`

```
None content_panel.ContentPanel.redraw_graph (  
    self)
```

Refreshes the graph visualization.

Clears the current visualization and redraws all entities and analytical overlays. Here is the call graph for this function:

8.2.2.29 set_highlighted_mine()

```
None content_panel.ContentPanel.set_highlighted_mine (
    self,
    str | None mine_id)
```

Highlights a selected mine.

Updates the currently highlighted mine and refreshes the visualization to reflect the selection state.

Parameters

<i>mine_id</i>	Identifier of the mine to highlight or None to clear the current highlight.
----------------	---

Here is the call graph for this function:

8.2.3 Member Data Documentation

8.2.3.1 btn_cached

```
content_panel.ContentPanel.btn_cached
```

Initial value:

```
= QPushButton(
    "Use Cache"
)
```

8.2.3.2 btn_recompute

```
content_panel.ContentPanel.btn_recompute
```

Initial value:

```
= QPushButton(
    "Run Analysis"
)
```

8.2.3.3 canvas

```
content_panel.ContentPanel.canvas
```

Initial value:

```
= FigureCanvas(
    self.figure
)
```

8.2.3.4 current_file_path

```
tuple content_panel.ContentPanel.current_file_path
```

Initial value:

```
= (
    None
)
```

8.2.3.5 figure

```
content_panel.ContentPanel.figure
```

Initial value:

```
= Figure(  
    dpi=160  
)
```

8.2.3.6 highlighted_mine

```
tuple content_panel.ContentPanel.highlighted_mine
```

Initial value:

```
= (  
    None  
)
```

8.2.3.7 log_index

```
int content_panel.ContentPanel.log_index = 0
```

8.2.3.8 mode_group

```
content_panel.ContentPanel.mode_group
```

Initial value:

```
= QButtonGroup(  
    self  
)
```

8.2.3.9 show_labels

```
bool content_panel.ContentPanel.show_labels = True
```

8.2.3.10 stats

```
content_panel.ContentPanel.stats = QTextEdit()
```

8.2.3.11 toggle_labels_button

```
tuple content_panel.ContentPanel.toggle_labels_button
```

Initial value:

```
= (  
    create_toolbar_button(  
        "Disable Labels"  
    )  
)
```

8.2.3.12 toolbar

content_panel.ContentPanel.toolbar

Initial value:

```
= NavigationToolbar2QT(
    self.canvas,
    self,
)
```

The documentation for this class was generated from the following file:

- interface/[content_panel.py](#)

8.3 control_panel.ControlPanel Class Reference

Analysis control panel.

Inheritance diagram for control_panel.ControlPanel:

Collaboration diagram for control_panel.ControlPanel:

Public Member Functions

- None [__init__](#) (self)
Creates and initializes the control panel.
- None [on_start](#) (self)
Starts algorithm execution.
- None [run_algorithm](#) (self, str flow_algorithm, str cost_algorithm, str geometry_algorithm)
Executes the selected analysis pipeline.
- None [on_find_loudest_dwarf](#) (self)
Locates the loudest dwarf within a range.
- None [save_result](#) (self)
Exports the current analysis result.

Public Attributes

- tuple [cached_result](#)
- tuple [cache_hit](#)
- tuple [execution_signature](#)
- tuple [execution_failed](#)
- [maxflow_group](#)
- [mincost_group](#)
- [geo_group](#)
- [seg_group](#)
- [seg_box](#)
- [range_input](#) = QLineEdit()
- [iterations_input](#) = QSpinBox()
- [time_label](#)
- [result_label](#)
- [start_btn](#)
- [dwarf_btn](#)
- [save_btn](#)

Static Public Attributes

- `log_signal` = `Signal(str)`
Emitted when a new log entry is generated.
- `refresh_graph_signal` = `Signal()`
Emitted when the visualization requires refreshing.

Protected Member Functions

- `None` `_initialize_ui` (self)
Builds the control panel user interface.
- `tuple[str, str, str]|None` `_get_selected_algorithms` (self)
Retrieves the selected algorithms.
- `tuple[int, int]|None` `_parse_range_expression` (self)
Parses a range query expression.
- `bool` `_validate_dwarf_search` (self)
Validates dwarf search prerequisites.
- `WorldData|None` `_get_validated_dataset` (self, str error_message)
Loads and validates the active dataset.
- `bool` `_load_memory_cache` (self, str execution_signature)
Attempts to load a cached memory result.
- `bool` `_load_file_cache` (self, Path result_file_path, str execution_signature, str output_filename)
Attempts to load a cached file result.
- `None` `_set_status` (self, str result_text, str|None time_text=None)
Updates execution status information.
- `None` `_update_buttons_state` (self)
Updates action button availability.
- `None` `_reset_execution_state` (self)
Resets execution state flags.
- `None` `_initialize_execution_status` (self)
Initializes execution status indicators.
- `None` `_finalize_execution` (self)
Finalizes a successful execution.
- `None` `_handle_execution_success` (self, dict[str, dict[str, object],] execution_result, str execution_signature)
Processes successful execution results.
- `None` `_handle_execution_errors` (self, list[str] execution_errors)
Processes algorithm execution errors.
- `None` `_handle_execution_exception` (self, Exception error)
Handles unexpected execution exceptions.
- `None` `_handle_benchmark_results` (self, int execution_count, dict[str, float] benchmark_metrics, float total_execution_time_ms)
Processes benchmark execution results.
- `list[dict[str, object]]|None` `_get_convex_hull_points` (self)
Retrieves convex hull points.
- `None` `_highlight_loudest_dwarf` (self, dict[str, object]|None loudest_dwarf)
Highlights the loudest dwarf location.
- `ContentPanel|None` `_get_content_panel` (self)
Resolves the associated content panel.
- `int` `_get_execution_count` (self, int execution_iterations)
Determines the effective execution count.

- dict[str, float] [_run_benchmark_iterations](#) (self, int execution_count, str flow_algorithm, str cost_algorithm, str geometry_algorithm)
Executes benchmark iterations.
- None [_log_execution_completion](#) (self, int execution_iterations, int execution_count)
Logs execution completion information.
- None [_handle_single_execution_results](#) (self, float total_execution_time_ms)
Processes single-run benchmark results.
- None [_handle_multi_execution_results](#) (self, int execution_count, dict[str, float] benchmark_metrics, float total_execution_time_ms)
Processes multi-run benchmark results.
- None [_handle_dwarf_search_error](#) (self, dict[str, object] dwarf_search_result)
Handles range query execution errors.
- None [_log_flow_summary](#) (self, dict[str, object] assignment_result, dict[str, object] benchmark_result, bool use_average_metrics)
Logs flow optimization results.
- None [_log_assignment_summary](#) (self, list[dict[str, object]] assignment_entries)
Logs resource allocation assignments.
- list[dict[str, object]] [_log_geometry_summary](#) (self, dict[str, object] geometry_result, dict[str, object] benchmark_result, bool use_average_metrics)
Logs geometry analysis results.
- None [_log_boundary_points](#) (self, list[dict[str, object]] convex_hull_points)
Logs convex hull boundary points.
- None [_log_boundary_segments](#) (self, list[dict[str, object]] convex_hull_points)
Logs boundary route segments.
- None [_log_result_summary](#) (self, dict[str, object] result)
Generates a complete execution report.
- None [_log_dwarf_result](#) (self, dict[str, object] result, str selected_algorithm)
Logs range query search results.

Static Protected Member Functions

- str|None [_get_selected](#) (QButtonGroup group)
Returns the selected button label.
- dict[str, dict[str, object]] [_execute_algorithms](#) (WorldData dataset, str flow_algorithm, str cost_algorithm, str geometry_algorithm)
Executes the selected native algorithms.
- list[str] [_collect_execution_errors](#) (dict[str, dict[str, object]] execution_result)
Extracts execution errors from results.
- dict[str, float] [_initialize_benchmark_metrics](#) ()
Creates benchmark metric accumulators.
- None [_update_benchmark_metrics](#) (dict[str, float] benchmark_metrics, dict[str, object] cached_result)
Updates aggregated benchmark statistics.
- float [_calculate_total_execution_time](#) (float execution_start_time)
Calculates total execution duration.
- dict[str, object] [_build_ordered_payload_from_hull](#) (list[dict[str, object]] hull, WorldData data)
Builds a range query payload from a convex hull.
- dict[str, object] [_execute_dwarf_search](#) (str selected_range_algorithm, dict[str, object] ordered_payload, int range_start, int range_end)
Executes a range query search.
- tuple[str, str] [_build_execution_signature](#) (WorldData dataset, str flow_algorithm, str cost_algorithm, str geometry_algorithm)

- Generates a unique execution signature.*
 - Path `_get_result_file_path` (str output_filename)
 - Resolves the output artifact path.*
 - dict[str, object] `_build_export_payload` (str flow_algorithm, str cost_algorithm, str geometry_algorithm, dict[str, object] canonical_dataset, dict[str, object] cached_result)
 - Builds an exportable result payload.*
 - None `_export_compressed_payload` (Path output_file_path, dict[str, object] export_payload)
 - Exports a compressed result artifact.*
 - tuple[str, int, float, float] `_extract_dwarf_data` (dict[str, object] dwarf)
 - Extracts dwarf information from a result entry.*
 - str `_tree_prefix` (int index, int size)
 - Generates a tree-structure prefix.*
 - str `_branch_row` (str label, str value)
 - Formats an intermediate tree row.*
 - str `_terminal_row` (str label, str value)
 - Formats an intermediate tree row.*

8.3.1 Detailed Description

Analysis control panel.

Serves as the primary interface for configuring optimization, geometry, and range query algorithms.

The panel manages algorithm execution, result caching, benchmarking, and communication with the visualization subsystem.

8.3.2 Constructor & Destructor Documentation

8.3.2.1 `__init__()`

```
None control_panel.ControlPanel.__init__ (
    self)
```

Creates and initializes the control panel.

Configures execution state, cache management, signal infrastructure, and user interface components used to control algorithm execution. Here is the call graph for this function: Here is the caller graph for this function:

8.3.3 Member Function Documentation

8.3.3.1 _branch_row()

```
str control_panel.ControlPanel._branch_row (
    str label,
    str value) [static], [protected]
```

Formats an intermediate tree row.

Creates a formatted log row representing a non-terminal branch entry.

Parameters

<i>label</i>	Entry label.
<i>value</i>	Entry value.

Returns

str Formatted log row.

Here is the caller graph for this function:

8.3.3.2 _build_execution_signature()

```
tuple[str, str] control_panel.ControlPanel._build_execution_signature (
    WorldData dataset,
    str flow_algorithm,
    str cost_algorithm,
    str geometry_algorithm) [static], [protected]
```

Generates a unique execution signature.

Creates deterministic identifiers based on the selected algorithms and canonical dataset content. The generated signature is used for cache lookup and result file naming.

Parameters

<i>dataset</i>	Input dataset.
<i>flow_algorithm</i>	Selected maximum flow algorithm.
<i>cost_algorithm</i>	Selected minimum-cost pathfinding algorithm.
<i>geometry_algorithm</i>	Selected geometry algorithm.

Returns

tuple[str, str] Execution signature and output filename.

Here is the caller graph for this function:

8.3.3.3 `_build_export_payload()`

```
dict[str, object] control_panel.ControlPanel._build_export_payload (
    str flow_algorithm,
    str cost_algorithm,
    str geometry_algorithm,
    dict[str, object] canonical_dataset,
    dict[str, object] cached_result) [static], [protected]
```

Builds an exportable result payload.

Combines algorithm configuration, canonical input data, and execution results into a single structure suitable for persistence.

Parameters

<i>flow_algorithm</i>	Selected maximum flow algorithm.
<i>cost_algorithm</i>	Selected minimum-cost pathfinding algorithm.
<i>geometry_algorithm</i>	Selected geometry algorithm.
<i>canonical_dataset</i>	Canonical dataset representation.
<i>cached_result</i>	Cached execution result.

Returns

dict[str, object] Export payload.

Here is the caller graph for this function:

8.3.3.4 `_build_ordered_payload_from_hull()`

```
dict[str, object] control_panel.ControlPanel._build_ordered_payload_from_hull (
    list[dict[str, object]] hull,
    WorldData data) [static], [protected]
```

Builds a range query payload from a convex hull.

Converts convex hull points into an ordered payload containing mine locations and associated warden loudness values.

Parameters

<i>hull</i>	Convex hull points.
<i>data</i>	Input dataset.

Returns

dict[str, object] Ordered range query payload.

Here is the caller graph for this function:

8.3.3.5 _calculate_total_execution_time()

```
float control_panel.ControlPanel._calculate_total_execution_time (
    float execution_start_time) [static], [protected]
```

Calculates total execution duration.

Computes the elapsed execution time in milliseconds from the provided start timestamp.

Parameters

<i>execution_start_time</i>	Execution start timestamp.
-----------------------------	----------------------------

Returns

float Total execution time in milliseconds.

Here is the caller graph for this function:

8.3.3.6 _collect_execution_errors()

```
list[str] control_panel.ControlPanel._collect_execution_errors (
    dict[str, dict[str, object]] execution_result) [static], [protected]
```

Extracts execution errors from results.

Scans algorithm outputs and collects formatted error messages reported by failed executions.

Parameters

<i>execution_result</i>	Aggregated algorithm results.
-------------------------	-------------------------------

Returns

list[str] Collection of execution error messages.

Here is the caller graph for this function:

8.3.3.7 _execute_algorithms()

```
dict[str, dict[str, object]] control_panel.ControlPanel._execute_algorithms (
    WorldData dataset,
    str flow_algorithm,
    str cost_algorithm,
    str geometry_algorithm) [static], [protected]
```

Executes the selected native algorithms.

Invokes the configured optimization and geometry executables and collects their results into a unified response structure.

Parameters

<i>dataset</i>	Input dataset supplied to the algorithms.
<i>flow_algorithm</i>	Selected maximum flow algorithm.
<i>cost_algorithm</i>	Selected minimum-cost pathfinding algorithm.
<i>geometry_algorithm</i>	Selected convex hull algorithm.

Returns

dict[str, dict[str, object]] Aggregated algorithm results.

Here is the caller graph for this function:

8.3.3.8 _execute_dwarf_search()

```
dict[str, object] control_panel.ControlPanel._execute_dwarf_search (
    str selected_range_algorithm,
    dict[str, object] ordered_payload,
    int range_start,
    int range_end) [static], [protected]
```

Executes a range query search.

Invokes the selected range query algorithm using the specified search interval and payload.

Parameters

<i>selected_range_algorithm</i>	Selected range query algorithm.
<i>ordered_payload</i>	Ordered convex hull payload.
<i>range_start</i>	Lower search boundary.
<i>range_end</i>	Upper search boundary.

Returns

dict[str, object] Range query result.

Here is the caller graph for this function:

8.3.3.9 _export_compressed_payload()

```
None control_panel.ControlPanel._export_compressed_payload (
    Path output_file_path,
    dict[str, object] export_payload) [static], [protected]
```

Exports a compressed result artifact.

Serializes the provided payload, compresses it using Huffman coding, and writes the resulting artifact to disk.

Parameters

<i>output_file_path</i>	Destination file path.
<i>export_payload</i>	Payload to export.

Here is the caller graph for this function:

8.3.3.10 _extract_dwarf_data()

```
tuple[str, int, float, float] control_panel.ControlPanel._extract_dwarf_data (
    dict[str, object] dwarf) [static], [protected]
```

Extracts dwarf information from a result entry.

Converts a serialized dwarf record into strongly typed values used by the reporting subsystem.

Parameters

<i>dwarf</i>	Dwarf result entry.
--------------	---------------------

Returns

tuple[str, int, float, float] Identifier, loudness, and coordinates.

Here is the caller graph for this function:

8.3.3.11 _finalize_execution()

```
None control_panel.ControlPanel._finalize_execution (
    self) [protected]
```

Finalizes a successful execution.

Updates status information, logs result summaries, and triggers visualization refresh operations. Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.12 _get_content_panel()

```
ContentPanel | None control_panel.ControlPanel._get_content_panel (
    self) [protected]
```

Resolves the associated content panel.

Traverses the widget hierarchy and returns the content panel connected to the control panel.

Returns

ContentPanel | None Associated content panel instance.

Here is the caller graph for this function:

8.3.3.13 __get_convex_hull_points()

```
list[dict[str, object]] | None control_panel.ControlPanel._get_convex_hull_points (
    self) [protected]
```

Retrieves convex hull points.

Extracts the convex hull generated by the geometry algorithm from the cached execution result.

Returns

list[dict[str, object]] | None Convex hull points or None when unavailable.

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.14 __get_execution_count()

```
int control_panel.ControlPanel._get_execution_count (
    self,
    int execution_iterations) [protected]
```

Determines the effective execution count.

Returns the number of benchmark iterations that should be executed, taking cache mode into account.

Parameters

<i>execution_iterations</i>	User-requested iteration count.
-----------------------------	---------------------------------

Returns

int Effective execution count.

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.15 __get_result_file_path()

```
Path control_panel.ControlPanel._get_result_file_path (
    str output_filename) [static], [protected]
```

Resolves the output artifact path.

Creates the artifacts directory when necessary and returns the destination path for a generated result file.

Parameters

<i>output_filename</i>	Result filename.
------------------------	------------------

Returns

Path Full artifact file path.

Here is the caller graph for this function:

8.3.3.16 __get_selected()

```
str | None control_panel.ControlPanel.__get_selected (
    QButtonGroup group) [static], [protected]
```

Returns the selected button label.

Retrieves the text associated with the currently selected button in the specified button group.

Parameters

<i>group</i>	Button group to inspect.
--------------	--------------------------

Returns

str | None Selected button label or None when no button is selected.

Here is the caller graph for this function:

8.3.3.17 __get_selected_algorithms()

```
tuple[str, str, str] | None control_panel.ControlPanel.__get_selected_algorithms (
    self) [protected]
```

Retrieves the selected algorithms.

Obtains the currently selected flow, pathfinding, and geometry algorithms and validates that all required selections have been made.

Returns

tuple[str, str, str] | None Selected algorithm names or None when the configuration is incomplete.

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.18 __get_validated_dataset()

```
WorldData | None control_panel.ControlPanel.__get_validated_dataset (
    self,
    str error_message) [protected]
```

Loads and validates the active dataset.

Retrieves the current dataset and applies failure handling when no data is available.

Parameters

<i>error_message</i>	Log message emitted when validation fails.
----------------------	--

Returns

WorldData | None Valid dataset or None when unavailable.

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.19 `_handle_benchmark_results()`

```
None control_panel.ControlPanel._handle_benchmark_results (
    self,
    int execution_count,
    dict[str, float] benchmark_metrics,
    float total_execution_time_ms) [protected]
```

Processes benchmark execution results.

Updates performance statistics and dispatches benchmark summaries according to the number of executed iterations.

Parameters

<i>execution_count</i>	Number of completed executions.
<i>benchmark_metrics</i>	Collected benchmark statistics.
<i>total_execution_time_ms</i>	Total benchmark duration.

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.20 `_handle_dwarf_search_error()`

```
None control_panel.ControlPanel._handle_dwarf_search_error (
    self,
    dict[str, object] dwarf_search_result) [protected]
```

Handles range query execution errors.

Reports search failures, updates execution state, and disables functionality dependent on valid range query results.

Parameters

<i>dwarf_search_result</i>	Failed range query result.
----------------------------	----------------------------

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.21 `_handle_execution_errors()`

```
None control_panel.ControlPanel._handle_execution_errors (
    self,
    list[str] execution_errors) [protected]
```

Processes algorithm execution errors.

Logs reported execution failures and updates the control panel status accordingly.

Parameters

<i>execution_errors</i>	Collection of execution error messages.
-------------------------	---

Here is the call graph for this function: [Here is the caller graph for this function:](#)

8.3.3.22 _handle_execution_exception()

```
None control_panel.ControlPanel._handle_execution_exception (  
    self,  
    Exception error) [protected]
```

Handles unexpected execution exceptions.

Reports an unhandled execution failure and transitions the control panel into an error state.

Parameters

<i>error</i>	Exception raised during execution.
--------------	------------------------------------

Here is the call graph for this function: [Here is the caller graph for this function:](#)

8.3.3.23 _handle_execution_success()

```
None control_panel.ControlPanel._handle_execution_success (  
    self,  
    dict[ str, dict[str, object], ] execution_result,  
    str execution_signature) [protected]
```

Processes successful execution results.

Stores execution results, updates cache metadata, and enables functionality dependent on analysis output.

Parameters

<i>execution_result</i>	Aggregated algorithm results.
<i>execution_signature</i>	Unique execution signature.

Here is the call graph for this function: [Here is the caller graph for this function:](#)

8.3.3.24 `_handle_multi_execution_results()`

```
None control_panel.ControlPanel._handle_multi_execution_results (
    self,
    int execution_count,
    dict[str, float] benchmark_metrics,
    float total_execution_time_ms) [protected]
```

Processes multi-run benchmark results.

Computes average execution metrics and stores benchmark statistics within the cached result.

Parameters

<i>execution_count</i>	Number of benchmark runs.
<i>benchmark_metrics</i>	Aggregated benchmark metrics.
<i>total_execution_time_ms</i>	Total benchmark duration.

Here is the caller graph for this function:

8.3.3.25 `_handle_single_execution_results()`

```
None control_panel.ControlPanel._handle_single_execution_results (
    self,
    float total_execution_time_ms) [protected]
```

Processes single-run benchmark results.

Updates execution statistics after a single algorithm execution.

Parameters

<i>total_execution_time_ms</i>	Total execution duration.
--------------------------------	---------------------------

Here is the caller graph for this function:

8.3.3.26 `_highlight_loudest_dwarf()`

```
None control_panel.ControlPanel._highlight_loudest_dwarf (
    self,
    dict[str, object] | None loudest_dwarf) [protected]
```

Highlights the loudest dwarf location.

Updates the graph visualization to emphasize the mine associated with the detected loudest dwarf.

Parameters

<i>loudest_dwarf</i>	Loudest dwarf search result.
----------------------	------------------------------

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.27 _initialize_benchmark_metrics()

```
dict[str, float] control_panel.ControlPanel._initialize_benchmark_metrics () [static], [protected]
```

Creates benchmark metric accumulators.

Initializes containers used to aggregate performance measurements across benchmark runs.

Returns

dict[str, float] Initialized benchmark metrics.

Here is the caller graph for this function:

8.3.3.28 _initialize_execution_status()

```
None control_panel.ControlPanel._initialize_execution_status (  
    self) [protected]
```

Initializes execution status indicators.

Updates the user interface to reflect that an analysis is currently running. Here is the call graph for this function:

Here is the caller graph for this function:

8.3.3.29 _initialize_ui()

```
None control_panel.ControlPanel._initialize_ui (  
    self) [protected]
```

Builds the control panel user interface.

Creates algorithm selection controls, execution parameter inputs, status indicators, and action buttons used to configure and execute analyses.

The interface provides configuration options for:

- maximum flow algorithms,
- minimum-cost pathfinding algorithms,
- convex hull algorithms,
- range query algorithms,
- benchmarking parameters.

Here is the caller graph for this function:

8.3.3.30 `_load_file_cache()`

```
bool control_panel.ControlPanel._load_file_cache (  
    self,  
    Path result_file_path,  
    str execution_signature,  
    str output_filename) [protected]
```

Attempts to load a cached file result.

Restores a previously exported analysis result from persistent storage when cache mode is enabled.

Parameters

<i>result_file_path</i>	Cached result file location.
<i>execution_signature</i>	Unique execution signature.
<i>output_filename</i>	Cached output filename.

Returns

bool True when a cached result was loaded.

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.31 `_load_memory_cache()`

```
bool control_panel.ControlPanel._load_memory_cache (  
    self,  
    str execution_signature) [protected]
```

Attempts to load a cached memory result.

Reuses an already available execution result when the current execution signature matches the previously executed configuration.

Parameters

<i>execution_signature</i>	Unique execution signature.
----------------------------	-----------------------------

Returns

bool True when a cached result was loaded.

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.32 _log_assignment_summary()

```
None control_panel.ControlPanel._log_assignment_summary (
    self,
    list[ dict[str, object] ] assignment_entries) [protected]
```

Logs resource allocation assignments.

Generates a formatted summary of all miner-to-mine assignments produced by the flow optimization stage.

Parameters

<i>assignment_entries</i>	Collection of assignment records.
---------------------------	-----------------------------------

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.33 _log_boundary_points()

```
None control_panel.ControlPanel._log_boundary_points (
    self,
    list[dict[str, object]] convex_hull_points) [protected]
```

Logs convex hull boundary points.

Outputs all points belonging to the protected kingdom boundary in traversal order.

Parameters

<i>convex_hull_points</i>	Convex hull vertices.
---------------------------	-----------------------

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.34 _log_boundary_segments()

```
None control_panel.ControlPanel._log_boundary_segments (
    self,
    list[dict[str, object]] convex_hull_points) [protected]
```

Logs boundary route segments.

Generates a sequential representation of the inspection route formed by convex hull edges.

Parameters

<i>convex_hull_points</i>	Convex hull vertices.
---------------------------	-----------------------

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.35 _log_dwarf_result()

```
None control_panel.ControlPanel._log_dwarf_result (
    self,
    dict[str, object] result,
    str selected_algorithm) [protected]
```

Logs range query search results.

Reports the loudest detected dwarf together with search statistics and execution timing.

Parameters

<i>result</i>	Range query result.
<i>selected_algorithm</i>	Executed range query algorithm.

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.36 _log_execution_completion()

```
None control_panel.ControlPanel._log_execution_completion (
    self,
    int execution_iterations,
    int execution_count) [protected]
```

Logs execution completion information.

Records benchmark and execution completion information in the system log.

Parameters

<i>execution_iterations</i>	Requested iteration count.
<i>execution_count</i>	Actual execution count.

Here is the caller graph for this function:

8.3.3.37 _log_flow_summary()

```
None control_panel.ControlPanel._log_flow_summary (
    self,
    dict[str, object] assignment_result,
    dict[str, object] benchmark_result,
    bool use_average_metrics) [protected]
```

Logs flow optimization results.

Generates a formatted summary containing optimization statistics, algorithm selections, objective values, and execution timings.

Parameters

<i>assignment_result</i>	Flow optimization result.
<i>benchmark_result</i>	Benchmark statistics.
<i>use_average_metrics</i>	Determines whether benchmark averages or single-execution timings should be reported.

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.38 _log_geometry_summary()

```
list[dict[str, object]] control_panel.ControlPanel._log_geometry_summary (
    self,
    dict[str, object] geometry_result,
    dict[str, object] benchmark_result,
    bool use_average_metrics) [protected]
```

Logs geometry analysis results.

Reports convex hull statistics and execution timing information for the selected geometry algorithm.

Parameters

<i>geometry_result</i>	Geometry analysis result.
<i>benchmark_result</i>	Benchmark statistics.
<i>use_average_metrics</i>	Determines whether average benchmark metrics should be reported.

Returns

list[dict[str, object]] Convex hull points.

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.39 _log_result_summary()

```
None control_panel.ControlPanel._log_result_summary (
    self,
    dict[str, object] result) [protected]
```

Generates a complete execution report.

Aggregates optimization, assignment, geometry, and boundary information into a structured execution summary written to the system log.

Parameters

<i>result</i>	Combined execution result.
---------------	----------------------------

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.40 `_parse_range_expression()`

```
tuple[int, int] | None control_panel.ControlPanel._parse_range_expression (
    self) [protected]
```

Parses a range query expression.

Converts a textual range expression into a numerical interval used by range query algorithms.

Returns

tuple[int, int] | None Parsed range boundaries or None when the expression is invalid.

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.41 `_reset_execution_state()`

```
None control_panel.ControlPanel._reset_execution_state (
    self) [protected]
```

Resets execution state flags.

Clears temporary execution indicators and prepares the control panel for a new analysis. Here is the caller graph for this function:

8.3.3.42 `_run_benchmark_iterations()`

```
dict[str, float] control_panel.ControlPanel._run_benchmark_iterations (
    self,
    int execution_count,
    str flow_algorithm,
    str cost_algorithm,
    str geometry_algorithm) [protected]
```

Executes benchmark iterations.

Repeatedly runs the selected algorithms and accumulates benchmark performance metrics.

Parameters

<i>execution_count</i>	Number of executions to perform.
<i>flow_algorithm</i>	Selected maximum flow algorithm.
<i>cost_algorithm</i>	Selected minimum-cost pathfinding algorithm.
<i>geometry_algorithm</i>	Selected geometry algorithm.

Returns

dict[str, float] Aggregated benchmark metrics.

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.43 _set_status()

```
None control_panel.ControlPanel._set_status (
    self,
    str result_text,
    str | None time_text = None) [protected]
```

Updates execution status information.

Updates the status and execution time labels displayed within the control panel.

Parameters

<i>result_text</i>	Status message to display.
<i>time_text</i>	Optional execution time value.

Here is the caller graph for this function:

8.3.3.44 _terminal_row()

```
str control_panel.ControlPanel._terminal_row (
    str label,
    str value) [static], [protected]
```

Formats an intermediate tree row.

Creates a formatted log row representing a non-terminal branch entry.

Parameters

<i>label</i>	Entry label.
<i>value</i>	Entry value.

Returns

str Formatted log row.

Here is the caller graph for this function:

8.3.3.45 _tree_prefix()

```
str control_panel.ControlPanel._tree_prefix (
    int index,
    int size) [static], [protected]
```

Generates a tree-structure prefix.

Produces a textual tree connector used when formatting hierarchical log entries.

Parameters

<i>index</i>	Current element index.
<i>size</i>	Total number of elements.

Returns

str Tree connector prefix.

Here is the caller graph for this function:

8.3.3.46 _update_benchmark_metrics()

```
None control_panel.ControlPanel._update_benchmark_metrics (
    dict[str, float] benchmark_metrics,
    dict[str, object] cached_result) [static], [protected]
```

Updates aggregated benchmark statistics.

Extracts timing information from the current execution result and accumulates benchmark metrics.

Parameters

<i>benchmark_metrics</i>	Metric accumulators updated in place.
<i>cached_result</i>	Result containing timing information.

Here is the caller graph for this function:

8.3.3.47 _update_buttons_state()

```
None control_panel.ControlPanel._update_buttons_state (
    self) [protected]
```

Updates action button availability.

Enables or disables controls according to the current algorithm selection and result state. Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.48 _validate_dwarf_search()

```
bool control_panel.ControlPanel._validate_dwarf_search (
    self) [protected]
```

Validates dwarf search prerequisites.

Ensures that a valid analysis result exists before executing a range query operation.

Returns

bool True when the search can proceed.

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.49 on_find_loudest_dwarf()

```
None control_panel.ControlPanel.on_find_loudest_dwarf (
    self)
```

Locates the loudest dwarf within a range.

Executes the selected range query algorithm, identifies the loudest dwarf contained within the specified search interval, and highlights the corresponding mine on the visualization. Here is the call graph for this function:

8.3.3.50 on_start()

```
None control_panel.ControlPanel.on_start (
    self)
```

Starts algorithm execution.

Validates the current configuration, executes the selected algorithms, collects benchmarking metrics, and updates the user interface with execution results. Here is the call graph for this function:

8.3.3.51 run_algorithm()

```
None control_panel.ControlPanel.run_algorithm (
    self,
    str flow_algorithm,
    str cost_algorithm,
    str geometry_algorithm)
```

Executes the selected analysis pipeline.

Performs cache validation, executes optimization and geometry algorithms when necessary, handles execution errors, and stores successful results.

Parameters

<i>flow_algorithm</i>	Selected maximum flow algorithm.
<i>cost_algorithm</i>	Selected minimum-cost pathfinding algorithm.
<i>geometry_algorithm</i>	Selected convex hull algorithm.

Here is the call graph for this function: Here is the caller graph for this function:

8.3.3.52 save_result()

```
None control_panel.ControlPanel.save_result (
    self)
```

Exports the current analysis result.

Builds an export payload containing the selected algorithm configuration, input dataset, and analysis results, then stores it as a compressed output file. Here is the call graph for this function:

8.3.4 Member Data Documentation

8.3.4.1 cache_hit

`tuple control_panel.ControlPanel.cache_hit`

Initial value:

```
= (
    False
)
```

8.3.4.2 cached_result

`control_panel.ControlPanel.cached_result`

Initial value:

```
= (
    None
)
```

8.3.4.3 dwarf_btn

`control_panel.ControlPanel.dwarf_btn`

Initial value:

```
= QPushButton(
    "Locate"
)
```

8.3.4.4 execution_failed

`tuple control_panel.ControlPanel.execution_failed`

Initial value:

```
= (
    False
)
```

8.3.4.5 execution_signature

`tuple control_panel.ControlPanel.execution_signature`

Initial value:

```
= (
    None
)
```

8.3.4.6 geo_group

`control_panel.ControlPanel.geo_group`

Initial value:

```
= QButtonGroup(
    self
)
```

8.3.4.7 iterations_input

```
control_panel.ControlPanel.iterations_input = QSpinBox()
```

8.3.4.8 log_signal

```
control_panel.ControlPanel.log_signal = Signal(str) [static]
```

Emitted when a new log entry is generated.

8.3.4.9 maxflow_group

```
control_panel.ControlPanel.maxflow_group
```

Initial value:

```
= QButtonGroup(  
    self  
)
```

8.3.4.10 mincost_group

```
control_panel.ControlPanel.mincost_group
```

Initial value:

```
= QButtonGroup(  
    self  
)
```

8.3.4.11 range_input

```
control_panel.ControlPanel.range_input = QLineEdit()
```

8.3.4.12 refresh_graph_signal

```
control_panel.ControlPanel.refresh_graph_signal = Signal() [static]
```

Emitted when the visualization requires refreshing.

8.3.4.13 result_label

```
control_panel.ControlPanel.result_label
```

Initial value:

```
= QLabel(  
    "<b>Status:</b> ---"  
)
```

8.3.4.14 save_btn

```
control_panel.ControlPanel.save_btn
```

Initial value:

```
= QPushButton(  
    "Export"  
)
```

8.3.4.15 seg_box

```
control_panel.ControlPanel.seg_box
```

Initial value:

```
= QGroupBox(  
    "Range Query Algorithms"  
)
```

8.3.4.16 seg_group

```
control_panel.ControlPanel.seg_group
```

Initial value:

```
= QButtonGroup(  
    self  
)
```

8.3.4.17 start_btn

```
control_panel.ControlPanel.start_btn
```

Initial value:

```
= QPushButton(  
    "Analyze"  
)
```

8.3.4.18 time_label

```
control_panel.ControlPanel.time_label
```

Initial value:

```
= QLabel(  
    "<b>Time:</b> ---"  
)
```

The documentation for this class was generated from the following file:

- interface/[control_panel.py](#)

8.4 data_dialog.DataDialog Class Reference

World data management dialogue.

Inheritance diagram for data_dialog.DataDialog:

Collaboration diagram for data_dialog.DataDialog:

Public Member Functions

- None [__init__](#) (self)
Creates and initializes the data dialogue.
- WorldData [get_data](#) (self)
Constructs a world model from dialogue data.

Public Attributes

- tuple [miners_table](#)
- tuple [mines_table](#)
- tuple [wardens_table](#)
- tuple [delete_miner_button](#)
- tuple [delete_mine_button](#)

Static Public Attributes

- [data_changed](#) = [Signal\(\)](#)
Emitted after world data has been modified.

Protected Member Functions

- None [_initialize_ui](#) (self)
Builds the dialogue user interface.
- None [_refresh_identifiers](#) (self)
Regenerates entity identifiers.
- list[str] [_validate](#) (self)
Validates all world data entered by the user.
- None [_add_miner](#) (self)
Adds a new miner entry to the table.
- None [_add_mine](#) (self)
Adds a new mine entry to the table.
- None [_remove_selected_rows](#) (self, QTableWidgetItem table_widget)
Removes selected rows from a table.
- None [_delete_selected_mines](#) (self)
Removes selected mines and their wardens.
- None [_update_delete_buttons](#) (self)
Updates deletion button availability.
- None [_load_to_tables](#) (self, WorldData world_data)
Loads world data into the user interface.
- None [_save](#) (self)
Validates and persists world data.

Static Protected Member Functions

- `QWidget _build_table` (list[str] headers)
Creates and configures a table widget.
- `QWidgetItem _build_identifier_item` (str text)
Creates a read-only identifier table item.
- `float _safe_float` (str input_value)
Safely converts text to a floating-point value.
- `int _safe_int` (str input_value)
Safely converts text to an integer value.
- `str _safe_str` (str input_text, str default="")
Safely normalizes text input.
- `str _get_cell_text` (QWidget table_widget, int row_index, int column_index)
Retrieves text from a table cell.
- `None _mark_cell` (QWidget table_widget, int row_index, int column_index)
Marks a table cell as invalid.
- `None _clear_mark` (QWidget table_widget, int row_index, int column_index)
Removes validation feedback from a table cell.

8.4.1 Detailed Description

World data management dialogue.

Provides an interactive interface for managing miners, mines, and wardens. The dialogue allows users to create, modify, remove, and persist world entities used by the optimization engine.

8.4.2 Constructor & Destructor Documentation

8.4.2.1 __init__()

```
None data_dialog.DataDialog.__init__ (
    self)
```

Creates and initializes the data dialogue.

Configures the dialogue window, builds the user interface, loads persisted world data, and populates all data tables. Here is the call graph for this function: Here is the caller graph for this function:

8.4.3 Member Function Documentation

8.4.3.1 _add_mine()

```
None data_dialog.DataDialog._add_mine (
    self) [protected]
```

Adds a new mine entry to the table.

Creates a new mine row, assigns a unique identifier, initializes resource selection controls, and creates the corresponding warden entry associated with the mine. Here is the call graph for this function: Here is the caller graph for this function:

8.4.3.2 _add_miner()

```
None data_dialog.DataDialog._add_miner (  
    self) [protected]
```

Adds a new miner entry to the table.

Creates a new miner row, assigns a unique identifier, initializes default values, and configures the resource selection widget. Here is the call graph for this function: Here is the caller graph for this function:

8.4.3.3 _build_identifier_item()

```
QTableWidgetItem data_dialog.DataDialog._build_identifier_item (  
    str text) [static], [protected]
```

Creates a read-only identifier table item.

Generates a bold table item intended for displaying immutable entity identifiers.

Parameters

<i>text</i>	Identifier text displayed by the item.
-------------	--

Returns

QTableWidgetItem Configured identifier table item.

Here is the caller graph for this function:

8.4.3.4 _build_table()

```
QTableWidget data_dialog.DataDialog._build_table (  
    list[str] headers) [static], [protected]
```

Creates and configures a table widget.

Initializes a table with the specified column headers and configures row-based multi-selection.

Parameters

<i>headers</i>	Column header labels displayed by the table.
----------------	--

Returns

QTableWidget Configured table widget instance.

8.4.3.5 `_clear_mark()`

```
None data_dialog.DataDialog._clear_mark (
    QTableWidgetItem table_widget,
    int row_index,
    int column_index) [static], [protected]
```

Removes validation feedback from a table cell.

Restores the default appearance of the specified cell and removes any validation tooltip previously applied.

Parameters

<i>table_widget</i>	Target table widget.
<i>row_index</i>	Index of the target row.
<i>column_index</i>	Index of the target column.

Here is the caller graph for this function:

8.4.3.6 `_delete_selected_mines()`

```
None data_dialog.DataDialog._delete_selected_mines (
    self) [protected]
```

Removes selected mines and their wardens.

Deletes selected mine entries together with their corresponding warden records to preserve data consistency across related tables. Here is the call graph for this function:

8.4.3.7 `_get_cell_text()`

```
str data_dialog.DataDialog._get_cell_text (
    QTableWidgetItem table_widget,
    int row_index,
    int column_index) [static], [protected]
```

Retrieves text from a table cell.

Returns an empty string when the specified cell does not contain a table item.

Parameters

<i>table_widget</i>	Source table widget.
<i>row_index</i>	Index of the target row.
<i>column_index</i>	Index of the target column.

Returns

str Cell text or an empty string.

Here is the caller graph for this function:

8.4.3.8 _initialize_ui()

```
None data_dialog.DataDialog._initialize_ui (
    self) [protected]
```

Builds the dialogue user interface.

Creates all layouts, tables, labels, and action buttons used for managing world data. The method also establishes signal connections and configures default widget states. Here is the caller graph for this function:

8.4.3.9 _load_to_tables()

```
None data_dialog.DataDialog._load_to_tables (
    self,
    WorldData world_data) [protected]
```

Loads world data into the user interface.

Populates miner, mine, and warden tables using data obtained from the specified world model. Existing entities are reconstructed together with their resources, capacities, positions, and associated warden information.

Parameters

<i>world_data</i>	World data model used to populate the dialogue.
-------------------	---

Here is the call graph for this function: Here is the caller graph for this function:

8.4.3.10 _mark_cell()

```
None data_dialog.DataDialog._mark_cell (
    QTableWidgetItem table_widget,
    int row_index,
    int column_index) [static], [protected]
```

Marks a table cell as invalid.

Applies visual validation feedback to the specified cell by changing its background colour and displaying an explanatory tooltip message.

Parameters

<i>table_widget</i>	Target table widget.
<i>row_index</i>	Index of the target row.
<i>column_index</i>	Index of the target column.

Here is the caller graph for this function:

8.4.3.11 _refresh_identifiers()

```
None data_dialog.DataDialog._refresh_identifiers (
    self) [protected]
```

Regenerates entity identifiers.

Updates miner, mine, and warden identifiers to ensure sequential numbering after rows have been added, removed, or reordered. Here is the call graph for this function: Here is the caller graph for this function:

8.4.3.12 _remove_selected_rows()

```
None data_dialog.DataDialog._remove_selected_rows (
    self,
    QTableWidgetItem table_widget) [protected]
```

Removes selected rows from a table.

Deletes all currently selected rows from the specified table, refreshes entity identifiers, and updates the state of deletion controls.

Parameters

<i>table_widget</i>	Table containing rows selected for removal.
---------------------	---

Here is the call graph for this function:

8.4.3.13 _safe_float()

```
float data_dialog.DataDialog._safe_float (
    str input_value) [static], [protected]
```

Safely converts text to a floating-point value.

Returns zero when the provided value cannot be converted to a valid floating-point number.

Parameters

<i>input_value</i>	Text representation of a numeric value.
--------------------	---

Returns

float Parsed floating-point value or zero.

Here is the caller graph for this function:

8.4.3.14 `_safe_int()`

```
int data_dialog.DataDialog._safe_int (
    str input_value) [static], [protected]
```

Safely converts text to an integer value.

Returns zero when the provided value cannot be converted to a valid integer.

Parameters

<i>input_value</i>	Text representation of a numeric value.
--------------------	---

Returns

int Parsed integer value or zero.

Here is the caller graph for this function:

8.4.3.15 `_safe_str()`

```
str data_dialog.DataDialog._safe_str (
    str input_text,
    str default = "") [static], [protected]
```

Safely normalizes text input.

Removes surrounding whitespace and returns a fallback value when the resulting text is empty.

Parameters

<i>input_text</i>	Text value to normalize.
<i>default</i>	Value returned when the input is empty.

Returns

str Normalized text value.

Here is the caller graph for this function:

8.4.3.16 `_save()`

```
None data_dialog.DataDialog._save (
    self) [protected]
```

Validates and persists world data.

Executes data validation, displays validation feedback when errors are detected, persists the current world state, emits the data change notification signal, and closes the dialogue. Here is the call graph for this function:

8.4.3.17 `_update_delete_buttons()`

```
None data_dialog.DataDialog._update_delete_buttons (
    self) [protected]
```

Updates deletion button availability.

Enables or disables deletion controls according to the current selection state of miner and mine tables. Here is the caller graph for this function:

8.4.3.18 `_validate()`

```
list[str] data_dialog.DataDialog._validate (
    self) [protected]
```

Validates all world data entered by the user.

Performs structural, numerical, and geometric validation of miners, mines, and wardens.

The validation process verifies:

- minimum entity count requirements,
- mine geometry constraints,
- coordinate completeness,
- mine capacity values,
- warden alert volume values.

Invalid cells are highlighted and validation states are propagated to the corresponding user interface widgets.

Returns

list[str] Collection of validation error messages.

Here is the call graph for this function: Here is the caller graph for this function:

8.4.3.19 `get_data()`

```
WorldData data_dialog.DataDialog.get_data (
    self)
```

Constructs a world model from dialogue data.

Extracts miners, mines, and wardens from the user interface tables and converts them into domain objects used by the optimization engine.

Resource selections, coordinates, capacities, and warden assignments are transformed into a complete WorldData instance.

Returns

WorldData World model reconstructed from the current dialogue state.

Here is the call graph for this function: Here is the caller graph for this function:

8.4.4 Member Data Documentation

8.4.4.1 data_changed

`data_dialog.DataDialog.data_changed = Signal() [static]`

Emitted after world data has been modified.

8.4.4.2 delete_mine_button

`data_dialog.DataDialog.delete_mine_button`

Initial value:

```
= (
    QPushButton("Abandon Mine")
)
```

8.4.4.3 delete_miner_button

`data_dialog.DataDialog.delete_miner_button`

Initial value:

```
= (
    QPushButton("Dismiss Miner")
)
```

8.4.4.4 miners_table

`data_dialog.DataDialog.miners_table`

Initial value:

```
= (
    self._build_table(
        [
            "Identifier",
            "Name",
            "Resource",
            "Position X",
            "Position Y",
        ]
    )
)
```

8.4.4.5 mines_table

`data_dialog.DataDialog.mines_table`

Initial value:

```
= (
    self._build_table(
        [
            "Identifier",
            "Resource",
            "Resource Capacity",
            "Position X",
            "Position Y",
        ]
    )
)
```

8.4.4.6 wardens_table

`data_dialog.DataDialog.wardens_table`

Initial value:

```
= (
    self._build_table(
        [
            "Identifier",
            "Alert Volume",
        ]
    )
)
```

The documentation for this class was generated from the following file:

- [interface/data_dialog.py](#)

8.5 DEsopoPapeStrategy Class Reference

D'Esopo-Pape shortest-path strategy.

```
#include <desopopape_strategy.hpp>
```

Inheritance diagram for DEsopoPapeStrategy:

Collaboration diagram for DEsopoPapeStrategy:

Public Member Functions

- `bool findPath (FlowNetwork &graph, int source, int sink, vector< int > &parentVertex, vector< size_t > &parentEdge, vector< int > &distance) override`
Finds a minimum-cost augmenting path.

Public Member Functions inherited from IShortestPathStrategy

- `virtual ~IShortestPathStrategy ()=default`

8.5.1 Detailed Description

D'Esopo-Pape shortest-path strategy.

Computes shortest paths in the residual network using a deque-based relaxation mechanism that often performs efficiently on sparse graphs.

This strategy is used during Minimum-Cost Maximum-Flow optimization.

8.5.2 Member Function Documentation

8.5.2.1 findPath()

```
bool DEsopoPapeStrategy::findPath (
    FlowNetwork & graph,
    int source,
    int sink,
    vector< int > & parentVertex,
    vector< size_t > & parentEdge,
    vector< int > & distance) [inline], [override], [virtual]
```

Finds a minimum-cost augmenting path.

Executes the D'Esopo-Pape shortest-path algorithm on the residual network and reconstructs the resulting path.

Parameters

<i>graph</i>	Residual flow network.
<i>source</i>	Source vertex.
<i>sink</i>	Sink vertex.
<i>parentVertex</i>	Parent vertex table used for path reconstruction.
<i>parentEdge</i>	Parent edge table used for path reconstruction.
<i>distance</i>	Computed shortest-path distances.

Returns

bool True when a valid augmenting path exists.

Implements [IShortestPathStrategy](#).

The documentation for this class was generated from the following file:

- [engine/engines/network_flow/desopopape_strategy.hpp](#)

8.6 models.Dwarf Class Reference

Base dwarf entity.

Inheritance diagram for models.Dwarf:

8.6.1 Detailed Description

Base dwarf entity.

Represents a named entity occupying a position within the world.

The documentation for this class was generated from the following file:

- [domain/models.py](#)

8.7 Edge Struct Reference

Residual network edge.

```
#include <flow_network.hpp>
```

Public Attributes

- `size_t` [destination](#)
- `size_t` [reverseEdgeIndex](#)
- `int` [capacity](#)
- `int` [cost](#)

8.7.1 Detailed Description

Residual network edge.

Represents a directed edge within the residual flow network together with its reverse edge reference.

8.7.2 Member Data Documentation

8.7.2.1 capacity

```
int Edge::capacity
```

8.7.2.2 cost

```
int Edge::cost
```

8.7.2.3 destination

```
size_t Edge::destination
```

8.7.2.4 reverseEdgeIndex

```
size_t Edge::reverseEdgeIndex
```

The documentation for this struct was generated from the following file:

- `engine/engines/network_flow/`[flow_network.hpp](#)

8.8 EdmondsKarpStrategy Class Reference

Edmonds-Karp flow augmentation strategy.

```
#include <edmonds_karp_strategy.hpp>
```

Inheritance diagram for EdmondsKarpStrategy:

Collaboration diagram for EdmondsKarpStrategy:

Public Member Functions

- int [computeFlow](#) ([FlowNetwork](#) &graph, int source, int sink, vector< int > &parentVertex, vector< size_t > &parentEdge) override
Computes bottleneck flow capacity.
- void [augment](#) ([FlowNetwork](#) &graph, int source, int sink, vector< int > &parentVertex, vector< size_t > &parentEdge, int flow, int &totalCost) override
Augments the residual network.
- int [calculatePathLength](#) (int source, int sink, const vector< int > &parentVertex) override
Calculates augmenting path length.

Public Member Functions inherited from [IMaxFlowStrategy](#)

- virtual [~IMaxFlowStrategy](#) ()=default

8.8.1 Detailed Description

Edmonds-Karp flow augmentation strategy.

Computes augmenting flow values and updates the residual network using paths discovered through breadth-first search.

Complexity per augmentation: $O(L)$

where L is the length of the augmenting path.

8.8.2 Member Function Documentation

8.8.2.1 augment()

```
void EdmondsKarpStrategy::augment (
    FlowNetwork & graph,
    int source,
    int sink,
    vector< int > & parentVertex,
    vector< size_t > & parentEdge,
    int flow,
    int & totalCost) [inline], [override], [virtual]
```

Augments the residual network.

Updates forward and reverse edge capacities along the augmenting path and accumulates the corresponding transportation cost.

Parameters

<i>graph</i>	Residual flow network.
<i>source</i>	Source vertex.
<i>sink</i>	Sink vertex.
<i>parentVertex</i>	Parent vertex table describing the path.
<i>parentEdge</i>	Parent edge table describing the path.
<i>flow</i>	Flow value to augment.
<i>totalCost</i>	Accumulated transportation cost.

Implements [IMaxFlowStrategy](#).

8.8.2.2 calculatePathLength()

```
int EdmondsKarpStrategy::calculatePathLength (
    int source,
    int sink,
    const vector< int > & parentVertex) [inline], [override], [virtual]
```

Calculates augmenting path length.

Counts the number of edges forming the reconstructed source-to-sink path.

Parameters

<i>source</i>	Source vertex.
<i>sink</i>	Sink vertex.
<i>parentVertex</i>	Parent vertex table describing the path.

Returns

int Number of edges in the augmenting path.

Reimplemented from [IMaxFlowStrategy](#).

8.8.2.3 computeFlow()

```
int EdmondsKarpStrategy::computeFlow (
    FlowNetwork & graph,
    int source,
    int sink,
    vector< int > & parentVertex,
    vector< size_t > & parentEdge) [inline], [override], [virtual]
```

Computes bottleneck flow capacity.

Traverses the augmenting path and determines the minimum residual capacity available along the route.

Parameters

<i>graph</i>	Residual flow network.
<i>source</i>	Source vertex.
<i>sink</i>	Sink vertex.
<i>parentVertex</i>	Parent vertex table describing the path.
<i>parentEdge</i>	Parent edge table describing the path.

Returns

int Maximum admissible augmenting flow.

Implements [IMaxFlowStrategy](#).

The documentation for this class was generated from the following file:

- [engine/engines/network_flow/edmonds_karp_strategy.hpp](#)

8.9 FlowOptimizer Class Reference

Minimum-Cost Maximum-Flow optimizer.

```
#include <flow_optimizer.hpp>
```

Collaboration diagram for FlowOptimizer:

Public Member Functions

- [FlowOptimizer](#) (size_t vertices, [IMaxFlowStrategy](#) *maxflowStrategy, [IShortestPathStrategy](#) *mincostStrategy)
Creates a flow optimizer.
- void [addEdge](#) (size_t from, size_t to, int capacity, int cost)
Adds a directed edge to the network.
- [FlowStatistics](#) solve (int source, int sink)
Solves the Minimum-Cost Maximum-Flow problem.
- const [FlowNetwork](#) & [getGraph](#) () const
Returns the residual flow network.

Private Attributes

- [FlowNetwork flowNetwork](#)
Residual flow network.
- [IMaxFlowStrategy * maxflowStrategy](#)
Maximum-flow strategy.
- [IShortestPathStrategy * mincostStrategy](#)
Shortest-path strategy.

8.9.1 Detailed Description

Minimum-Cost Maximum-Flow optimizer.

Maintains a residual flow network and coordinates shortest-path and maximum-flow strategies to compute an optimal resource allocation.

8.9.2 Constructor & Destructor Documentation

8.9.2.1 FlowOptimizer()

```
FlowOptimizer::FlowOptimizer (
    size_t vertices,
    IMaxFlowStrategy * maxflowStrategy,
    IShortestPathStrategy * mincostStrategy) [inline]
```

Creates a flow optimizer.

Initializes the flow network and attaches the selected maximum-flow and shortest-path strategies.

Parameters

<i>vertices</i>	Number of vertices in the flow network.
<i>maxflowStrategy</i>	Maximum-flow strategy implementation.
<i>mincostStrategy</i>	Shortest-path strategy implementation.

8.9.3 Member Function Documentation

8.9.3.1 addEdge()

```
void FlowOptimizer::addEdge (
    size_t from,
    size_t to,
    int capacity,
    int cost) [inline]
```

Adds a directed edge to the network.

Creates both forward and residual edges and inserts them into the residual graph.

Parameters

<i>from</i>	Source vertex.
<i>to</i>	Destination vertex.
<i>capacity</i>	Edge capacity.
<i>cost</i>	Edge traversal cost.

Here is the caller graph for this function:

8.9.3.2 getGraph()

```
const FlowNetwork & FlowOptimizer::getGraph () const [inline]
```

Returns the residual flow network.

Provides read-only access to the current network state after optimization.

Returns

const [FlowNetwork](#)& Residual flow network.

Here is the caller graph for this function:

8.9.3.3 solve()

```
FlowStatistics FlowOptimizer::solve (  
    int source,  
    int sink) [inline]
```

Solves the Minimum-Cost Maximum-Flow problem.

Repeatedly searches for augmenting paths, computes admissible flow values, updates the residual network, and gathers execution statistics until no additional path exists.

Parameters

<i>source</i>	Source vertex.
<i>sink</i>	Sink vertex.

Returns

[FlowStatistics](#) Optimization result and performance metrics.

Here is the call graph for this function: Here is the caller graph for this function:

8.9.4 Member Data Documentation

8.9.4.1 flowNetwork

`FlowNetwork` FlowOptimizer::flowNetwork [private]

Residual flow network.

8.9.4.2 maxflowStrategy

`IMaxFlowStrategy*` FlowOptimizer::maxflowStrategy [private]

Maximum-flow strategy.

8.9.4.3 mincostStrategy

`IShortestPathStrategy*` FlowOptimizer::mincostStrategy [private]

Shortest-path strategy.

The documentation for this class was generated from the following file:

- engine/engines/network_flow/[flow_optimizer.hpp](#)

8.10 FlowStatistics Struct Reference

Flow optimization statistics.

```
#include <flow_optimizer.hpp>
```

Public Attributes

- int `maximumFlow` {0}
- int `minimumCost` {0}
- int `augmentationCount` {0}
- double `averagePathLength` {0.0}
- int `totalPathLength` {0}
- double `pathfindingTimeMs` {0.0}
- double `augmentationTimeMs` {0.0}
- double `totalTimeMs` {0.0}

8.10.1 Detailed Description

Flow optimization statistics.

Stores optimization results, execution metrics, and benchmarking information generated during a Minimum-Cost Maximum-Flow computation.

8.10.2 Member Data Documentation

8.10.2.1 augmentationCount

```
int FlowStatistics::augmentationCount {0}
```

8.10.2.2 augmentationTimeMs

```
double FlowStatistics::augmentationTimeMs {0.0}
```

8.10.2.3 averagePathLength

```
double FlowStatistics::averagePathLength {0.0}
```

8.10.2.4 maximumFlow

```
int FlowStatistics::maximumFlow {0}
```

8.10.2.5 minimumCost

```
int FlowStatistics::minimumCost {0}
```

8.10.2.6 pathfindingTimeMs

```
double FlowStatistics::pathfindingTimeMs {0.0}
```

8.10.2.7 totalPathLength

```
int FlowStatistics::totalPathLength{0}
```

8.10.2.8 totalTimeMs

```
double FlowStatistics::totalTimeMs {0.0}
```

The documentation for this struct was generated from the following file:

- engine/engines/network_flow/[flow_optimizer.hpp](#)

8.11 FordFulkersonStrategy Class Reference

Ford-Fulkerson flow augmentation strategy.

```
#include <ford_fulkerson_strategy.hpp>
```

Inheritance diagram for FordFulkersonStrategy:

Collaboration diagram for FordFulkersonStrategy:

Public Member Functions

- int [computeFlow](#) ([FlowNetwork](#) &graph, int source, int sink, vector< int > &parentVertex, vector< size_t > &parentEdge) override
Computes bottleneck flow capacity.
- void [augment](#) ([FlowNetwork](#) &graph, int source, int sink, vector< int > &parentVertex, vector< size_t > &parentEdge, int flow, int &totalCost) override
Augments the residual network.

Public Member Functions inherited from [IMaxFlowStrategy](#)

- virtual int [calculatePathLength](#) (int source, int sink, const vector< int > &parentVertex)
Calculates augmenting path length.
- virtual [~IMaxFlowStrategy](#) ()=default

8.11.1 Detailed Description

Ford-Fulkerson flow augmentation strategy.

Computes augmenting flow values and updates the residual network using the classical Ford-Fulkerson approach.

Complexity per augmentation: $O(L)$

where L is the length of the augmenting path.

8.11.2 Member Function Documentation

8.11.2.1 [augment\(\)](#)

```
void FordFulkersonStrategy::augment (
    FlowNetwork & graph,
    int source,
    int sink,
    vector< int > & parentVertex,
    vector< size_t > & parentEdge,
    int flow,
    int & totalCost) [inline], [override], [virtual]
```

Augments the residual network.

Updates forward and reverse edge capacities along the augmenting path and accumulates the corresponding transportation cost.

Parameters

<i>graph</i>	Residual flow network.
<i>source</i>	Source vertex.
<i>sink</i>	Sink vertex.
<i>parentVertex</i>	Parent vertex table describing the path.
<i>parentEdge</i>	Parent edge table describing the path.
<i>flow</i>	Flow value to augment.
<i>totalCost</i>	Accumulated transportation cost.

Implements [IMaxFlowStrategy](#).

8.11.2.2 computeFlow()

```
int FordFulkersonStrategy::computeFlow (
    FlowNetwork & graph,
    int source,
    int sink,
    vector< int > & parentVertex,
    vector< size_t > & parentEdge) [inline], [override], [virtual]
```

Computes bottleneck flow capacity.

Traverses the augmenting path and determines the minimum residual capacity available along the route.

Parameters

<i>graph</i>	Residual flow network.
<i>source</i>	Source vertex.
<i>sink</i>	Sink vertex.
<i>parentVertex</i>	Parent vertex table describing the path.
<i>parentEdge</i>	Parent edge table describing the path.

Returns

int Maximum admissible augmenting flow.

Implements [IMaxFlowStrategy](#).

The documentation for this class was generated from the following file:

- engine/engines/network_flow/[ford_fulkerson_strategy.hpp](#)

8.12 huffman_codec.HuffmanNode Class Reference

Node of a Huffman coding tree.

Collaboration diagram for huffman_codec.HuffmanNode:

Public Member Functions

- None `__init__` (self, str|None symbol=None, int weight=0)
Creates a Huffman tree node.
- bool `__lt__` (self, "HuffmanNode" other)
Compares nodes by weight.

Public Attributes

- tuple `symbol`
- tuple `weight`
- `HuffmanNode`|None `left_branch` = None
- `HuffmanNode`|None `right_branch` = None

8.12.1 Detailed Description

Node of a Huffman coding tree.

Represents either a leaf node containing a symbol or an internal node used during tree construction.

8.12.2 Constructor & Destructor Documentation

8.12.2.1 `__init__()`

```
None huffman_codec.HuffmanNode.__init__ (
    self,
    str | None symbol = None,
    int weight = 0)
```

Creates a Huffman tree node.

Parameters

<i>symbol</i>	Symbol stored by the node. Internal nodes do not contain a symbol.
<i>weight</i>	Symbol frequency or subtree weight.

8.12.3 Member Function Documentation

8.12.3.1 `__lt__()`

```
bool huffman_codec.HuffmanNode.__lt__ (
    self,
    "HuffmanNode" other)
```

Compares nodes by weight.

Enables priority queue ordering during Huffman tree construction.

Parameters

<i>other</i>	Node used for comparison.
--------------	---------------------------

Returns

bool True if the current node has lower weight.

8.12.4 Member Data Documentation

8.12.4.1 `left_branch`

[HuffmanNode](#) | None `huffman_codec.HuffmanNode.left_branch = None`

8.12.4.2 `right_branch`

[HuffmanNode](#) | None `huffman_codec.HuffmanNode.right_branch = None`

8.12.4.3 `symbol`

`tuple huffman_codec.HuffmanNode.symbol`

Initial value:

```
= (
    symbol
)
```

8.12.4.4 `weight`

`tuple huffman_codec.HuffmanNode.weight`

Initial value:

```
= (
    weight
)
```

The documentation for this class was generated from the following file:

- [engine/compression/huffman_codec.py](#)

8.13 IMaxFlowStrategy Class Reference

Maximum-flow strategy interface.

```
#include <imaxflow_path_strategy.hpp>
```

Inheritance diagram for IMaxFlowStrategy:

Public Member Functions

- virtual int [computeFlow](#) ([FlowNetwork](#) &flowNetwork, int source, int sink, vector< int > &parentVertex, vector< size_t > &parentEdge)=0
Computes an augmenting flow value.
- virtual void [augment](#) ([FlowNetwork](#) &flowNetwork, int source, int sink, vector< int > &parentVertex, vector< size_t > &parentEdge, int flow, int &totalCost)=0
Augments the residual network.
- virtual int [calculatePathLength](#) (int source, int sink, const vector< int > &parentVertex)
Calculates augmenting path length.
- virtual [~IMaxFlowStrategy](#) ()=default

8.13.1 Detailed Description

Maximum-flow strategy interface.

Provides a common abstraction for flow augmentation algorithms used during Minimum-Cost Maximum-Flow optimization.

Implementations:

- [FordFulkersonStrategy](#)
- [EdmondsKarpStrategy](#)

8.13.2 Constructor & Destructor Documentation

8.13.2.1 ~IMaxFlowStrategy()

```
virtual IMaxFlowStrategy::~IMaxFlowStrategy () [virtual], [default]
```


8.13.3 Member Function Documentation

8.13.3.1 augment()

```
virtual void IMaxFlowStrategy::augment (
    FlowNetwork & flowNetwork,
    int source,
    int sink,
    vector< int > & parentVertex,
    vector< size_t > & parentEdge,
    int flow,
    int & totalCost) [pure virtual]
```

Augments the residual network.

Updates residual capacities along the augmenting path and accumulates the resulting transportation cost.

Parameters

<i>flowNetwork</i>	Residual flow network.
<i>source</i>	Source vertex.
<i>sink</i>	Sink vertex.
<i>parentVertex</i>	Parent vertex table describing the path.
<i>parentEdge</i>	Parent edge table describing the path.
<i>flow</i>	Flow value to augment.
<i>totalCost</i>	Accumulated transportation cost.

Implemented in [EdmondsKarpStrategy](#), and [FordFulkersonStrategy](#).

8.13.3.2 calculatePathLength()

```
virtual int IMaxFlowStrategy::calculatePathLength (
    int source,
    int sink,
    const vector< int > & parentVertex) [inline], [virtual]
```

Calculates augmenting path length.

Traverses the reconstructed path and returns the number of edges between the source and sink.

Parameters

<i>source</i>	Source vertex.
<i>sink</i>	Sink vertex.
<i>parentVertex</i>	Parent vertex table describing the path.

Returns

int Number of edges forming the path.

Reimplemented in [EdmondsKarpStrategy](#).

8.13.3.3 computeFlow()

```
virtual int IMaxFlowStrategy::computeFlow (
    FlowNetwork & flowNetwork,
    int source,
    int sink,
    vector< int > & parentVertex,
    vector< size_t > & parentEdge) [pure virtual]
```

Computes an augmenting flow value.

Determines the maximum admissible flow that can be sent through the currently discovered augmenting path.

Parameters

<i>flowNetwork</i>	Residual flow network.
<i>source</i>	Source vertex.
<i>sink</i>	Sink vertex.
<i>parentVertex</i>	Parent vertex table describing the path.
<i>parentEdge</i>	Parent edge table describing the path.

Returns

int Augmenting flow value.

Implemented in [EdmondsKarpStrategy](#), and [FordFulkersonStrategy](#).

The documentation for this class was generated from the following file:

- [engine/engines/network_flow/imaxflow_path_strategy.hpp](#)

8.14 serializer.InputParser Class Reference

World data text parser.

Static Public Member Functions

- WorldData [parse](#) (str text)
Parses world data from text.

8.14.1 Detailed Description

World data text parser.

Parses a structured text representation of miners and mines and converts it into a WorldData instance.

8.14.2 Member Function Documentation

8.14.2.1 parse()

```
WorldData serializer.InputParser.parse (
    str text) [static]
```

Parses world data from text.

Reads miners and mines defined in a structured text format and converts them into domain objects.

Supported sections:

- MINERS
- MINES

Parameters

<i>text</i>	Input text containing world data definitions.
-------------	---

Returns

WorldData Parsed world model.

Exceptions

<i>ValueError</i>	Raised when the input format is invalid or a data entry does not conform to the expected structure.
-------------------	---

The documentation for this class was generated from the following file:

- domain/[serializer.py](#)

8.15 IShortestPathStrategy Class Reference

Shortest-path strategy interface.

```
#include <ishortest_path_strategy.hpp>
```

Inheritance diagram for IShortestPathStrategy:

Public Member Functions

- virtual bool [findPath](#) ([FlowNetwork](#) &flowNetwork, int source, int sink, vector< int > &parentVertex, vector< size_t > &parentEdge, vector< int > &distance)=0
Searches for an augmenting path.
- virtual [~IShortestPathStrategy](#) ()=default

8.15.1 Detailed Description

Shortest-path strategy interface.

Provides a common abstraction for pathfinding algorithms used during Minimum-Cost Maximum-Flow optimization.

8.15.2 Constructor & Destructor Documentation

8.15.2.1 ~IShortestPathStrategy()

```
virtual IShortestPathStrategy::~IShortestPathStrategy () [virtual], [default]
```

8.15.3 Member Function Documentation

8.15.3.1 findPath()

```
virtual bool IShortestPathStrategy::findPath (
    FlowNetwork & flowNetwork,
    int source,
    int sink,
    vector< int > & parentVertex,
    vector< size_t > & parentEdge,
    vector< int > & distance) [pure virtual]
```

Searches for an augmenting path.

Executes a shortest-path algorithm on the residual flow network and stores the resulting path information.

Parameters

<i>flowNetwork</i>	Residual flow network.
<i>source</i>	Source vertex.
<i>sink</i>	Sink vertex.
<i>parentVertex</i>	Parent vertex table used for path reconstruction.
<i>parentEdge</i>	Parent edge table used for path reconstruction.
<i>distance</i>	Computed shortest-path distances.

Returns

bool True when a valid augmenting path exists.

Implemented in [BellmanFordStrategy](#), and [DEsopoPapeStrategy](#).

The documentation for this class was generated from the following file:

- [engine/engines/network_flow/ishortest_path_strategy.hpp](#)

8.16 main_window.MainWindow Class Reference

Main application window.

Inheritance diagram for main_window.MainWindow:

Collaboration diagram for main_window.MainWindow:

Public Member Functions

- None `__init__` (self)
Creates and initializes the main application window.
- None `resizeEvent` (self, QResizeEvent event)
Handles window resize events.

Public Attributes

- QWidget `background_widget` = QWidget()
- ContentPanel `content_panel` = ContentPanel()
- ControlPanel `control_panel` = ControlPanel()

Protected Member Functions

- None `_initialize_ui` (self)
Builds the user interface layout.
- None `_handle_log` (self, str log_entry)
Processes log messages emitted by the control panel.

8.16.1 Detailed Description

Main application window.

Serves as the central container of the graphical user interface. The window coordinates communication between the content and control panels, manages application-wide layout configuration, and applies responsive visual adjustments during resizing.

8.16.2 Constructor & Destructor Documentation

8.16.2.1 `__init__`()

```
None main_window.MainWindow.__init__ (
    self)
```

Creates and initializes the main application window.

Configures the window title, icon, dimensions, and initializes the graphical user interface.

Returns

None

Here is the call graph for this function: Here is the caller graph for this function:

8.16.3 Member Function Documentation

8.16.3.1 `_handle_log()`

```
None main_window.MainWindow._handle_log (  
    self,  
    str log_entry) [protected]
```

Processes log messages emitted by the control panel.

Forwards log entries to the content panel for visualization within the application log view.

Parameters

<i>log_entry</i>	Log message generated by the control panel.
------------------	---

Returns

None

8.16.3.2 `_initialize_ui()`

```
None main_window.MainWindow._initialize_ui (  
    self) [protected]
```

Builds the user interface layout.

Creates the root container, initializes content and control panels, establishes signal connections, and configures the main application layout.

Returns

None

Here is the caller graph for this function:

8.16.3.3 `resizeEvent()`

```
None main_window.MainWindow.resizeEvent (  
    self,  
    QResizeEvent event)
```

Handles window resize events.

Dynamically adjusts button dimensions, font sizes, spacing, and border radii to maintain a consistent visual appearance across different window sizes.

Parameters

<i>event</i>	Qt resize event containing the updated window size.
--------------	---

Returns

None

Here is the call graph for this function: Here is the caller graph for this function:

8.16.4 Member Data Documentation**8.16.4.1 background_widget**

```
main_window.MainWindow.background_widget = QWidget()
```

8.16.4.2 content_panel

```
main_window.MainWindow.content_panel = ContentPanel()
```

8.16.4.3 control_panel

```
main_window.MainWindow.control_panel = ControlPanel()
```

The documentation for this class was generated from the following file:

- [interface/main_window.py](#)

8.17 Mine Struct Reference

```
#include <range_utils.hpp>
```

Collaboration diagram for Mine:

Public Attributes

- string [id](#)
- int [loudness](#) {0}
- [Vertex](#) [vertex](#)

8.17.1 Member Data Documentation**8.17.1.1 id**

```
string Mine::id
```

8.17.1.2 loudness

```
int Mine::loudness {0}
```

8.17.1.3 vertex

```
Vertex Mine::vertex
```

The documentation for this struct was generated from the following file:

- engine/engines/range_queries/[range_utils.hpp](#)

8.18 models.Mine Class Reference

Mining site.

8.18.1 Detailed Description

Mining site.

Represents a resource deposit together with its extraction capacity and assigned warden.

The documentation for this class was generated from the following file:

- domain/[models.py](#)

8.19 models.Miner Class Reference

Mining unit.

Inheritance diagram for models.Miner:

Collaboration diagram for models.Miner:

Static Public Attributes

- str [resource](#) = ""

8.19.1 Detailed Description

Mining unit.

Represents a dwarf assigned to collect a specific resource type.

8.19.2 Member Data Documentation

8.19.2.1 resource

```
str models.Miner.resource = "" [static]
```

The documentation for this class was generated from the following file:

- domain/[models.py](#)

8.20 models.Point Class Reference

Two-dimensional point.

8.20.1 Detailed Description

Two-dimensional point.

Represents a position within the world coordinate system used by miners, wardens, and mines.

The documentation for this class was generated from the following file:

- domain/[models.py](#)

8.21 Stopwatch Class Reference

High-resolution stopwatch.

```
#include <core_utils_stopwatch.hpp>
```

Public Member Functions

- [Stopwatch](#) ()
Creates and starts the stopwatch.
- void [restart](#) ()
Restarts the stopwatch.
- double [elapsedMilliseconds](#) () const
Returns elapsed time in milliseconds.

Private Attributes

- high_resolution_clock::time_point [startTime](#)
Timestamp marking the beginning of the measurement.

8.21.1 Detailed Description

High-resolution stopwatch.

Measures elapsed execution time using the C++ standard high-resolution clock.

8.21.2 Constructor & Destructor Documentation

8.21.2.1 Stopwatch()

```
Stopwatch::Stopwatch () [inline]
```

Creates and starts the stopwatch.

Initializes the starting timestamp to the current system time.

8.21.3 Member Function Documentation

8.21.3.1 elapsedMilliseconds()

```
double Stopwatch::elapsedMilliseconds () const [inline]
```

Returns elapsed time in milliseconds.

Calculates the duration between the current time and the most recent start or restart.

Returns

double Elapsed time in milliseconds.

Here is the caller graph for this function:

8.21.3.2 restart()

```
void Stopwatch::restart () [inline]
```

Restarts the stopwatch.

Resets the starting timestamp to the current time, discarding any previously measured interval. Here is the caller graph for this function:

8.21.4 Member Data Documentation

8.21.4.1 startTime

```
high_resolution_clock::time_point Stopwatch::startTime [private]
```

Timestamp marking the beginning of the measurement.

The documentation for this class was generated from the following file:

- engine/foundation/[core_utils_stopwatch.hpp](#)

8.22 Vertex Struct Reference

Two-dimensional geometric point.

```
#include <geometry_utils.hpp>
```

Public Attributes

- string `id`
- double `x` {0.0}
- double `y` {0.0}

8.22.1 Detailed Description

Two-dimensional geometric point.

Represents a labeled vertex used by computational geometry algorithms.

8.22.2 Member Data Documentation

8.22.2.1 `id`

```
string Vertex::id
```

8.22.2.2 `x`

```
double Vertex::x {0.0}
```

8.22.2.3 `y`

```
double Vertex::y {0.0}
```

The documentation for this struct was generated from the following file:

- [engine/engines/convex_hull/geometry_utils.hpp](#)

8.23 models.Warden Class Reference

Defensive unit.

Inheritance diagram for `models.Warden`:

Collaboration diagram for `models.Warden`:

8.23.1 Detailed Description

Defensive unit.

Represents a dwarf responsible for protecting a mining site and monitoring its surroundings.

The documentation for this class was generated from the following file:

- domain/[models.py](#)

8.24 models.WorldData Class Reference

World state container.

Public Member Functions

- dict[str, list[dict[str, object]]] [to_dict](#) (self)
Converts world data into a serializable dictionary.

Static Public Attributes

- list [miners](#) [[Miner](#)]
- list [mines](#) [[Mine](#)]

8.24.1 Detailed Description

World state container.

Aggregates all miners and mines currently participating in the simulation.

8.24.2 Member Function Documentation

8.24.2.1 to_dict()

```
dict[str, list[dict[str, object]]] models.WorldData.to_dict (  
    self)
```

Converts world data into a serializable dictionary.

Transforms domain objects into a structure suitable for JSON serialization and persistent storage.

Returns

dict[str, list[dict[str, object]]] Dictionary representation of the world state.

8.24.3 Member Data Documentation

8.24.3.1 miners

```
list models.WorldData.miners [Miner] [static]
```

8.24.3.2 mines

```
list models.WorldData.mines [Mine] [static]
```

The documentation for this class was generated from the following file:

- domain/[models.py](#)

Chapter 9

File Documentation

9.1 domain/models.py File Reference

Domain models used by the Aurumveil platform.

Classes

- class [models.Point](#)
Two-dimensional point.
- class [models.Dwarf](#)
Base dwarf entity.
- class [models.Miner](#)
Mining unit.
- class [models.Warden](#)
Defensive unit.
- class [models.Mine](#)
Mining site.
- class [models.WorldData](#)
World state container.

Namespaces

- namespace [models](#)

9.1.1 Detailed Description

Domain models used by the Aurumveil platform.

Defines geometric structures, dwarfs entities, mining infrastructure, and the world data model used throughout the application.

9.2 domain/repository.py File Reference

Persistent storage utilities for world data.

Namespaces

- namespace [repository](#)

Functions

- None [repository._ensure_data_directories](#) ()
- None [repository._validate_json_structure](#) (dict json_data)
Validates the structure of serialized world data.
- WorldData|None [repository.load_default_data](#) ()
Loads the default world state.
- WorldData [repository.load_from_file](#) (str file_path)
Loads world data from a compressed file.
- None [repository.save_default_data](#) (WorldData world_data)
Saves the world state to the default location.
- None [repository.save_data_to_path](#) (WorldData world_data, str file_path)
Saves world data to a compressed file.
- str [repository.save_raw_data](#) (WorldData world_data)
Saves a dataset snapshot to the dataset repository.
- dict[str, list[dict[str, object]]] [repository.to_canonical_dict](#) (WorldData world_data)
Creates a canonical representation of world data.
- None [repository.delete_data_file](#) ()
Removes the default runtime state file.

Variables

- tuple [repository.BASE_DIR](#)
- tuple [repository.DATA_DIRECTORY](#)
- tuple [repository.DEFAULT_DATA_FILE](#)
Default runtime state file.
- tuple [repository.RAW_DATA_DIRECTORY](#)
Directory containing importable datasets.

9.2.1 Detailed Description

Persistent storage utilities for world data.

Provides functionality for loading, validating, serializing, compressing, and storing world data used by the Aurumveil platform.

9.3 domain/serializer.py File Reference

Text parser for world data definitions.

Classes

- class [serializer.InputParser](#)
World data text parser.

Namespaces

- namespace [serializer](#)

9.3.1 Detailed Description

Text parser for world data definitions.

Provides functionality for converting textual world descriptions into domain objects used by the Aurumveil platform.

9.4 engine/compression/huffman_codec.py File Reference

Huffman-based text compression and decompression.

Classes

- class [huffman_codec.HuffmanNode](#)
Node of a Huffman coding tree.

Namespaces

- namespace [huffman_codec](#)

Functions

- [HuffmanNode](#) [huffman_codec.build_tree](#) (str text)
Constructs a Huffman tree.
- dict[str, str] [huffman_codec.build_codes](#) ([HuffmanNode](#) node, str prefix="", dict[str, str]|None code-book=None)
Generates Huffman codes from a tree.
- bytes [huffman_codec.compress_text](#) (str text)
Compresses text using Huffman coding.
- str [huffman_codec.decompress_text](#) (bytes data)
Decompresses Huffman-encoded data.

9.4.1 Detailed Description

Huffman-based text compression and decompression.

Implements Huffman coding for lossless text compression, including tree construction, code generation, encoding, and decoding operations.

9.5 engine/engines/convex_hull/convex_hull.hpp File Reference

Convex hull algorithm interfaces.

```
#include <vector>
#include "geometry_utils.hpp"
```

Include dependency graph for convex_hull.hpp: This graph shows which files directly or indirectly include this file:

Functions

- `vector< Vertex > computeConvexHullJarvis` (const `vector< Vertex > &vertices`)
Computes a convex hull using Jarvis March.
- `vector< Vertex > computeConvexHullGraham` (`vector< Vertex > vertices`)
Computes a convex hull using Graham Scan.
- `vector< Vertex > computeConvexHullMonotoneChain` (`vector< Vertex > vertices`)
Computes a convex hull using Monotone Chain.

9.5.1 Detailed Description

Convex hull algorithm interfaces.

Declares convex hull algorithms used to determine the protected boundary of the dwarven kingdom.

9.5.2 Function Documentation

9.5.2.1 computeConvexHullGraham()

```
vector< Vertex > computeConvexHullGraham (
    vector< Vertex > vertices)
```

Computes a convex hull using Graham Scan.

Sorts vertices by polar angle and incrementally constructs the convex hull using a stack-based approach.

Parameters

<i>vertices</i>	Input point set.
-----------------	------------------

Returns

vector<Vertex> Convex hull vertices in traversal order.

Selects the lowest pivot vertex, sorts all remaining vertices by polar angle, removes redundant collinear points, and incrementally constructs the convex hull using a stack-based approach.

Complexity: $O(n \log n)$

Parameters

<i>vertices</i>	Input vertex set.
-----------------	-------------------

Returns

vector<Vertex> Convex hull vertices in counterclockwise order.

Here is the call graph for this function: Here is the caller graph for this function:

9.5.2.2 computeConvexHullJarvis()

```
vector< Vertex > computeConvexHullJarvis (
    const vector< Vertex > & vertices)
```

Computes a convex hull using Jarvis March.

Constructs the convex hull by iteratively selecting the next boundary vertex with the smallest turning angle relative to the current hull edge.

Parameters

<i>vertices</i>	Input point set.
-----------------	------------------

Returns

vector<Vertex> Convex hull vertices in traversal order.

Starts from the lowest vertex and repeatedly selects the next boundary vertex by searching for the most counterclockwise point relative to the current hull edge.

Complexity: $O(nh)$ where n is the number of vertices and h is the number of hull vertices.

Parameters

<i>vertices</i>	Input vertex set.
-----------------	-------------------

Returns

vector<Vertex> Convex hull vertices in counterclockwise order.

Here is the call graph for this function: Here is the caller graph for this function:

9.5.2.3 computeConvexHullMonotoneChain()

```
vector< Vertex > computeConvexHullMonotoneChain (
    vector< Vertex > vertices)
```

Computes a convex hull using Monotone Chain.

Constructs lower and upper hulls independently and combines them into a complete convex boundary.

Parameters

<i>vertices</i>	Input point set.
-----------------	------------------

Returns

vector<Vertex> Convex hull vertices in traversal order.

Sorts vertices lexicographically, constructs the lower and upper hulls independently, and combines them into a complete convex boundary.

Complexity: $O(n \log n)$

Parameters

<i>vertices</i>	Input vertex set.
-----------------	-------------------

Returns

vector<Vertex> Convex hull vertices in counterclockwise order.

Here is the call graph for this function: Here is the caller graph for this function:

9.6 convex_hull.hpp

[Go to the documentation of this file.](#)

```
00001
00002
00003 #ifndef CORE_CONVEX_HULL_HPP
00004 #define CORE_CONVEX_HULL_HPP
00005
00006 #include <vector>
00007
00008 #include "geometry_utils.hpp"
00009
00010 using namespace std;
00011
00012 vector<Vertex> computeConvexHullJarvis(
00013     const vector<Vertex>& vertices
00014 );
00015
00016 vector<Vertex> computeConvexHullGraham(
00017     vector<Vertex> vertices
00018 );
00019
00020 vector<Vertex> computeConvexHullMonotoneChain(
00021     vector<Vertex> vertices
00022 );
00023
00024 #endif // CORE_CONVEX_HULL_HPP
```

9.7 engine/engines/convex_hull/geometry_utils.cpp File Reference

Implementation of geometric utility functions.

```
#include <tuple>
#include "geometry_utils.hpp"
Include dependency graph for geometry_utils.cpp:
```

Functions

- bool `compareVertices` (const `Vertex` &a, const `Vertex` &b)
Compares vertices lexicographically.
- double `crossProduct` (const `Vertex` &origin, const `Vertex` &a, const `Vertex` &b)
Computes a signed cross product.
- double `distanceSquared` (const `Vertex` &a, const `Vertex` &b)
Computes squared Euclidean distance.
- bool `comparePolar` (const `Vertex` &pivot, const `Vertex` &a, const `Vertex` &b)
Compares vertices by polar angle.

9.7.1 Detailed Description

Implementation of geometric utility functions.

Provides geometric operations used by convex hull algorithms, including orientation tests, distance calculations, and vertex ordering predicates.

9.7.2 Function Documentation

9.7.2.1 `comparePolar()`

```
bool comparePolar (
    const Vertex & pivot,
    const Vertex & a,
    const Vertex & b)
```

Compares vertices by polar angle.

Orders vertices according to their polar angle relative to a pivot point. Collinear points are ordered by distance from the pivot.

Parameters

<i>pivot</i>	Reference vertex.
<i>a</i>	First vertex.
<i>b</i>	Second vertex.

Returns

bool True when a precedes b in polar order.

Here is the call graph for this function:

9.7.2.2 compareVertices()

```
bool compareVertices (
    const Vertex & a,
    const Vertex & b)
```

Compares vertices lexicographically.

Orders vertices according to their coordinates. Used during preprocessing and sorting stages of convex hull algorithms.

Parameters

<i>a</i>	First vertex.
<i>b</i>	Second vertex.

Returns

bool True when *a* precedes *b*.

Here is the caller graph for this function:

9.7.2.3 crossProduct()

```
double crossProduct (
    const Vertex & origin,
    const Vertex & a,
    const Vertex & b)
```

Computes a signed cross product.

Computes the orientation of three points.

Computes a signed cross product.

Calculates the signed cross product of vectors OA and OB, where O is the origin point.

Positive values indicate a counterclockwise turn, negative values indicate a clockwise turn, and zero indicates collinearity.

Parameters

<i>origin</i>	Common origin point.
<i>a</i>	First endpoint.
<i>b</i>	Second endpoint.

Returns

double Signed cross product value.

Here is the caller graph for this function:

9.7.2.4 distanceSquared()

```
double distanceSquared (
    const Vertex & a,
    const Vertex & b)
```

Computes squared Euclidean distance.

Returns the squared distance between two vertices. The square root operation is intentionally avoided to improve performance.

Parameters

<i>a</i>	First vertex.
<i>b</i>	Second vertex.

Returns

double Squared Euclidean distance.

Here is the caller graph for this function:

9.8 engine/engines/convex_hull/geometry_utils.hpp File Reference

Geometric primitives and utility functions.

```
#include <string>
```

Include dependency graph for geometry_utils.hpp: This graph shows which files directly or indirectly include this file:

Classes

- struct [Vertex](#)
Two-dimensional geometric point.

Functions

- double [crossProduct](#) (const [Vertex](#) &origin, const [Vertex](#) &a, const [Vertex](#) &b)
Computes the orientation of three points.
- double [distanceSquared](#) (const [Vertex](#) &a, const [Vertex](#) &b)
Computes squared Euclidean distance.
- bool [compareVertices](#) (const [Vertex](#) &a, const [Vertex](#) &b)
Compares vertices lexicographically.
- bool [comparePolar](#) (const [Vertex](#) &pivot, const [Vertex](#) &a, const [Vertex](#) &b)
Compares vertices by polar angle.

Variables

- constexpr double `EPSILON` = 1e-12
Floating-point comparison tolerance.

9.8.1 Detailed Description

Geometric primitives and utility functions.

Provides fundamental geometric structures and operations used by convex hull algorithms.

9.8.2 Function Documentation

9.8.2.1 comparePolar()

```
bool comparePolar (
    const Vertex & pivot,
    const Vertex & a,
    const Vertex & b)
```

Compares vertices by polar angle.

Orders vertices according to their polar angle relative to a pivot point. Collinear points are ordered by distance from the pivot.

Parameters

<i>pivot</i>	Reference vertex.
<i>a</i>	First vertex.
<i>b</i>	Second vertex.

Returns

bool True when a precedes b in polar order.

Here is the call graph for this function:

9.8.2.2 compareVertices()

```
bool compareVertices (
    const Vertex & a,
    const Vertex & b)
```

Compares vertices lexicographically.

Orders vertices according to their coordinates. Used during preprocessing and sorting stages of convex hull algorithms.

Parameters

<i>a</i>	First vertex.
<i>b</i>	Second vertex.

Returns

bool True when *a* precedes *b*.

Here is the caller graph for this function:

9.8.2.3 crossProduct()

```
double crossProduct (
    const Vertex & origin,
    const Vertex & a,
    const Vertex & b)
```

Computes the orientation of three points.

Computes a signed cross product.

Calculates the signed cross product of vectors OA and OB, where O is the origin point.

Positive values indicate a counterclockwise turn, negative values indicate a clockwise turn, and zero indicates collinearity.

Parameters

<i>origin</i>	Common origin point.
<i>a</i>	First endpoint.
<i>b</i>	Second endpoint.

Returns

double Signed cross product value.

Computes the orientation of three points. Here is the caller graph for this function:

9.8.2.4 distanceSquared()

```
double distanceSquared (
    const Vertex & a,
    const Vertex & b)
```

Computes squared Euclidean distance.

Returns the squared distance between two vertices. The square root operation is intentionally avoided to improve performance.

Parameters

<i>a</i>	First vertex.
<i>b</i>	Second vertex.

Returns

double Squared Euclidean distance.

Here is the caller graph for this function:

9.8.3 Variable Documentation

9.8.3.1 EPSILON

```
double EPSILON = 1e-12 [constexpr]
```

Floating-point comparison tolerance.

Used to compensate for numerical precision errors during geometric computations.

9.9 geometry_utils.hpp

[Go to the documentation of this file.](#)

```
00001
00002
00003 #ifndef CORE_GEOMETRY_UTILS_HPP
00004 #define CORE_GEOMETRY_UTILS_HPP
00005
00006 #include <string>
00007
00008 constexpr double EPSILON = 1e-12;
00009
00010 using namespace std;
00011
00012 struct Vertex {
00013     string id;
00014     double x{0.0};
00015     double y{0.0};
00016 };
00017
00018 double crossProduct(
00019     const Vertex& origin,
00020     const Vertex& a,
00021     const Vertex& b
00022 );
```



```

00064
00081 double distanceSquared(
00082     const Vertex& a,
00083     const Vertex& b
00084 );
00085
00102 bool compareVertices(
00103     const Vertex& a,
00104     const Vertex& b
00105 );
00106
00126 bool comparePolar(
00127     const Vertex& pivot,
00128     const Vertex& a,
00129     const Vertex& b
00130 );
00131
00132 #endif // CORE_GEOMETRY_UTILS_HPP

```

9.10 engine/engines/convex_hull/graham_convex_hull.cpp File Reference

Graham Scan convex hull algorithm.

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
#include "convex_hull.hpp"
#include "geometry_utils.hpp"
#include "foundation/core_utils_json.hpp"
#include "foundation/core_utils_stopwatch.hpp"
Include dependency graph for graham_convex_hull.cpp:

```

Functions

- vector< [Vertex](#) > [computeConvexHullGraham](#) (vector< [Vertex](#) > vertices)
Computes a convex hull using Graham Scan.
- int [main](#) (int argc, char *argv[])
Application entry point.

9.10.1 Detailed Description

Graham Scan convex hull algorithm.

Implements the Graham Scan algorithm for constructing the convex hull of a set of two-dimensional vertices.

9.10.2 Function Documentation

9.10.2.1 computeConvexHullGraham()

```
vector< Vertex > computeConvexHullGraham (
    vector< Vertex > vertices)
```

Computes a convex hull using Graham Scan.

Selects the lowest pivot vertex, sorts all remaining vertices by polar angle, removes redundant collinear points, and incrementally constructs the convex hull using a stack-based approach.

Complexity: $O(n \log n)$

Parameters

<i>vertices</i>	Input vertex set.
-----------------	-------------------

Returns

vector<Vertex> Convex hull vertices in counterclockwise order.

Sorts vertices by polar angle and incrementally constructs the convex hull using a stack-based approach.

Parameters

<i>vertices</i>	Input point set.
-----------------	------------------

Returns

vector<Vertex> Convex hull vertices in traversal order.

Here is the call graph for this function: Here is the caller graph for this function:

9.10.2.2 main()

```
int main (
    int argc,
    char * argv[])
```

Application entry point.

Loads input vertices, executes the Graham Scan convex hull algorithm, and exports the resulting boundary as a JSON response.

Parameters

<i>argc</i>	Number of command-line arguments.
<i>argv</i>	Command-line arguments.

Returns

int EXIT_SUCCESS on success, otherwise EXIT_FAILURE.

Here is the call graph for this function:

9.11 engine/engines/convex_hull/jarvis_convex_hull.cpp File Reference

Jarvis March convex hull algorithm.

```
#include <iostream>
#include <vector>
#include <cmath>
#include "convex_hull.hpp"
#include "geometry_utils.hpp"
#include "foundation/core_utils_json.hpp"
#include "foundation/core_utils_stopwatch.hpp"
Include dependency graph for jarvis_convex_hull.cpp:
```

Functions

- `vector< Vertex > computeConvexHullJarvis` (const vector< [Vertex](#) > &vertices)
Computes a convex hull using Jarvis March.
- `int main` (int argc, char *argv[])
Application entry point.

9.11.1 Detailed Description

Jarvis March convex hull algorithm.

Implements the Jarvis March (Gift Wrapping) algorithm for constructing the convex hull of a set of two-dimensional vertices.

9.11.2 Function Documentation

9.11.2.1 computeConvexHullJarvis()

```
vector< Vertex > computeConvexHullJarvis (
    const vector< Vertex > & vertices)
```

Computes a convex hull using Jarvis March.

Starts from the lowest vertex and repeatedly selects the next boundary vertex by searching for the most counter-clockwise point relative to the current hull edge.

Complexity: $O(nh)$ where n is the number of vertices and h is the number of hull vertices.

Parameters

<i>vertices</i>	Input vertex set.
-----------------	-------------------

Returns

vector<Vertex> Convex hull vertices in counterclockwise order.

Constructs the convex hull by iteratively selecting the next boundary vertex with the smallest turning angle relative to the current hull edge.

Parameters

<i>vertices</i>	Input point set.
-----------------	------------------

Returns

vector<Vertex> Convex hull vertices in traversal order.

Here is the call graph for this function: Here is the caller graph for this function:

9.11.2.2 main()

```
int main (
    int argc,
    char * argv[])
```

Application entry point.

Loads input vertices, executes the Jarvis convex hull algorithm, and exports the resulting boundary as a JSON response.

Parameters

<i>argc</i>	Number of command-line arguments.
<i>argv</i>	Command-line arguments.

Returns

int EXIT_SUCCESS on success, otherwise EXIT_FAILURE.

Here is the call graph for this function:

9.12 engine/engines/convex_hull/monotone_convex_hull.cpp File Reference

Monotone Chain convex hull algorithm.

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
#include "convex_hull.hpp"
#include "geometry_utils.hpp"
#include "foundation/core_utils_json.hpp"
#include "foundation/core_utils_stopwatch.hpp"
Include dependency graph for monotone_convex_hull.cpp:
```

Functions

- vector< [Vertex](#) > [computeConvexHullMonotoneChain](#) (vector< [Vertex](#) > vertices)
Computes a convex hull using Monotone Chain.
- int [main](#) (int argc, char *argv[])
Application entry point.

9.12.1 Detailed Description

Monotone Chain convex hull algorithm.

Implements Andrew's Monotone Chain algorithm for constructing the convex hull of a set of two-dimensional vertices.

9.12.2 Function Documentation

9.12.2.1 [computeConvexHullMonotoneChain\(\)](#)

```
vector< Vertex > computeConvexHullMonotoneChain (
    vector< Vertex > vertices)
```

Computes a convex hull using Monotone Chain.

Sorts vertices lexicographically, constructs the lower and upper hulls independently, and combines them into a complete convex boundary.

Complexity: $O(n \log n)$

Parameters

<i>vertices</i>	Input vertex set.
-----------------	-------------------

Returns

vector<Vertex> Convex hull vertices in counterclockwise order.

Constructs lower and upper hulls independently and combines them into a complete convex boundary.

Parameters

<i>vertices</i>	Input point set.
-----------------	------------------

Returns

vector<Vertex> Convex hull vertices in traversal order.

Here is the call graph for this function: Here is the caller graph for this function:

9.12.2.2 main()

```
int main (
    int argc,
    char * argv[])
```

Application entry point.

Loads input vertices, executes the Monotone Chain convex hull algorithm, and exports the resulting boundary as a JSON response.

Parameters

<i>argc</i>	Number of command-line arguments.
<i>argv</i>	Command-line arguments.

Returns

int EXIT_SUCCESS on success, otherwise EXIT_FAILURE.

Here is the call graph for this function:

9.13 engine/engines/network_flow/bellman_ford_strategy.hpp File Reference

Bellman-Ford shortest-path strategy.

```
#include <queue>
#include <vector>
#include "ishortest_path_strategy.hpp"
```

Include dependency graph for bellman_ford_strategy.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class [BellmanFordStrategy](#)
Bellman-Ford shortest-path strategy.

9.13.1 Detailed Description

Bellman-Ford shortest-path strategy.

Implements a Bellman-Ford based shortest-path algorithm for finding minimum-cost augmenting paths in a residual flow network.

9.14 bellman_ford_strategy.hpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #ifndef BELLMAN_FORD_STRATEGY_HPP
00011 #define BELLMAN_FORD_STRATEGY_HPP
00012
00013 #include <queue>
00014 #include <vector>
00015
00016 #include "ishortest_path_strategy.hpp"
00017
00018 using namespace std;
00019
00020 class BellmanFordStrategy
00021 : public IShortestPathStrategy
00022 {
00023 public:
00024
00025     bool findPath(
00026         FlowNetwork& graph,
00027         int source,
00028         int sink,
00029         vector<int>& parentVertex,
00030         vector<size_t>& parentEdge,
00031         vector<int>& distance
00032     ) override {
00033
00034         const int vertexCount =
00035             static_cast<int>(
00036                 graph.size()
00037             );
00038
00039         distance.assign(
00040             vertexCount,
00041             DISTANCE_INFINITY
00042         );
00043
00044         parentVertex.assign(
00045             vertexCount,
00046             -1
00047         );
00048
00049         parentEdge.assign(
00050             graph.size(),
00051             INVALID_EDGE
00052         );
00053
00054         vector<bool> isInQueue(
00055             vertexCount,
00056             false
00057         );
00058
00059         vector<int> relaxationCount(
00060             vertexCount,
00061             0
00062         );
00063
00064         queue<int> vertexQueue;
00065
00066         distance[source] = 0;
00067
00068         vertexQueue.push(
00069             source
00070         );
00071
00072         isInQueue[source] = true;
00073
00074         while (!vertexQueue.empty()) {
00075             const int currentVertex =
00076                 vertexQueue.front();
00077
00078             vertexQueue.pop();
00079
00080             isInQueue[
00081                 currentVertex
00082             ] = false;
00083
00084             for (
00085                 size_t edgeIndex = 0;
00086                 edgeIndex < graph[currentVertex].size();
00087                 ++edgeIndex
00088             ) {
00089                 Edge& currentEdge =

```

```

00133         graph[currentVertex]
00134             [edgeIndex];
00135
00136         const int destinationVertex =
00137             static_cast<int>({
00138                 currentEdge
00139                     .destination
00140             });
00141
00142         if (currentEdge.capacity <= 0) {
00143             continue;
00144         }
00145
00146         if (distance[currentVertex] == DISTANCE_INFINITY) {
00147             continue;
00148         }
00149
00150         if (distance[destinationVertex] > distance[currentVertex] + currentEdge.cost) {
00151             distance[
00152                 destinationVertex
00153             ] =
00154                 distance[
00155                     currentVertex
00156                 ] + currentEdge.cost;
00157
00158             parentVertex[
00159                 destinationVertex
00160             ] = currentVertex;
00161
00162             parentEdge[
00163                 destinationVertex
00164             ] = edgeIndex;
00165
00166             ++relaxationCount[
00167                 destinationVertex
00168             ];
00169
00170             if (relaxationCount[destinationVertex] > vertexCount) {
00171                 return false;
00172             }
00173
00174             if (!isInQueue[destinationVertex]) {
00175                 vertexQueue.push(
00176                     destinationVertex
00177                 );
00178
00179                 isInQueue[
00180                     destinationVertex
00181                 ] = true;
00182             }
00183         }
00184     }
00185 }
00186
00187     return distance[sink] != DISTANCE_INFINITY;
00188 }
00189 };
00190
00191 #endif // BELLMAN_FORD_STRATEGY_HPP

```

9.15 engine/engines/network_flow/desopopape_strategy.hpp File Reference

```

#include <climits>
#include <deque>
#include <vector>
#include "ishortest_path_strategy.hpp"

```

Include dependency graph for desopopape_strategy.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class [DEsopoPapeStrategy](#)
D'Esopo-Pape shortest-path strategy.

9.16 desopopape_strategy.hpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #ifndef DESOPO_PAPE_STRATEGY_HPP
00011 #define DESOPO_PAPE_STRATEGY_HPP
00012
00013 #include <climits>
00014 #include <deque>
00015 #include <vector>
00016
00017 #include "ishortest_path_strategy.hpp"
00018
00019 using namespace std;
00020
00031 class DEsopoPapeStrategy
00032 : public IShortestPathStrategy
00033 {
00034 public:
00035
00064     bool findPath(
00065         FlowNetwork& graph,
00066         int source,
00067         int sink,
00068         vector<int>& parentVertex,
00069         vector<size_t>& parentEdge,
00070         vector<int>& distance
00071     ) override {
00072
00073         const int vertexCount =
00074             static_cast<int>(
00075                 graph.size()
00076             );
00077
00078         distance.assign(
00079             vertexCount,
00080             DISTANCE_INFINITY
00081         );
00082
00083         parentVertex.assign(
00084             vertexCount,
00085             -1
00086         );
00087
00088         parentEdge.assign(
00089             vertexCount,
00090             INVALID_EDGE
00091         );
00092
00093         vector<bool> inQueue(
00094             vertexCount,
00095             false
00096         );
00097
00098         deque<int> vertexDeque;
00099
00100         distance[source] = 0;
00101
00102         vertexDeque.push_back(
00103             source
00104         );
00105
00106         inQueue[source] = true;
00107
00108         while (!vertexDeque.empty()) {
00109             const int currentVertex =
00110                 vertexDeque.front();
00111
00112             vertexDeque.pop_front();
00113
00114             inQueue[
00115                 currentVertex
00116             ] = false;
00117
00118             for (
00119                 size_t edgeIndex = 0;
00120                 edgeIndex < graph[currentVertex].size();
00121                 ++edgeIndex
00122             ) {
00123                 Edge& edge =
00124                     graph[currentVertex]
00125                     [edgeIndex];
00126
00127                 if (

```

```

00128         edge.capacity > 0
00129         && distance[currentVertex] != DISTANCE_INFINITY
00130         && distance[currentVertex] + edge.cost < distance[edge.destination]
00131     ) {
00132         const int previousDistance =
00133             distance[edge.destination];
00134
00135         distance[edge.destination] =
00136             distance[currentVertex]
00137             + edge.cost;
00138
00139         parentVertex[
00140             edge.destination
00141         ] = currentVertex;
00142
00143         parentEdge[
00144             edge.destination
00145         ] = edgeIndex;
00146
00147         if (!inQueue[edge.destination]) {
00148             if (previousDistance == DISTANCE_INFINITY) {
00149                 vertexDeque.push_back(
00150                     edge.destination
00151                 );
00152             }
00153             else {
00154                 vertexDeque.push_front(
00155                     edge.destination
00156                 );
00157             }
00158
00159             inQueue[
00160                 edge.destination
00161             ] = true;
00162         }
00163     }
00164 }
00165 }
00166
00167     return distance[sink] != DISTANCE_INFINITY;
00168 }
00169 };
00170
00171 #endif // DESOPO_PAPE_STRATEGY_HPP

```

9.17 engine/engines/network_flow/edmonds_karp_strategy.hpp File Reference

Edmonds-Karp maximum-flow strategy.

```
#include <algorithm>
```

```
#include "imaxflow_path_strategy.hpp"
```

Include dependency graph for edmonds_karp_strategy.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class [EdmondsKarpStrategy](#)
Edmonds-Karp flow augmentation strategy.

9.17.1 Detailed Description

Edmonds-Karp maximum-flow strategy.

Implements flow computation and residual network augmentation according to the Edmonds-Karp method.

9.18 edmonds_karp_strategy.hpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #ifndef EDMONDS_KARP_STRATEGY_HPP
00011 #define EDMONDS_KARP_STRATEGY_HPP
00012
00013 #include <algorithm>
00014
00015 #include "imaxflow_path_strategy.hpp"
00016
00017 using namespace std;
00018
00031 class EdmondsKarpStrategy
00032 : public IMaxFlowStrategy
00033 {
00034 public:
00035
00061 int computeFlow(
00062     FlowNetwork& graph,
00063     int source,
00064     int sink,
00065     vector<int>& parentVertex,
00066     vector<size_t>& parentEdge
00067 ) override {
00068     constexpr int MAX_FLOW_LIMIT =
00069         1'000'000'000;
00070
00071     int pathFlow =
00072         MAX_FLOW_LIMIT;
00073
00074     int currentVertex =
00075         sink;
00076
00077     while (currentVertex != source) {
00078         const int previousVertex =
00079             parentVertex[currentVertex];
00080
00081         Edge& edge =
00082             graph[previousVertex]
00083                 [parentEdge[currentVertex]];
00084
00085         pathFlow = min(
00086             pathFlow,
00087             edge.capacity
00088         );
00089
00090         currentVertex =
00091             previousVertex;
00092     }
00093
00094     return pathFlow;
00095 }
00096
00125 void augment(
00126     FlowNetwork& graph,
00127     int source,
00128     int sink,
00129     vector<int>& parentVertex,
00130     vector<size_t>& parentEdge,
00131     int flow,
00132     int& totalCost
00133 ) override {
00134     int currentVertex = sink;
00135
00136     while (currentVertex != source) {
00137         const int previousVertex =
00138             parentVertex[currentVertex];
00139
00140         Edge& edge =
00141             graph[previousVertex]
00142                 [parentEdge[currentVertex]];
00143
00144         Edge& reverse =
00145             graph[currentVertex]
00146                 [edge.reverseEdgeIndex];
00147
00148         edge.capacity -= flow;
00149         reverse.capacity += flow;
00150
00151         totalCost +=
00152             flow * edge.cost;
00153
00154         currentVertex =

```

```

00155         previousVertex;
00156     }
00157 }
00158
00177 int calculatePathLength(
00178     int source,
00179     int sink,
00180     const vector<int>& parentVertex
00181 ) override {
00182     int pathLength = 0;
00183
00184     for (
00185         int vertex = sink;
00186         vertex != source;
00187         vertex = parentVertex[vertex]
00188     ) {
00189         ++pathLength;
00190     }
00191
00192     return pathLength;
00193 }
00194 };
00195
00196 #endif // EDMONDS_KARP_STRATEGY_HPP

```

9.19 engine/engines/network_flow/flow_network.hpp File Reference

Residual flow network data structures.

```

#include <vector>
#include <limits>

```

Include dependency graph for flow_network.hpp: This graph shows which files directly or indirectly include this file:

Classes

- struct [Edge](#)
Residual network edge.

Typedefs

- using [FlowNetwork](#)
Residual flow network.

Variables

- constexpr int [DISTANCE_INFINITY](#) = numeric_limits<int>::max()
Infinite distance sentinel value.
- constexpr size_t [INVALID_EDGE](#) = numeric_limits<size_t>::max()
Invalid edge index sentinel.

9.19.1 Detailed Description

Residual flow network data structures.

Defines graph structures, edge representations, and utility constants used by flow optimization algorithms.

9.19.2 Typedef Documentation

9.19.2.1 FlowNetwork

using `FlowNetwork`

Initial value:

```
vector<vector<Edge>>
```

Residual flow network.

Adjacency-list representation of a directed residual graph used by flow optimization algorithms.

9.19.3 Variable Documentation

9.19.3.1 DISTANCE_INFINITY

```
int DISTANCE_INFINITY = numeric_limits<int>::max() [constexpr]
```

Infinite distance sentinel value.

Used to initialize shortest-path distances before pathfinding algorithms are executed.

9.19.3.2 INVALID_EDGE

```
size_t INVALID_EDGE = numeric_limits<size_t>::max() [constexpr]
```

Invalid edge index sentinel.

Represents a non-existing edge reference during path reconstruction.

9.20 flow_network.hpp

[Go to the documentation of this file.](#)

```
00001
00009
00010 #ifndef FLOW_NETWORK_HPP
00011 #define FLOW_NETWORK_HPP
00012
00013 #include <vector>
00014 #include <limits>
00015
00016 using namespace std;
00017
00024 constexpr int DISTANCE_INFINITY = numeric_limits<int>::max();
00025
00032 constexpr size_t INVALID_EDGE = numeric_limits<size_t>::max();
00033
00041 struct Edge {
00042     size_t destination;
00043     size_t reverseEdgeIndex;
00044
00045     int capacity;
00046     int cost;
00047 };
00048
00056 using FlowNetwork =
00057     vector<vector<Edge>>;
00058
00059 #endif //FLOW_NETWORK_HPP
```

9.21 engine/engines/network_flow/flow_optimizer.cpp File Reference

Minimum-Cost Maximum-Flow application.

```
#include <fstream>
#include <iostream>
#include "flow_optimizer.hpp"
#include "imaxflow_path_factory.hpp"
#include "ishortest_path_factory.hpp"
#include "foundation/core_utils_json.hpp"
#include "foundation/core_utils_stopwatch.hpp"
Include dependency graph for flow_optimizer.cpp:
```

Typedefs

- using `json` = `nlohmann::json`

Functions

- int `main` (int argc, char *argv[])
Application entry point.

9.21.1 Detailed Description

Minimum-Cost Maximum-Flow application.

Builds a flow network from the input dataset, executes the selected optimization algorithms, and exports miner assignment results as JSON.

9.21.2 Typedef Documentation

9.21.2.1 json

```
using json = nlohmann::json
```

9.21.3 Function Documentation

9.21.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Application entry point.

Loads the input dataset, constructs a resource allocation flow network, executes the selected maximum-flow and shortest-path algorithms, and exports optimization results as a JSON response.

Parameters

<i>argc</i>	Number of command-line arguments.
<i>argv</i>	Command-line arguments.

Returns

int EXIT_SUCCESS on success, otherwise EXIT_FAILURE.

Here is the call graph for this function:

9.22 engine/engines/network_flow/flow_optimizer.hpp File Reference

Minimum-Cost Maximum-Flow optimization engine.

```
#include <vector>
#include "imaxflow_path_strategy.hpp"
#include "ishortest_path_strategy.hpp"
#include "foundation/core_utils_stopwatch.hpp"
Include dependency graph for flow_optimizer.hpp: This graph shows which files directly or indirectly include this file:
```

Classes

- struct [FlowStatistics](#)
Flow optimization statistics.
- class [FlowOptimizer](#)
Minimum-Cost Maximum-Flow optimizer.

9.22.1 Detailed Description

Minimum-Cost Maximum-Flow optimization engine.

Provides the core optimization framework used for resource allocation between miners and mines.

The optimizer combines a shortest-path strategy with a maximum-flow strategy to compute a minimum-cost maximum-flow solution.

9.23 flow_optimizer.hpp

[Go to the documentation of this file.](#)

```

00001
00012
00013 #ifndef FLOW_OPTIMIZER_HPP
00014 #define FLOW_OPTIMIZER_HPP
00015
00016 #include <vector>
00017
00018 #include "imaxflow_path_strategy.hpp"
00019 #include "ishortest_path_strategy.hpp"
00020
00021 #include "foundation/core_utils_stopwatch.hpp"
00022
00023 using namespace std;
00024
00032 struct FlowStatistics {
00033     int maximumFlow{0};
00034     int minimumCost{0};
00035
00036     int augmentationCount{0};
00037
00038     double averagePathLength{0.0};
00039     int totalPathLength{0};
00040
00041     double pathfindingTimeMs{0.0};
00042     double augmentationTimeMs{0.0};
00043     double totalTimeMs{0.0};
00044 };
00045
00053 class FlowOptimizer {
00054 public:
00055
00072     FlowOptimizer(
00073         size_t vertices,
00074         IMaxFlowStrategy* maxflowStrategy,
00075         IShortestPathStrategy* mincostStrategy
00076     )
00077         : flowNetwork(vertices),
00078           maxflowStrategy(maxflowStrategy),
00079           mincostStrategy(mincostStrategy) {}
00080
00099     void addEdge(
00100         size_t from,
00101         size_t to,
00102         int capacity,
00103         int cost
00104     ) {
00105         Edge forwardEdge{
00106             to,
00107             flowNetwork[to].size(),
00108             capacity,
00109             cost
00110         };
00111
00112         Edge backwardEdge{
00113             from,
00114             flowNetwork[from].size(),
00115             0,
00116             -cost
00117         };
00118
00119         flowNetwork[from].push_back(
00120             forwardEdge
00121         );
00122
00123         flowNetwork[to].push_back(
00124             backwardEdge
00125         );
00126     }
00127
00145     FlowStatistics solve(
00146         int source,
00147         int sink
00148     ) {
00149         Stopwatch totalStopwatch;
00150         Stopwatch phaseStopwatch;
00151
00152         FlowStatistics statistics;
00153
00154         vector<int> parentVertex;
00155         vector<size_t> parentEdge;
00156         vector<int> distance;
00157

```



```

00158         while (true) {
00159             phaseStopwatch.restart();
00160
00161             const bool pathFound =
00162                 mincostStrategy->findPath(
00163                     flowNetwork,
00164                     source,
00165                     sink,
00166                     parentVertex,
00167                     parentEdge,
00168                     distance
00169                 );
00170
00171             statistics.pathfindingTimeMs +=
00172                 phaseStopwatch
00173                     .elapsedMilliseconds();
00174
00175             if (!pathFound) {
00176                 break;
00177             }
00178
00179             phaseStopwatch.restart();
00180
00181             const int flow =
00182                 maxflowStrategy->computeFlow(
00183                     flowNetwork,
00184                     source,
00185                     sink,
00186                     parentVertex,
00187                     parentEdge
00188                 );
00189
00190             ++statistics.augmentationCount;
00191
00192             statistics.totalPathLength +=
00193                 maxflowStrategy->calculatePathLength(
00194                     source,
00195                     sink,
00196                     parentVertex
00197                 );
00198
00199             if (flow <= 0) {
00200                 break;
00201             }
00202
00203             maxflowStrategy->augment(
00204                 flowNetwork,
00205                 source,
00206                 sink,
00207                 parentVertex,
00208                 parentEdge,
00209                 flow,
00210                 statistics.minimumCost
00211             );
00212
00213             statistics.maximumFlow +=
00214                 flow;
00215
00216             statistics.augmentationTimeMs +=
00217                 phaseStopwatch
00218                     .elapsedMilliseconds();
00219         }
00220
00221         statistics.totalTimeMs =
00222             totalStopwatch
00223                 .elapsedMilliseconds();
00224
00225         if (statistics.augmentationCount > 0) {
00226             statistics.averagePathLength =
00227                 static_cast<double>(
00228                     statistics.totalPathLength
00229                 ) / statistics.augmentationCount;
00230         }
00231
00232         return statistics;
00233     }
00234
00244     const FlowNetwork& getGraph() const {
00245         return flowNetwork;
00246     }
00247
00248 private:
00249
00251     FlowNetwork flowNetwork;
00252
00254     IMaxFlowStrategy* maxflowStrategy;
00255

```

```

00257     IShortestPathStrategy* mincostStrategy;
00258 };
00259
00260 #endif // FLOW_OPTIMIZER_HPP

```

9.24 engine/engines/network_flow/ford_fulkerson_strategy.hpp File Reference

Ford-Fulkerson maximum-flow strategy.

```

#include <algorithm>
#include "imaxflow_path_strategy.hpp"

```

Include dependency graph for ford_fulkerson_strategy.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class [FordFulkersonStrategy](#)
Ford-Fulkerson flow augmentation strategy.

9.24.1 Detailed Description

Ford-Fulkerson maximum-flow strategy.

Implements flow computation and residual network augmentation according to the Ford-Fulkerson method.

9.25 ford_fulkerson_strategy.hpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #ifndef FORD_FULKERSON_STRATEGY_HPP
00011 #define FORD_FULKERSON_STRATEGY_HPP
00012
00013 #include <algorithm>
00014
00015 #include "imaxflow_path_strategy.hpp"
00016
00017 using namespace std;
00018
00031 class FordFulkersonStrategy
00032 : public IMaxFlowStrategy
00033 {
00034 public:
00035
00061     int computeFlow(
00062         FlowNetwork& graph,
00063         int source,
00064         int sink,
00065         vector<int>& parentVertex,
00066         vector<size_t>& parentEdge
00067     ) override {
00068         constexpr int MAX_FLOW_LIMIT =
00069             1'000'000'000;
00070
00071         int pathFlow =
00072             MAX_FLOW_LIMIT;
00073
00074         for (
00075             int vertex = sink;

```

```

00076         vertex != source;
00077         vertex = parentVertex[vertex]
00078     ) {
00079         const int previousVertex =
00080             parentVertex[vertex];
00081
00082         Edge& currentEdge =
00083             graph[previousVertex]
00084                 [parentEdge[vertex]];
00085
00086         pathFlow = min(
00087             pathFlow,
00088             currentEdge.capacity
00089         );
00090     }
00091
00092     return pathFlow;
00093 }
00094
00123 void augment(
00124     FlowNetwork& graph,
00125     int source,
00126     int sink,
00127     vector<int>& parentVertex,
00128     vector<size_t>& parentEdge,
00129     int flow,
00130     int& totalCost
00131 ) override {
00132     for (
00133         int vertex = sink;
00134         vertex != source;
00135         vertex = parentVertex[vertex]
00136     ) {
00137         const int previousVertex =
00138             parentVertex[vertex];
00139
00140         Edge& currentEdge =
00141             graph[previousVertex]
00142                 [parentEdge[vertex]];
00143
00144         Edge& reverseEdge =
00145             graph[vertex]
00146                 [currentEdge.reverseEdgeIndex];
00147
00148         currentEdge.capacity -= flow;
00149         reverseEdge.capacity += flow;
00150
00151         totalCost +=
00152             flow * currentEdge.cost;
00153     }
00154 }
00155 };
00156
00157 #endif // FORD_FULKERSON_STRATEGY_HPP

```

9.26 engine/engines/network_flow/imaxflow_path_factory.hpp File Reference

Maximum-flow strategy factory.

```

#include <string>
#include <memory>
#include "imaxflow_path_strategy.hpp"
#include "ford_fulkerson_strategy.hpp"
#include "edmonds_karp_strategy.hpp"

```

Include dependency graph for imaxflow_path_factory.hpp: This graph shows which files directly or indirectly include this file:

Functions

- `unique_ptr< IMaxFlowStrategy > createMaxFlowStrategy (const string &strategyName)`
Creates a maximum-flow strategy.

9.26.1 Detailed Description

Maximum-flow strategy factory.

Provides creation utilities for maximum-flow algorithm implementations used by the flow optimization engine.

9.26.2 Function Documentation

9.26.2.1 createMaxFlowStrategy()

```
unique_ptr< IMaxFlowStrategy > createMaxFlowStrategy (
    const string & strategyName) [inline]
```

Creates a maximum-flow strategy.

Instantiates the selected maximum-flow algorithm implementation based on its textual identifier.

Supported algorithms:

- Ford-Fulkerson
- Edmonds-Karp

Parameters

<i>strategyName</i>	Maximum-flow algorithm name.
---------------------	------------------------------

Returns

unique_ptr<IMaxFlowStrategy> Strategy instance implementing the requested algorithm.

Exceptions

<i>runtime_error</i>	Raised when the requested algorithm is not supported.
----------------------	---

Here is the caller graph for this function:

9.27 imaxflow_path_factory.hpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #ifndef IMAXFLOW_PATH_FACTORY_HPP
00011 #define IMAXFLOW_PATH_FACTORY_HPP
00012
00013 #include <string>
00014 #include <memory>
00015
00016 #include "imaxflow_path_strategy.hpp"
00017
00018 #include "ford_fulkerson_strategy.hpp"
00019 #include "edmonds_karp_strategy.hpp"
00020
00021 using namespace std;
00022
00044 inline unique_ptr<IMaxFlowStrategy> createMaxFlowStrategy(const string& strategyName) {
00045     if (strategyName == "Ford-Fulkerson") {
00046         return make_unique<FordFulkersonStrategy>();
00047     }
00048
00049     if (strategyName == "Edmonds-Karp") {
00050         return make_unique<EdmondsKarpStrategy>();
00051     }
00052
00053     throw runtime_error(
00054         "Unsupported maximum flow algorithm: "
00055         + strategyName
00056     );
00057 }
00058
00059 #endif // IMAXFLOW_PATH_FACTORY_HPP

```

9.28 engine/engines/network_flow/imaxflow_path_strategy.hpp File Reference

Maximum-flow strategy interface.

```

#include <vector>
#include "flow_network.hpp"

```

Include dependency graph for imaxflow_path_strategy.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class [IMaxFlowStrategy](#)
Maximum-flow strategy interface.

9.28.1 Detailed Description

Maximum-flow strategy interface.

Defines the common interface implemented by all maximum-flow algorithms used by the flow optimization engine.

9.29 imaxflow_path_strategy.hpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #ifndef IMAXFLOW_PATH_STRATEGY_HPP
00011 #define IMAXFLOW_PATH_STRATEGY_HPP
00012
00013 #include <vector>
00014
00015 #include "flow_network.hpp"
00016
00017 using namespace std;
00018
00030 class IMaxFlowStrategy {
00031 public:
00032
00058     virtual int computeFlow(
00059         FlowNetwork& flowNetwork,
00060         int source,
00061         int sink,
00062         vector<int>& parentVertex,
00063         vector<size_t>& parentEdge
00064     ) = 0;
00065
00094     virtual void augment(
00095         FlowNetwork& flowNetwork,
00096         int source,
00097         int sink,
00098         vector<int>& parentVertex,
00099         vector<size_t>& parentEdge,
00100         int flow,
00101         int& totalCost
00102     ) = 0;
00103
00122     virtual int calculatePathLength(
00123         int source,
00124         int sink,
00125         const vector<int>& parentVertex
00126     ) {
00127         int pathLength = 0;
00128
00129         for (
00130             int vertex = sink;
00131             vertex != source;
00132             vertex = parentVertex[vertex]
00133         ) {
00134             ++pathLength;
00135         }
00136
00137         return pathLength;
00138     }
00139
00140     virtual ~IMaxFlowStrategy() = default;
00141 };
00142
00143 #endif // IMAXFLOW_PATH_STRATEGY_HPP

```

9.30 engine/engines/network_flow/ishortest_path_factory.hpp File Reference

Shortest-path strategy factory.

```

#include <string>
#include <memory>
#include "bellman_ford_strategy.hpp"
#include "desopopape_strategy.hpp"

```

Include dependency graph for ishortest_path_factory.hpp: This graph shows which files directly or indirectly include this file:

Functions

- `unique_ptr< IShortestPathStrategy > createShortestPathStrategy (const string &strategyName)`
Creates a shortest-path strategy.

9.30.1 Detailed Description

Shortest-path strategy factory.

Provides creation utilities for shortest-path algorithm implementations used by the flow optimization engine.

9.30.2 Function Documentation

9.30.2.1 createShortestPathStrategy()

```
unique_ptr< IShortestPathStrategy > createShortestPathStrategy (
    const string & strategyName) [inline]
```

Creates a shortest-path strategy.

Instantiates the selected shortest-path algorithm implementation based on its textual identifier.

Supported algorithms:

- Bellman-Ford
- D'Esopo-Pape

Parameters

<i>strategyName</i>	Shortest-path algorithm name.
---------------------	-------------------------------

Returns

unique_ptr<IShortestPathStrategy> Strategy instance implementing the requested algorithm.

Exceptions

<i>runtime_error</i>	Raised when the requested algorithm is not supported.
----------------------	---

Here is the caller graph for this function:

9.31 ishortest_path_factory.hpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #ifndef ISHORTEST_PATH_FACTORY_HPP
00011 #define ISHORTEST_PATH_FACTORY_HPP
00012
00013 #include <string>
00014 #include <memory>
00015
00016 #include "bellman_ford_strategy.hpp"
00017 #include "desopopape_strategy.hpp"
00018
00019 using namespace std;
00020
00043 inline unique_ptr<IShortestPathStrategy> createShortestPathStrategy(const string& strategyName) {
00044     if (strategyName == "Bellman-Ford") {
00045         return make_unique<BellmanFordStrategy>();
00046     }
00047
00048     if (strategyName == "D'Esopo-Pape") {
00049         return make_unique<DEsopoPapeStrategy>();
00050     }
00051
00052     throw runtime_error(
00053         "Unsupported shortest path algorithm: "
00054         + strategyName
00055     );
00056 }
00057
00058 #endif // ISHORTEST_PATH_FACTORY_HPP

```

9.32 engine/engines/network_flow/ishortest_path_strategy.hpp File Reference

Shortest-path strategy interface.

```

#include <vector>
#include "flow_network.hpp"

```

Include dependency graph for ishortest_path_strategy.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class [IShortestPathStrategy](#)
Shortest-path strategy interface.

9.32.1 Detailed Description

Shortest-path strategy interface.

Defines the common interface implemented by all shortest-path algorithms used by the flow optimization engine.

9.33 ishortest_path_strategy.hpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #ifndef ISHORTEST_PATH_STRATEGY_HPP
00011 #define ISHORTEST_PATH_STRATEGY_HPP
00012
00013 #include <vector>
00014
00015 #include "flow_network.hpp"
00016
00017 using namespace std;
00018
00026 class IShortestPathStrategy {
00027 public:
00028
00057     virtual bool findPath(
00058         FlowNetwork& flowNetwork,
00059         int source,
00060         int sink,
00061         vector<int>& parentVertex,
00062         vector<size_t>& parentEdge,
00063         vector<int>& distance
00064     ) = 0;
00065
00066     virtual ~IShortestPathStrategy() = default;
00067 };
00068
00069 #endif // ISHORTEST_PATH_STRATEGY_HPP

```

9.34 engine/engines/range_queries/brute_force.cpp File Reference

Brute-force range query algorithm.

```

#include <iostream>
#include <vector>
#include "range_utils.hpp"
#include "foundation/core_utils_json.hpp"
#include "foundation/core_utils_stopwatch.hpp"

```

Include dependency graph for brute_force.cpp:

Functions

- [Mine findLoudest](#) (const vector< [Mine](#) > &minePoints, size_t from, size_t to)
Finds the loudest mine within a range.
- int [main](#) (int argc, char *argv[])
Application entry point.

9.34.1 Detailed Description

Brute-force range query algorithm.

Implements a linear scan approach for locating the loudest mine within a specified boundary range.

9.34.2 Function Documentation

9.34.2.1 findLoudest()

```
Mine findLoudest (
    const vector< Mine > & minePoints,
    size_t from,
    size_t to)
```

Finds the loudest mine within a range.

Performs a linear scan over the specified interval and returns the mine with the highest loudness.

Circular indexing is applied to support ranges that wrap around the convex hull boundary.

Parameters

<i>minePoints</i>	Ordered boundary mines.
<i>from</i>	Start index of the query range.
<i>to</i>	End index of the query range.

Returns

[Mine](#) Loudest mine within the specified interval.

Here is the caller graph for this function:

9.34.2.2 main()

```
int main (
    int argc,
    char * argv[ ])
```

Application entry point.

Loads input data, executes the brute-force range query algorithm, and exports the result as a JSON response.

Parameters

<i>argc</i>	Number of command-line arguments.
<i>argv</i>	Command-line arguments.

Returns

int EXIT_SUCCESS on success, otherwise EXIT_FAILURE.

Here is the call graph for this function:

9.35 engine/engines/range_queries/range_utils.hpp File Reference

```
#include <string>
#include <limits>
#include <nlohmann/json.hpp>
#include "../convex_hull/geometry_utils.hpp"
```

Include dependency graph for range_utils.hpp: This graph shows which files directly or indirectly include this file:

Classes

- struct [Mine](#)

Functions

- [Mine loudnessMax](#) (const [Mine](#) &a, const [Mine](#) &b)

Returns the louder mine.

Variables

- const [Mine](#) [neutralMine](#)

Neutral mine value.

9.35.1 Function Documentation

9.35.1.1 loudnessMax()

```
Mine loudnessMax (
    const Mine & a,
    const Mine & b) [inline]
```

Returns the louder mine.

Compares two mines and returns the one with the greater loudness value.

Parameters

<i>a</i>	First mine.
<i>b</i>	Second mine.

Returns

[Mine](#) [Mine](#) with the highest loudness.

Here is the caller graph for this function:

9.35.2 Variable Documentation

9.35.2.1 neutralMine

```
const Mine neutralMine [inline]
```

Initial value:

```

    = {
        "",
        numeric_limits<int>::min(),
        {
            "",
            0.0,
            0.0
        }
    }
}
```

Neutral mine value.

Sentinel object used as an identity element during maximum loudness queries.

9.36 range_utils.hpp

[Go to the documentation of this file.](#)

```

00001
00008
00009 #ifndef MINE_UTILS_HPP
00010 #define MINE_UTILS_HPP
00011
00012 #include <string>
00013 #include <limits>
00014
00015 #include <nlohmann/json.hpp>
00016
00017 #include "../convex_hull/geometry_utils.hpp"
00018
00019 using namespace std;
00020 using json = nlohmann::json;
00021
00022 struct Mine {
00023     string id;
00024
00025     int loudness{0};
00026
00027     Vertex vertex;
00028 };
00029
00030 inline const Mine neutralMine = {
00031     "",
00032     numeric_limits<int>::min(),
00033     {
00034         "",
00035         0.0,
00036         0.0
00037     }
00038 };
00039
00040 inline Mine loudnessMax(const Mine& a, const Mine& b) {
00041     if (a.loudness >= b.loudness) {
00042         return a;
00043     }
00044     return b;
00045 }
00046
00047 #endif // MINE_UTILS_HPP
```

9.37 engine/engines/range_queries/segment_tree.cpp File Reference

Segment tree range query algorithm.

```
#include <iostream>
#include <algorithm>
#include <vector>
#include "range_utils.hpp"
#include "foundation/core_utils_json.hpp"
#include "foundation/core_utils_stopwatch.hpp"
Include dependency graph for segment_tree.cpp:
```

Functions

- void `buildTree` (vector< `Mine` > &segmentTree, const vector< `Mine` > &mines, size_t nodeIndex, size_t nodeLeft, size_t nodeRight)
Builds a segment tree.
- `Mine queryRange` (const vector< `Mine` > &segmentTree, size_t nodeIndex, size_t nodeLeft, size_t nodeRight, size_t queryLeft, size_t queryRight)
Executes a segment tree range query.
- `Mine queryModuloRange` (const vector< `Mine` > &segmentTree, size_t totalMines, size_t from, size_t to)
Executes a circular range query.
- int `main` (int argc, char *argv[])
Application entry point.

9.37.1 Detailed Description

Segment tree range query algorithm.

Implements an efficient range maximum query structure used to locate the loudest mine along the kingdom boundary.

9.37.2 Function Documentation

9.37.2.1 buildTree()

```
void buildTree (
    vector< Mine > & segmentTree,
    const vector< Mine > & mines,
    size_t nodeIndex,
    size_t nodeLeft,
    size_t nodeRight)
```

Builds a segment tree.

Recursively constructs a segment tree storing the loudest mine for every interval.

Parameters

<i>segmentTree</i>	Segment tree storage.
<i>mines</i>	Input mine collection.
<i>nodeIndex</i>	Current tree node index.
<i>nodeLeft</i>	Left boundary of the represented interval.
<i>nodeRight</i>	Right boundary of the represented interval.

Here is the call graph for this function: Here is the caller graph for this function:

9.37.2.2 main()

```
int main (
    int argc,
    char * argv[])
```

Application entry point.

Loads boundary data, constructs a segment tree, executes a range maximum query, and exports the result as a JSON response.

Parameters

<i>argc</i>	Number of command-line arguments.
<i>argv</i>	Command-line arguments.

Returns

int EXIT_SUCCESS on success, otherwise EXIT_FAILURE.

Here is the call graph for this function:

9.37.2.3 queryModuloRange()

```
Mine queryModuloRange (
    const vector< Mine > & segmentTree,
    size_t totalMines,
    size_t from,
    size_t to)
```

Executes a circular range query.

Supports boundary intervals that wrap around the convex hull by combining one or two segment tree queries.

Parameters

<i>segmentTree</i>	Segment tree storage.
--------------------	-----------------------

<i>totalMines</i>	Number of mines stored in the structure.
<i>from</i>	Query interval start.
<i>to</i>	Query interval end.

Returns

[Mine](#) Loudest mine within the circular interval.

Here is the call graph for this function: [Here is the caller graph for this function:](#)

9.37.2.4 queryRange()

```
Mine queryRange (
    const vector< Mine > & segmentTree,
    size_t nodeIndex,
    size_t nodeLeft,
    size_t nodeRight,
    size_t queryLeft,
    size_t queryRight)
```

Executes a segment tree range query.

Searches the specified interval and returns the mine with the greatest loudness value.

Parameters

<i>segmentTree</i>	Segment tree storage.
<i>nodeIndex</i>	Current tree node index.
<i>nodeLeft</i>	Left boundary of the current node interval.
<i>nodeRight</i>	Right boundary of the current node interval.
<i>queryLeft</i>	Query interval start.
<i>queryRight</i>	Query interval end.

Returns

[Mine](#) Loudest mine within the query interval.

Here is the call graph for this function: [Here is the caller graph for this function:](#)

9.38 engine/foundation/core_utils_json.hpp File Reference

```
#include <fstream>
#include <string>
#include <utility>
#include <vector>
#include <nlohmann/json.hpp>
#include "engines/convex_hull/geometry_utils.hpp"
#include "engines/range_queries/range_utils.hpp"
#include "engines/network_flow/flow_optimizer.hpp"
Include dependency graph for core_utils_json.hpp: This graph shows which files directly or indirectly include this file:
```

Functions

- `json readInput` (const string &filePath)
Reads and parses a JSON input file.
- `vector< Vertex > extractPoints` (const json &inputData)
Extracts mine locations as geometry vertices.
- `vector< Mine > extractMines` (const json &inputData)
Extracts mines for range query processing.
- `pair< size_t, size_t > extractRange` (const json &inputData, size_t totalVertices)
Extracts a circular range query interval.
- `pair< string, string > extractAlgorithms` (const json &inputData)
Extracts selected optimization algorithms.
- `json buildMineOutput` (const Mine &mine, const string &key, double executionTimeMs)
Creates a range query result payload.
- `json buildConvexOutput` (const vector< Vertex > &convexHull, double executionTimeMs)
Creates a convex hull result payload.
- `json buildAssignments` (const json &inputData, const vector< vector< Edge > > &graph, size_t minersCount, size_t minesCount)
Builds miner-to-mine assignment records.
- `json buildFlowOutput` (const FlowStatistics &flowResult, const json &assignments, const string &maxflowAlgorithm, const string &mincostAlgorithm, double executionTimeMs)
Creates a flow optimization result payload.

9.38.1 Function Documentation

9.38.1.1 buildAssignments()

```
json buildAssignments (
    const json & inputData,
    const vector< vector< Edge > > & graph,
    size_t minersCount,
    size_t minesCount) [inline]
```

Builds miner-to-mine assignment records.

Traverses the residual flow network and extracts successful miner assignments represented by flow edges connecting miners to mining facilities.

Parameters

<i>inputData</i>	Original input dataset.
<i>graph</i>	Residual flow network.
<i>minersCount</i>	Number of miners in the network.
<i>minesCount</i>	Number of mines in the network.

Returns

json Collection of assignment records.

Here is the caller graph for this function:

9.38.1.2 buildConvexOutput()

```
json buildConvexOutput (
    const vector< Vertex > & convexHull,
    double executionTimeMs) [inline]
```

Creates a convex hull result payload.

Converts a convex hull into a JSON representation together with geometry algorithm execution timing.

Parameters

<i>convexHull</i>	Convex hull vertices.
<i>executionTimeMs</i>	Algorithm execution time in milliseconds.

Returns

json Serialized convex hull result.

Here is the caller graph for this function:

9.38.1.3 buildFlowOutput()

```
json buildFlowOutput (
    const FlowStatistics & flowResult,
    const json & assignments,
    const string & maxflowAlgorithm,
    const string & mincostAlgorithm,
    double executionTimeMs) [inline]
```

Creates a flow optimization result payload.

Builds a complete JSON response containing optimization statistics, selected algorithms, execution timings, and miner assignment data.

Parameters

<i>flowResult</i>	Flow optimization statistics.
<i>assignments</i>	Generated miner assignments.
<i>maxflowAlgorithm</i>	Selected maximum-flow algorithm.
<i>mincostAlgorithm</i>	Selected minimum-cost pathfinding algorithm.
<i>executionTimeMs</i>	Total executable runtime in milliseconds.

Returns

json Serialized flow optimization result.

Here is the caller graph for this function:

9.38.1.4 buildMineOutput()

```
json buildMineOutput (
    const Mine & mine,
    const string & key,
    double executionTimeMs) [inline]
```

Creates a range query result payload.

Builds a JSON response containing information about a detected mine together with algorithm execution timing statistics.

Parameters

<i>mine</i>	Mine included in the result.
<i>key</i>	JSON field used to store the result entry.
<i>executionTimeMs</i>	Algorithm execution time in milliseconds.

Returns

json Serialized range query result.

Here is the caller graph for this function:

9.38.1.5 extractAlgorithms()

```
pair< string, string > extractAlgorithms (
    const json & inputData) [inline]
```

Extracts selected optimization algorithms.

Reads the configured maximum-flow and minimum-cost algorithms from the input configuration.

Parameters

<i>inputData</i>	Parsed input dataset.
------------------	-----------------------

Returns

pair<string, string> Selected flow and pathfinding algorithms.

Here is the caller graph for this function:

9.38.1.6 extractMines()

```
vector< Mine > extractMines (
    const json & inputData) [inline]
```

Extracts mines for range query processing.

Converts serialized boundary points into [Mine](#) objects used by range query algorithms.

Parameters

<i>inputData</i>	Parsed input dataset.
------------------	-----------------------

Returns

vector<Mine> Collection of extracted mines.

Here is the caller graph for this function:

9.38.1.7 extractPoints()

```
vector< Vertex > extractPoints (
    const json & inputData) [inline]
```

Extracts mine locations as geometry vertices.

Converts mining facility entries into vertex structures used by convex hull algorithms.

Parameters

<i>inputData</i>	Parsed input dataset.
------------------	-----------------------

Returns

vector<Vertex> Collection of extracted vertices.

Here is the caller graph for this function:

9.38.1.8 extractRange()

```
pair< size_t, size_t > extractRange (
    const json & inputData,
    size_t totalVertices) [inline]
```

Extracts a circular range query interval.

Reads the requested range boundaries and adjusts wrap-around intervals when necessary.

Parameters

<i>inputData</i>	Parsed input dataset.
<i>totalVertices</i>	Total number of boundary vertices.

Returns

pair<size_t, size_t> Range query interval.

Here is the caller graph for this function:

9.38.1.9 readInput()

```
json readInput (
    const string & filePath) [inline]
```

Reads and parses a JSON input file.

Loads a JSON document from disk and converts it into a nlohmann::json object.

Parameters

<i>filePath</i>	Path to the input file.
-----------------	-------------------------

Returns

json Parsed JSON document.

Exceptions

<i>std::runtime_error</i>	Raised when the file cannot be opened.
---------------------------	--

Here is the caller graph for this function:

9.39 core_utils_json.hpp

[Go to the documentation of this file.](#)

```
00001
00009
00010 #ifndef CORE_UTILS_JSON_HPP
00011 #define CORE_UTILS_JSON_HPP
00012
00013 #include <fstream>
00014 #include <string>
00015 #include <utility>
00016 #include <vector>
00017
00018 #include <nlohmann/json.hpp>
00019
00020 #include "engines/convex_hull/geometry_utils.hpp"
00021 #include "engines/range_queries/range_utils.hpp"
00022 #include "engines/network_flow/flow_optimizer.hpp"
00023
```

```

00024 using namespace std;
00025 using json = nlohmann::json;
00026
00042 inline json readInput(const string& filePath) {
00043     ifstream inputStream(filePath);
00044
00045     if (!inputStream) {
00046         throw runtime_error("Failed to open file: " + filePath);
00047     }
00048
00049     json inputData;
00050     inputStream » inputData;
00051
00052     return inputData;
00053 }
00054
00067 inline vector<Vertex> extractPoints(const json& inputData) {
00068     vector<Vertex> vertices;
00069
00070     for (const auto& node : inputData["mines"]) {
00071         vertices.push_back({
00072             node.value("id", ""),
00073             node.value("x", 0.0),
00074             node.value("y", 0.0)
00075         });
00076     }
00077
00078     return vertices;
00079 }
00080
00093 inline vector<Mine> extractMines(const json& inputData) {
00094     vector<Mine> mines;
00095
00096     for (const auto& node : inputData["points"]) {
00097         mines.push_back({
00098             node.value("id", ""),
00099             node.value("loudness", 0),
00100             {
00101                 "",
00102                 node.value("x", 0.0),
00103                 node.value("y", 0.0)
00104             }
00105         });
00106     }
00107
00108     return mines;
00109 }
00110
00126 inline pair<size_t, size_t> extractRange(const json& inputData, size_t totalVertices) {
00127     const size_t from =
00128         static_cast<size_t>(
00129             inputData["range"][0]
00130         ) - 1;
00131
00132     size_t to =
00133         static_cast<size_t>(
00134             inputData["range"][1]
00135         ) - 1;
00136
00137     if (to < from) {
00138         to += totalVertices;
00139     }
00140
00141     return {
00142         from,
00143         to
00144     };
00145 }
00146
00159 inline pair<string, string> extractAlgorithms(const json& inputData) {
00160     const string maxflowAlgorithm =
00161         inputData["config"]
00162             ["flow_algorithm"];
00163
00164     const string mincostAlgorithm =
00165         inputData["config"]
00166             ["cost_algorithm"];
00167
00168     return {
00169         maxflowAlgorithm,
00170         mincostAlgorithm
00171     };
00172 }
00173
00193 inline json buildMineOutput(const Mine& mine, const string& key, double executionTimeMs) {
00194     json result;
00195

```

```

00196     result["execution_time_ms"] = executionTimeMs;
00197
00198     result[key].push_back({
00199         {"id", mine.id},
00200         {"x", mine.vertex.x},
00201         {"y", mine.vertex.y},
00202         {"loudness", mine.loudness}
00203     });
00204
00205     return result;
00206 }
00207
00223 inline json buildConvexOutput(const vector<Vertex>& convexHull, double executionTimeMs) {
00224     json result;
00225
00226     result["execution_time_ms"] = executionTimeMs;
00227
00228     for (const auto& vertex : convexHull) {
00229         result["convex_hull"].push_back({
00230             {"id", vertex.id},
00231             {"x", vertex.x},
00232             {"y", vertex.y}
00233         });
00234     }
00235
00236     return result;
00237 }
00238
00261 inline json buildAssignments(
00262     const json& inputData,
00263     const vector<vector<Edge>& graph,
00264     size_t minersCount,
00265     size_t minesCount
00266 ) {
00267     json result;
00268
00269     for (size_t i = 0; i < minersCount; ++i) {
00270         const auto& miner =
00271             inputData["miners"][i];
00272
00273         for (const auto& edge : graph[i + 1]) {
00274             const bool isMineNode =
00275                 edge.destination > minersCount &&
00276                 edge.destination <=
00277                     minersCount + minesCount;
00278
00279             const bool hasFlow =
00280                 edge.capacity == 0;
00281
00282             if (!isMineNode || !hasFlow) {
00283                 continue;
00284             }
00285
00286             const int mineIndex =
00287                 edge.destination -
00288                 minersCount -
00289                 1;
00290
00291             const auto& mine =
00292                 inputData["mines"]
00293                     [mineIndex];
00294
00295             result.push_back({
00296                 {"miner", miner["id"]},
00297
00298                 {"miner_x", miner["x"]},
00299                 {"miner_y", miner["y"]},
00300
00301                 {"mine", mine["id"]},
00302
00303                 {"mine_x", mine["x"]},
00304                 {"mine_y", mine["y"]}
00305             });
00306         }
00307     }
00308
00309     return result;
00310 }
00311
00337 inline json buildFlowOutput(
00338     const FlowStatistics& flowResult,
00339     const json& assignments,
00340     const string& maxflowAlgorithm,
00341     const string& mincostAlgorithm,
00342     double executionTimeMs
00343 ) {
00344     json result;

```

```

00345
00346     result["pathfinding_time_ms"] =
00347         flowResult.pathfindingTimeMs;
00348
00349     result["augmentation_time_ms"] =
00350         flowResult.augmentationTimeMs;
00351
00352     result["total_algorithm_time_ms"] =
00353         flowResult.totalTimeMs;
00354
00355     result["execution_time_ms"] =
00356         executionTimeMs;
00357
00358     result["maxflow_algorithm"] =
00359         maxflowAlgorithm;
00360
00361     result["mincost_algorithm"] =
00362         mincostAlgorithm;
00363
00364     result["max_flow"] =
00365         flowResult.maximumFlow;
00366
00367     result["min_cost"] =
00368         flowResult.minimumCost;
00369
00370     result["assignments"] =
00371         assignments;
00372
00373     return result;
00374 }
00375
00376 #endif // CORE_UTILS_JSON_HPP

```

9.40 engine/foundation/core_utils_stopwatch.hpp File Reference

```
#include <chrono>
```

Include dependency graph for core_utils_stopwatch.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class [Stopwatch](#)
High-resolution stopwatch.

9.41 core_utils_stopwatch.hpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #ifndef CORE_UTILS_STOPWATCH_HPP
00011 #define CORE_UTILS_STOPWATCH_HPP
00012
00013 #include <chrono>
00014
00015 using namespace std;
00016 using namespace chrono;
00017
00024 class Stopwatch {
00025 public:
00026
00033     Stopwatch()
00034         : startTime(high_resolution_clock::now()) {}
00035
00042     void restart() {
00043         startTime = high_resolution_clock::now();
00044     }
00045
00055     double elapsedMilliseconds() const {
00056         const auto currentTime =

```

```

00057         high_resolution_clock::now();
00058
00059         return duration<double, milli>(
00060             currentTime - startTime
00061         ).count();
00062     }
00063
00064 private:
00065
00067     high_resolution_clock::time_point startTime;
00068 };
00069
00070 #endif // CORE_UTILS_STOPWATCH_HPP

```

9.42 engine/orchestration/execution_engine.py File Reference

Native algorithm execution utilities.

Namespaces

- namespace [execution_engine](#)

Functions

- Path [execution_engine.get_executable_path](#) (str executable_name)
Resolves an executable path.
- dict[str, object] [execution_engine.run_cpp_algorithm](#) (str algorithm_name, WorldData|dict[str, object] input↔_data, tuple[int, int]|None value_range=None)
Executes a native algorithm executable.

Variables

- tuple [execution_engine.EXECUTABLE_DIRECTORY](#)
Directory containing compiled executables.
- tuple [execution_engine.PLATFORM_EXECUTABLE_SUFFIX](#)
Platform-specific executable extension.
- dict [execution_engine.EXECUTABLES](#)
Registry of available native algorithms.

9.42.1 Detailed Description

Native algorithm execution utilities.

Provides functionality for locating, invoking, and communicating with external C++ algorithm executables used by the Aurumveil platform.

9.43 engine/orchestration/runtime_initializer.py File Reference

Runtime initialization and build utilities.

Namespaces

- namespace [runtime_initializer](#)

Functions

- None [runtime_initializer.run_process](#) (list[str] command, Path working_directory)
Executes an external process and streams its output.
- None [runtime_initializer.clear_terminal](#) ()
Clears the terminal window.
- None [runtime_initializer.build_application](#) ()
Configures and builds the native engine.

Variables

- [runtime_initializer.level](#)
- [runtime_initializer.format](#)
- tuple [runtime_initializer.PROCESS_FLAGS](#)
Platform-specific process creation flags.

9.43.1 Detailed Description

Runtime initialization and build utilities.

Provides functionality for configuring, building, and launching the native algorithm engine used by the Aurumveil platform.

9.44 interface/content_panel.py File Reference

Visualization and dataset management panel.

Classes

- class [content_panel.ContentPanel](#)

Namespaces

- namespace [content_panel](#)

9.44.1 Detailed Description

Visualization and dataset management panel.

Provides graphical data visualization, dataset management operations, logging facilities, and result presentation for the Aurumveil platform.

9.45 interface/control_panel.py File Reference

Analysis and execution control panel.

Classes

- class [control_panel.ControlPanel](#)
Analysis control panel.

Namespaces

- namespace [control_panel](#)

Variables

- dict [control_panel.MAXFLOW_NAMES](#)
- dict [control_panel.MINCOST_NAMES](#)

9.45.1 Detailed Description

Analysis and execution control panel.

Provides algorithm configuration, execution, benchmarking, caching, and result management functionality for the Aurumveil platform.

9.46 interface/data_dialog.py File Reference

Data management dialogue for the Aurumveil platform.

Classes

- class [data_dialog.DataDialog](#)
World data management dialogue.

Namespaces

- namespace [data_dialog](#)

Variables

- list [data_dialog.RESOURCE](#)

9.46.1 Detailed Description

Data management dialogue for the Aurumveil platform.

Provides functionality for viewing, editing, validating, and persisting miners, mines, and wardens stored within the application data model.

9.47 interface/main_window.py File Reference

Main application window for the Aurumveil platform.

Classes

- class [main_window.MainWindow](#)
Main application window.

Namespaces

- namespace [main_window](#)

9.47.1 Detailed Description

Main application window for the Aurumveil platform.

Defines the primary user interface container responsible for hosting the content and control panels, managing inter-panel communication, and handling responsive user interface scaling.

9.48 interface/widget_effects.py File Reference

Utility functions for Qt widget visual effects.

Namespaces

- namespace [widget_effects](#)

Functions

- None [widget_effects.apply_shadow](#) (QWidget widget)
Applies a drop shadow effect to a widget.

9.48.1 Detailed Description

Utility functions for Qt widget visual effects.

Provides reusable helpers responsible for enhancing the appearance of user interface components.

9.49 main.py File Reference

Application entry point for the Aurumveil platform.

Namespaces

- namespace [main](#)

Functions

- None [main.pause_terminal](#) ()
Pauses the terminal before application shutdown.
- int [main.launch_application](#) ()
Builds and launches the Aurumveil application.

Variables

- [main.level](#)
- [main.format](#)

9.49.1 Detailed Description

Application entry point for the Aurumveil platform.

Responsible for runtime initialization, Qt application configuration, and launching the main graphical interface.

9.50 stylesheet.py File Reference

Global Qt stylesheet for the Aurumveil application.

Namespaces

- namespace [stylesheet](#)

Variables

- str [stylesheet.APP_STYLESHEET](#)
Global application stylesheet.

9.50.1 Detailed Description

Global Qt stylesheet for the Aurumveil application.

Defines the visual appearance of all user interface components used throughout the application.

Index

- `__init__`
 - `content_panel.ContentPanel`, 36
 - `control_panel.ControlPanel`, 50
 - `data_dialog.DataDialog`, 74
 - `huffman_codec.HuffmanNode`, 94
 - `main_window.MainWindow`, 101
 - `__lt__`
 - `huffman_codec.HuffmanNode`, 95
 - `_add_mine`
 - `data_dialog.DataDialog`, 74
 - `_add_miner`
 - `data_dialog.DataDialog`, 74
 - `_branch_row`
 - `control_panel.ControlPanel`, 51
 - `_build_execution_signature`
 - `control_panel.ControlPanel`, 51
 - `_build_export_payload`
 - `control_panel.ControlPanel`, 51
 - `_build_identifier_item`
 - `data_dialog.DataDialog`, 75
 - `_build_legend`
 - `content_panel.ContentPanel`, 37
 - `_build_ordered_payload_from_hull`
 - `control_panel.ControlPanel`, 52
 - `_build_table`
 - `data_dialog.DataDialog`, 75
 - `_build_toolbar`
 - `content_panel.ContentPanel`, 37
 - `_calculate_total_execution_time`
 - `control_panel.ControlPanel`, 52
 - `_clear_data`
 - `content_panel.ContentPanel`, 37
 - `_clear_graph`
 - `content_panel.ContentPanel`, 37
 - `_clear_mark`
 - `data_dialog.DataDialog`, 75
 - `_collect_execution_errors`
 - `control_panel.ControlPanel`, 53
 - `_configure_graph`
 - `content_panel.ContentPanel`, 37
 - `_delete_selected_mines`
 - `data_dialog.DataDialog`, 76
 - `_draw_assignments`
 - `content_panel.ContentPanel`, 38
 - `_draw_convex_hull`
 - `content_panel.ContentPanel`, 38
 - `_draw_miners`
 - `content_panel.ContentPanel`, 38
 - `_draw_mines`
 - `content_panel.ContentPanel`, 39
 - `_draw_points`
 - `content_panel.ContentPanel`, 39
 - `_ensure_data_directories`
 - `repository`, 25
 - `_execute_algorithms`
 - `control_panel.ControlPanel`, 53
 - `_execute_dwarf_search`
 - `control_panel.ControlPanel`, 54
 - `_export_compressed_payload`
 - `control_panel.ControlPanel`, 54
 - `_extract_dwarf_data`
 - `control_panel.ControlPanel`, 55
 - `_finalize_execution`
 - `control_panel.ControlPanel`, 55
 - `_get_cached_result`
 - `content_panel.ContentPanel`, 39
 - `_get_cell_text`
 - `data_dialog.DataDialog`, 76
 - `_get_content_panel`
 - `control_panel.ControlPanel`, 55
 - `_get_convex_hull_points`
 - `control_panel.ControlPanel`, 55
 - `_get_execution_count`
 - `control_panel.ControlPanel`, 56
 - `_get_result_file_path`
 - `control_panel.ControlPanel`, 56
 - `_get_selected`
 - `control_panel.ControlPanel`, 56
 - `_get_selected_algorithms`
 - `control_panel.ControlPanel`, 57
 - `_get_validated_dataset`
 - `control_panel.ControlPanel`, 57
 - `_handle_benchmark_results`
 - `control_panel.ControlPanel`, 57
 - `_handle_dwarf_search_error`
 - `control_panel.ControlPanel`, 58
 - `_handle_execution_errors`
 - `control_panel.ControlPanel`, 58
 - `_handle_execution_exception`
 - `control_panel.ControlPanel`, 59
 - `_handle_execution_success`
 - `control_panel.ControlPanel`, 59
 - `_handle_log`
 - `main_window.MainWindow`, 102
 - `_handle_multi_execution_results`
 - `control_panel.ControlPanel`, 59
 - `_handle_single_execution_results`
 - `control_panel.ControlPanel`, 60

- `_highlight_loudest_dwarf`
 - `control_panel.ControlPanel`, 60
- `_initialize_benchmark_metrics`
 - `control_panel.ControlPanel`, 60
- `_initialize_execution_status`
 - `control_panel.ControlPanel`, 61
- `_initialize_ui`
 - `content_panel.ContentPanel`, 40
 - `control_panel.ControlPanel`, 61
 - `data_dialog.DataDialog`, 76
 - `main_window.MainWindow`, 102
- `_load_file`
 - `content_panel.ContentPanel`, 40
- `_load_file_cache`
 - `control_panel.ControlPanel`, 61
- `_load_memory_cache`
 - `control_panel.ControlPanel`, 62
- `_load_to_tables`
 - `data_dialog.DataDialog`, 77
- `_log_assignment_summary`
 - `control_panel.ControlPanel`, 62
- `_log_boundary_points`
 - `control_panel.ControlPanel`, 63
- `_log_boundary_segments`
 - `control_panel.ControlPanel`, 63
- `_log_dwarf_result`
 - `control_panel.ControlPanel`, 63
- `_log_execution_completion`
 - `control_panel.ControlPanel`, 64
- `_log_flow_summary`
 - `control_panel.ControlPanel`, 64
- `_log_geometry_summary`
 - `control_panel.ControlPanel`, 65
- `_log_result_summary`
 - `control_panel.ControlPanel`, 65
- `_mark_cell`
 - `data_dialog.DataDialog`, 77
- `_on_data_changed`
 - `content_panel.ContentPanel`, 40
- `_open_data_dialog`
 - `content_panel.ContentPanel`, 40
- `_open_logs_window`
 - `content_panel.ContentPanel`, 41
- `_parse_range_expression`
 - `control_panel.ControlPanel`, 65
- `_prepare_graph`
 - `content_panel.ContentPanel`, 41
- `_refresh_identifiers`
 - `data_dialog.DataDialog`, 77
- `_refresh_visualization`
 - `content_panel.ContentPanel`, 41
- `_remove_selected_rows`
 - `data_dialog.DataDialog`, 78
- `_reset_execution_state`
 - `control_panel.ControlPanel`, 66
- `_run_benchmark_iterations`
 - `control_panel.ControlPanel`, 66
- `_safe_float`
 - `data_dialog.DataDialog`, 78
- `_safe_int`
 - `data_dialog.DataDialog`, 78
- `_safe_str`
 - `data_dialog.DataDialog`, 79
- `_save`
 - `content_panel.ContentPanel`, 41
 - `data_dialog.DataDialog`, 79
- `_save_as`
 - `content_panel.ContentPanel`, 42
- `_save_existing_dataset`
 - `content_panel.ContentPanel`, 42
- `_save_new_raw_dataset`
 - `content_panel.ContentPanel`, 42
- `_set_status`
 - `control_panel.ControlPanel`, 66
- `_show_save_error`
 - `content_panel.ContentPanel`, 43
- `_terminal_row`
 - `control_panel.ControlPanel`, 67
- `_toggle_data_labels`
 - `content_panel.ContentPanel`, 43
- `_tree_prefix`
 - `control_panel.ControlPanel`, 67
- `_update_benchmark_metrics`
 - `control_panel.ControlPanel`, 68
- `_update_buttons_state`
 - `control_panel.ControlPanel`, 68
- `_update_delete_buttons`
 - `data_dialog.DataDialog`, 79
- `_validate`
 - `data_dialog.DataDialog`, 80
- `_validate_dataset_for_save`
 - `content_panel.ContentPanel`, 43
- `_validate_dwarf_search`
 - `control_panel.ControlPanel`, 68
- `_validate_json_structure`
 - `repository`, 25
- `~IMaxFlowStrategy`
 - `IMaxFlowStrategy`, 96
- `~IShortestPathStrategy`
 - `IShortestPathStrategy`, 100
- `addEdge`
 - `FlowOptimizer`, 88
- `APP_STYLESHEET`
 - `stylesheet`, 32
- `append_system_log`
 - `content_panel.ContentPanel`, 43
- `apply_shadow`
 - `widget_effects`, 32
- `augment`
 - `EdmondsKarpStrategy`, 86
 - `FordFulkersonStrategy`, 92
 - `IMaxFlowStrategy`, 97
- `augmentationCount`
 - `FlowStatistics`, 91
- `augmentationTimeMs`
 - `FlowStatistics`, 91

- averagePathLength
 - FlowStatistics, 91
- background_widget
 - main_window.MainWindow, 103
- BASE_DIR
 - repository, 29
- BellmanFordStrategy, 33
 - findPath, 34
- brute_force.cpp
 - findLoudest, 146
 - main, 146
- btn_cached
 - content_panel.ContentPanel, 45
- btn_recompute
 - content_panel.ContentPanel, 45
- build_application
 - runtime_initializer, 30
- build_codes
 - huffman_codec, 21
- build_tree
 - huffman_codec, 21
- buildAssignments
 - core_utils_json.hpp, 152
- buildConvexOutput
 - core_utils_json.hpp, 152
- buildFlowOutput
 - core_utils_json.hpp, 153
- buildMineOutput
 - core_utils_json.hpp, 153
- buildTree
 - segment_tree.cpp, 149
- cache_hit
 - control_panel.ControlPanel, 70
- cached_result
 - control_panel.ControlPanel, 70
- calculatePathLength
 - EdmondsKarpStrategy, 86
 - IMaxFlowStrategy, 97
- canvas
 - content_panel.ContentPanel, 45
- capacity
 - Edge, 84
- clear_terminal
 - runtime_initializer, 30
- comparePolar
 - geometry_utils.cpp, 115
 - geometry_utils.hpp, 118
- compareVertices
 - geometry_utils.cpp, 115
 - geometry_utils.hpp, 118
- compress_text
 - huffman_codec, 22
- computeConvexHullGraham
 - convex_hull.hpp, 112
 - graham_convex_hull.cpp, 122
- computeConvexHullJarvis
 - convex_hull.hpp, 113
 - jarvis_convex_hull.cpp, 123
- computeConvexHullMonotoneChain
 - convex_hull.hpp, 113
 - monotone_convex_hull.cpp, 125
- computeFlow
 - EdmondsKarpStrategy, 86
 - FordFulkersonStrategy, 93
 - IMaxFlowStrategy, 97
- content_panel, 17
 - main_window.MainWindow, 103
- content_panel.ContentPanel, 34
 - _init_, 36
 - _build_legend, 37
 - _build_toolbar, 37
 - _clear_data, 37
 - _clear_graph, 37
 - _configure_graph, 37
 - _draw_assignments, 38
 - _draw_convex_hull, 38
 - _draw_miners, 38
 - _draw_mines, 39
 - _draw_points, 39
 - _get_cached_result, 39
 - _initialize_ui, 40
 - _load_file, 40
 - _on_data_changed, 40
 - _open_data_dialog, 40
 - _open_logs_window, 41
 - _prepare_graph, 41
 - _refresh_visualization, 41
 - _save, 41
 - _save_as, 42
 - _save_existing_dataset, 42
 - _save_new_raw_dataset, 42
 - _show_save_error, 43
 - _toggle_data_labels, 43
 - _validate_dataset_for_save, 43
- append_system_log, 43
- btn_cached, 45
- btn_recompute, 45
- canvas, 45
- current_file_path, 45
- figure, 45
- highlighted_mine, 46
- is_cached_mode, 44
- log_index, 46
- mode_group, 46
- redraw_graph, 44
- set_highlighted_mine, 44
- show_labels, 46
- stats, 46
- toggle_labels_button, 46
- toolbar, 46
- control_panel, 17
 - main_window.MainWindow, 103
- MAXFLOW_NAMES, 17
- MINCOST_NAMES, 17
- control_panel.ControlPanel, 47

- [__init__, 50](#)
- [_branch_row, 51](#)
- [_build_execution_signature, 51](#)
- [_build_export_payload, 51](#)
- [_build_ordered_payload_from_hull, 52](#)
- [_calculate_total_execution_time, 52](#)
- [_collect_execution_errors, 53](#)
- [_execute_algorithms, 53](#)
- [_execute_dwarf_search, 54](#)
- [_export_compressed_payload, 54](#)
- [_extract_dwarf_data, 55](#)
- [_finalize_execution, 55](#)
- [_get_content_panel, 55](#)
- [_get_convex_hull_points, 55](#)
- [_get_execution_count, 56](#)
- [_get_result_file_path, 56](#)
- [_get_selected, 56](#)
- [_get_selected_algorithms, 57](#)
- [_get_validated_dataset, 57](#)
- [_handle_benchmark_results, 57](#)
- [_handle_dwarf_search_error, 58](#)
- [_handle_execution_errors, 58](#)
- [_handle_execution_exception, 59](#)
- [_handle_execution_success, 59](#)
- [_handle_multi_execution_results, 59](#)
- [_handle_single_execution_results, 60](#)
- [_highlight_loudest_dwarf, 60](#)
- [_initialize_benchmark_metrics, 60](#)
- [_initialize_execution_status, 61](#)
- [_initialize_ui, 61](#)
- [_load_file_cache, 61](#)
- [_load_memory_cache, 62](#)
- [_log_assignment_summary, 62](#)
- [_log_boundary_points, 63](#)
- [_log_boundary_segments, 63](#)
- [_log_dwarf_result, 63](#)
- [_log_execution_completion, 64](#)
- [_log_flow_summary, 64](#)
- [_log_geometry_summary, 65](#)
- [_log_result_summary, 65](#)
- [_parse_range_expression, 65](#)
- [_reset_execution_state, 66](#)
- [_run_benchmark_iterations, 66](#)
- [_set_status, 66](#)
- [_terminal_row, 67](#)
- [_tree_prefix, 67](#)
- [_update_benchmark_metrics, 68](#)
- [_update_buttons_state, 68](#)
- [_validate_dwarf_search, 68](#)
- [cache_hit, 70](#)
- [cached_result, 70](#)
- [dwarf_btn, 70](#)
- [execution_failed, 70](#)
- [execution_signature, 70](#)
- [geo_group, 70](#)
- [iterations_input, 70](#)
- [log_signal, 71](#)
- [maxflow_group, 71](#)
- [mincost_group, 71](#)
- [on_find_loudest_dwarf, 68](#)
- [on_start, 69](#)
- [range_input, 71](#)
- [refresh_graph_signal, 71](#)
- [result_label, 71](#)
- [run_algorithm, 69](#)
- [save_btn, 71](#)
- [save_result, 69](#)
- [seg_box, 72](#)
- [seg_group, 72](#)
- [start_btn, 72](#)
- [time_label, 72](#)
- [convex_hull.hpp](#)
 - [computeConvexHullGraham, 112](#)
 - [computeConvexHullJarvis, 113](#)
 - [computeConvexHullMonotoneChain, 113](#)
- [core_utils_json.hpp](#)
 - [buildAssignments, 152](#)
 - [buildConvexOutput, 152](#)
 - [buildFlowOutput, 153](#)
 - [buildMineOutput, 153](#)
 - [extractAlgorithms, 154](#)
 - [extractMines, 154](#)
 - [extractPoints, 155](#)
 - [extractRange, 155](#)
 - [readInput, 156](#)
- [cost](#)
 - [Edge, 84](#)
- [createMaxFlowStrategy](#)
 - [imaxflow_path_factory.hpp, 140](#)
- [createShortestPathStrategy](#)
 - [ishortest_path_factory.hpp, 143](#)
- [crossProduct](#)
 - [geometry_utils.cpp, 116](#)
 - [geometry_utils.hpp, 119](#)
- [current_file_path](#)
 - [content_panel.ContentPanel, 45](#)
- [data_changed](#)
 - [data_dialog.DataDialog, 81](#)
- [data_dialog, 18](#)
 - [RESOURCES, 18](#)
- [data_dialog.DataDialog, 73](#)
 - [__init__, 74](#)
 - [_add_mine, 74](#)
 - [_add_miner, 74](#)
 - [_build_identifier_item, 75](#)
 - [_build_table, 75](#)
 - [_clear_mark, 75](#)
 - [_delete_selected_mines, 76](#)
 - [_get_cell_text, 76](#)
 - [_initialize_ui, 76](#)
 - [_load_to_tables, 77](#)
 - [_mark_cell, 77](#)
 - [_refresh_identifiers, 77](#)
 - [_remove_selected_rows, 78](#)
 - [_safe_float, 78](#)
 - [_safe_int, 78](#)

- [_safe_str](#), 79
- [_save](#), 79
- [_update_delete_buttons](#), 79
- [_validate](#), 80
- [data_changed](#), 81
- [delete_mine_button](#), 81
- [delete_miner_button](#), 81
- [get_data](#), 80
- [miners_table](#), 81
- [mines_table](#), 81
- [wardens_table](#), 81
- DATA_DIRECTORY
 - [repository](#), 29
- [decompress_text](#)
 - [huffman_codec](#), 22
- DEFAULT_DATA_FILE
 - [repository](#), 29
- [delete_data_file](#)
 - [repository](#), 26
- [delete_mine_button](#)
 - [data_dialog.DataDialog](#), 81
- [delete_miner_button](#)
 - [data_dialog.DataDialog](#), 81
- DEsopoPapeStrategy, 82
 - [findPath](#), 83
- [destination](#)
 - [Edge](#), 84
- DISTANCE_INFINITY
 - [flow_network.hpp](#), 133
- [distanceSquared](#)
 - [geometry_utils.cpp](#), 116
 - [geometry_utils.hpp](#), 119
- [domain](#) Directory Reference, 14
- [domain/models.py](#), 109
- [domain/repository.py](#), 110
- [domain/serializer.py](#), 110
- [dwarf_btn](#)
 - [control_panel.ControlPanel](#), 70
- [Edge](#), 84
 - [capacity](#), 84
 - [cost](#), 84
 - [destination](#), 84
 - [reverseEdgeIndex](#), 84
- [EdmondsKarpStrategy](#), 85
 - [augment](#), 86
 - [calculatePathLength](#), 86
 - [computeFlow](#), 86
- [elapsedMilliseconds](#)
 - [Stopwatch](#), 106
- [engine](#) Directory Reference, 14
- [engine/compression](#) Directory Reference, 13
 - [huffman_codec.py](#), 111
- [engine/engines](#) Directory Reference, 14
 - [engine/engines/convex_hull](#) Directory Reference, 13
 - [convex_hull/convex_hull.hpp](#), 112, 114
 - [convex_hull/geometry_utils.cpp](#), 115
 - [convex_hull/geometry_utils.hpp](#), 117, 120
 - [engine/engines/convex_hull/graham_convex_hull.cpp](#), 121
 - [engine/engines/convex_hull/jarvis_convex_hull.cpp](#), 123
 - [engine/engines/convex_hull/monotone_convex_hull.cpp](#), 124
 - [engine/engines/network_flow](#) Directory Reference, 15
 - [engine/engines/network_flow/bellman_ford_strategy.hpp](#), 126, 127
 - [engine/engines/network_flow/desopopape_strategy.hpp](#), 128, 129
 - [engine/engines/network_flow/edmonds_karp_strategy.hpp](#), 130, 131
 - [engine/engines/network_flow/flow_network.hpp](#), 132, 133
 - [engine/engines/network_flow/flow_optimizer.cpp](#), 134
 - [engine/engines/network_flow/flow_optimizer.hpp](#), 135, 136
 - [engine/engines/network_flow/ford_fulkerson_strategy.hpp](#), 138
 - [engine/engines/network_flow/imaxflow_path_factory.hpp](#), 139, 141
 - [engine/engines/network_flow/imaxflow_path_strategy.hpp](#), 141, 142
 - [engine/engines/network_flow/ishortest_path_factory.hpp](#), 142, 144
 - [engine/engines/network_flow/ishortest_path_strategy.hpp](#), 144, 145
 - [engine/engines/range_queries](#) Directory Reference, 16
 - [engine/engines/range_queries/brute_force.cpp](#), 145
 - [engine/engines/range_queries/range_utils.hpp](#), 147, 148
 - [engine/engines/range_queries/segment_tree.cpp](#), 149
- [engine/foundation](#) Directory Reference, 14
 - [engine/foundation/core_utils_json.hpp](#), 151, 156
 - [engine/foundation/core_utils_stopwatch.hpp](#), 159
- [engine/orchestration](#) Directory Reference, 15
 - [engine/orchestration/execution_engine.py](#), 160
 - [engine/orchestration/runtime_initializer.py](#), 160
- EPSILON
 - [geometry_utils.hpp](#), 120
- EXECUTABLE_DIRECTORY
 - [execution_engine](#), 20
- EXECUTABLES
 - [execution_engine](#), 20
- [execution_engine](#), 18
 - EXECUTABLE_DIRECTORY, 20
 - EXECUTABLES, 20
 - [get_executable_path](#), 19
 - PLATFORM_EXECUTABLE_SUFFIX, 20
 - [run_cpp_algorithm](#), 19
- [execution_failed](#)
 - [control_panel.ControlPanel](#), 70
- [execution_signature](#)
 - [control_panel.ControlPanel](#), 70
- [extractAlgorithms](#)
 - [core_utils_json.hpp](#), 154
- [extractMines](#)
 - [core_utils_json.hpp](#), 154

- extractPoints
 - core_utils_json.hpp, 155
- extractRange
 - core_utils_json.hpp, 155
- figure
 - content_panel.ContentPanel, 45
- findLoudest
 - brute_force.cpp, 146
- findPath
 - BellmanFordStrategy, 34
 - DEsopoPapeStrategy, 83
 - IShortestPathStrategy, 100
- flow_network.hpp
 - DISTANCE_INFINITY, 133
 - FlowNetwork, 133
 - INVALID_EDGE, 133
- flow_optimizer.cpp
 - json, 134
 - main, 134
- FlowNetwork
 - flow_network.hpp, 133
- flowNetwork
 - FlowOptimizer, 90
- FlowOptimizer, 87
 - addEdge, 88
 - flowNetwork, 90
 - FlowOptimizer, 88
 - getGraph, 89
 - maxflowStrategy, 90
 - mincostStrategy, 90
 - solve, 89
- FlowStatistics, 90
 - augmentationCount, 91
 - augmentationTimeMs, 91
 - averagePathLength, 91
 - maximumFlow, 91
 - minimumCost, 91
 - pathfindingTimeMs, 91
 - totalPathLength, 91
 - totalTimeMs, 91
- FordFulkersonStrategy, 92
 - augment, 92
 - computeFlow, 93
- format
 - main, 24
 - runtime_initializer, 31
- geo_group
 - control_panel.ControlPanel, 70
- geometry_utils.cpp
 - comparePolar, 115
 - compareVertices, 115
 - crossProduct, 116
 - distanceSquared, 116
- geometry_utils.hpp
 - comparePolar, 118
 - compareVertices, 118
 - crossProduct, 119
 - distanceSquared, 119
 - EPSILON, 120
- get_data
 - data_dialog.DataDialog, 80
- get_executable_path
 - execution_engine, 19
- getGraph
 - FlowOptimizer, 89
- graham_convex_hull.cpp
 - computeConvexHullGraham, 122
 - main, 122
- highlighted_mine
 - content_panel.ContentPanel, 46
- huffman_codec, 21
 - build_codes, 21
 - build_tree, 21
 - compress_text, 22
 - decompress_text, 22
- huffman_codec.HuffmanNode, 94
 - __init__, 94
 - __lt__, 95
 - left_branch, 95
 - right_branch, 95
 - symbol, 95
 - weight, 95
- id
 - Mine, 103
 - Vertex, 107
- imaxflow_path_factory.hpp
 - createMaxFlowStrategy, 140
- IMaxFlowStrategy, 96
 - ~IMaxFlowStrategy, 96
 - augment, 97
 - calculatePathLength, 97
 - computeFlow, 97
- interface Directory Reference, 15
- interface/content_panel.py, 161
- interface/control_panel.py, 162
- interface/data_dialog.py, 162
- interface/main_window.py, 163
- interface/widget_effects.py, 163
- INVALID_EDGE
 - flow_network.hpp, 133
- is_cached_mode
 - content_panel.ContentPanel, 44
- ishortest_path_factory.hpp
 - createShortestPathStrategy, 143
- IShortestPathStrategy, 99
 - ~IShortestPathStrategy, 100
 - findPath, 100
- iterations_input
 - control_panel.ControlPanel, 70
- jarvis_convex_hull.cpp
 - computeConvexHullJarvis, 123
 - main, 124
- json

- flow_optimizer.cpp, 134
- launch_application
 - main, 23
- left_branch
 - huffman_codec.HuffmanNode, 95
- level
 - main, 24
 - runtime_initializer, 31
- load_default_data
 - repository, 26
- load_from_file
 - repository, 26
- log_index
 - content_panel.ContentPanel, 46
- log_signal
 - control_panel.ControlPanel, 71
- loudness
 - Mine, 103
- loudnessMax
 - range_utils.hpp, 147
- main, 23
 - brute_force.cpp, 146
 - flow_optimizer.cpp, 134
 - format, 24
 - graham_convex_hull.cpp, 122
 - jarvis_convex_hull.cpp, 124
 - launch_application, 23
 - level, 24
 - monotone_convex_hull.cpp, 125
 - pause_terminal, 23
 - segment_tree.cpp, 150
- main.py, 164
- main_window, 24
- main_window.MainWindow, 101
 - __init__, 101
 - _handle_log, 102
 - _initialize_ui, 102
 - background_widget, 103
 - content_panel, 103
 - control_panel, 103
 - resizeEvent, 102
- maxflow_group
 - control_panel.ControlPanel, 71
- MAXFLOW_NAMES
 - control_panel, 17
- maxflowStrategy
 - FlowOptimizer, 90
- maximumFlow
 - FlowStatistics, 91
- mincost_group
 - control_panel.ControlPanel, 71
- MINCOST_NAMES
 - control_panel, 17
- mincostStrategy
 - FlowOptimizer, 90
- Mine, 103
 - id, 103
- loudness, 103
- vertex, 104
- miners
 - models.WorldData, 108
- miners_table
 - data_dialog.DataDialog, 81
- mines
 - models.WorldData, 108
- mines_table
 - data_dialog.DataDialog, 81
- minimumCost
 - FlowStatistics, 91
- mode_group
 - content_panel.ContentPanel, 46
- models, 24
 - Dwarf, 83
 - Mine, 104
 - Miner, 104
 - resource, 105
 - Point, 105
 - Warden, 107
 - WorldData, 108
 - miners, 108
 - mines, 108
 - to_dict, 108
- monotone_convex_hull.cpp
 - computeConvexHullMonotoneChain, 125
 - main, 125
- neutralMine
 - range_utils.hpp, 148
- on_find_loudest_dwarf
 - control_panel.ControlPanel, 68
- on_start
 - control_panel.ControlPanel, 69
- parse
 - serializer.InputParser, 99
- pathfindingTimeMs
 - FlowStatistics, 91
- pause_terminal
 - main, 23
- PLATFORM_EXECUTABLE_SUFFIX
 - execution_engine, 20
- PROCESS_FLAGS
 - runtime_initializer, 31
- queryModuloRange
 - segment_tree.cpp, 150
- queryRange
 - segment_tree.cpp, 151
- range_input
 - control_panel.ControlPanel, 71
- range_utils.hpp
 - loudnessMax, 147
 - neutralMine, 148
- RAW_DATA_DIRECTORY

- repository, 29
- readInput
 - core_utils_json.hpp, 156
- redraw_graph
 - content_panel.ContentPanel, 44
- refresh_graph_signal
 - control_panel.ControlPanel, 71
- repository, 25
 - _ensure_data_directories, 25
 - _validate_json_structure, 25
 - BASE_DIR, 29
 - DATA_DIRECTORY, 29
 - DEFAULT_DATA_FILE, 29
 - delete_data_file, 26
 - load_default_data, 26
 - load_from_file, 26
 - RAW_DATA_DIRECTORY, 29
 - save_data_to_path, 27
 - save_default_data, 27
 - save_raw_data, 28
 - to_canonical_dict, 28
- resizeEvent
 - main_window.MainWindow, 102
- resource
 - models.Miner, 105
- RESOURCES
 - data_dialog, 18
- restart
 - Stopwatch, 106
- result_label
 - control_panel.ControlPanel, 71
- reverseEdgeIndex
 - Edge, 84
- right_branch
 - huffman_codec.HuffmanNode, 95
- run_algorithm
 - control_panel.ControlPanel, 69
- run_cpp_algorithm
 - execution_engine, 19
- run_process
 - runtime_initializer, 30
- runtime_initializer, 29
 - build_application, 30
 - clear_terminal, 30
 - format, 31
 - level, 31
 - PROCESS_FLAGS, 31
 - run_process, 30
- save_btn
 - control_panel.ControlPanel, 71
- save_data_to_path
 - repository, 27
- save_default_data
 - repository, 27
- save_raw_data
 - repository, 28
- save_result
 - control_panel.ControlPanel, 69
- seg_box
 - control_panel.ControlPanel, 72
- seg_group
 - control_panel.ControlPanel, 72
- segment_tree.cpp
 - buildTree, 149
 - main, 150
 - queryModuloRange, 150
 - queryRange, 151
- serializer, 31
- serializer.InputParser, 98
 - parse, 99
- set_highlighted_mine
 - content_panel.ContentPanel, 44
- show_labels
 - content_panel.ContentPanel, 46
- solve
 - FlowOptimizer, 89
- start_btn
 - control_panel.ControlPanel, 72
- startTime
 - Stopwatch, 106
- stats
 - content_panel.ContentPanel, 46
- Stopwatch, 105
 - elapsedMilliseconds, 106
 - restart, 106
 - startTime, 106
 - Stopwatch, 106
- stylesheet, 31
 - APP_STYLESHEET, 32
- stylesheet.py, 164
- symbol
 - huffman_codec.HuffmanNode, 95
- time_label
 - control_panel.ControlPanel, 72
- to_canonical_dict
 - repository, 28
- to_dict
 - models.WorldData, 108
- toggle_labels_button
 - content_panel.ContentPanel, 46
- toolbar
 - content_panel.ContentPanel, 46
- totalPathLength
 - FlowStatistics, 91
- totalTimeMs
 - FlowStatistics, 91
- Vertex, 107
 - id, 107
 - x, 107
 - y, 107
- vertex
 - Mine, 104
- wardens_table
 - data_dialog.DataDialog, 81

weight
 huffman_codec.HuffmanNode, [95](#)
widget_effects, [32](#)
 apply_shadow, [32](#)

x
 Vertex, [107](#)

y
 Vertex, [107](#)